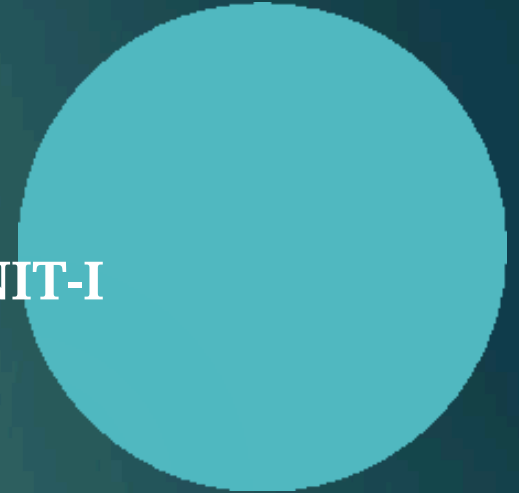


CMRIT

**SOFTWARE DESIGN AND
ENGINEERING
II YEAR B.TECH. I – SEM UNIT-I**



INTRODUCTION TO SOFTWARE ENGINEERING

Software:

- ▶ collection of integrated programs.
- ▶ carefully-organized instructions that provide desired features, function, and performance, when executed.
- ▶ Computer program and related documentation such as requirements, design models and user manuals.

Characteristics of Software:

- ▶ Software is developed or engineered; it is not manufactured in the classical sense.
- ▶ Software does not “wear out” (not susceptible to environment effects).
- ▶ Reusability of components.
- ▶ Flexibility of software.
- ▶ Maintainability of software.
- ▶ Scalability of software.

Engineering:

- ▶ application of scientific and practical knowledge to invent, design, build, maintain, and improve frameworks, processes, etc.

Software Engineering:

- ▶ The systematic, disciplined quantifiable approach to the development, operation and maintenance of software.
- ▶ includes activities to manage the project, develop tools, methods and theories that support the software production.
- ▶ provides a standard procedure to design and develop a software.

EVOLVING ROLE OF SOFTWARE:

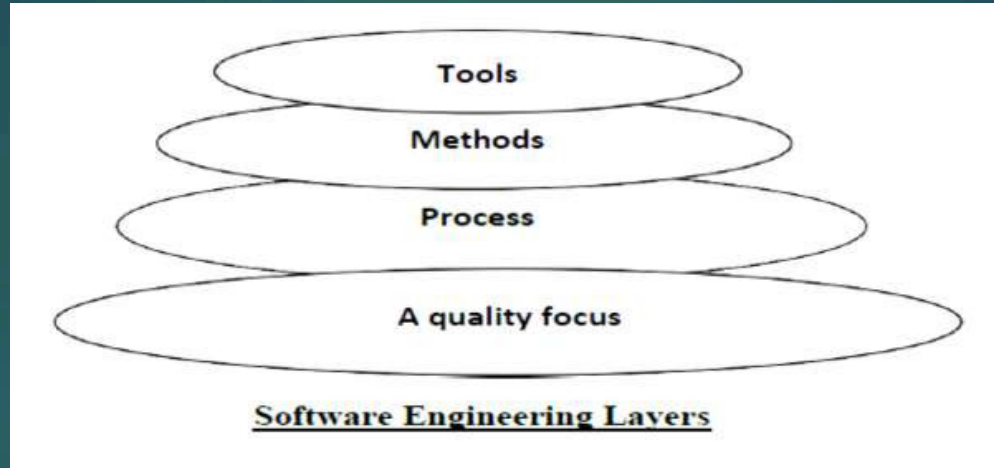
- ▶ Software takes dual role.
- ▶ **product:**
 - ▶ It delivers the computing potential embodied by computer Hardware or by a network of computers.
- ▶ **vehicle:**
 - ▶ It provides system functionality (e.g., payroll system)
 - ▶ It controls other software (e.g., an operating system)
 - ▶ It helps build other software (e.g., software tools)

Evolution:

- ▶ **1970s and 1980s**
- ▶ **1990s began**
- ▶ **Mid-1990s**
- ▶ **Later 1990s**
- ▶ **2000s progressed**
- ▶ Today a huge software industry has become a dominant factor in the economies of the industrialized world.



SOFTWARE ENGINEERING - A LAYERED TECHNOLOGY:



- ▶ The bedrock that supports software engineering is a quality focus.
- ▶ The foundation for software engineering is the process layer.
- ▶ Software engineering methods rely on a set of basic principles that govern area of the technology and include modeling activities.
- ▶ Software engineering tools provide automated or semi-automated support for the process and the methods.

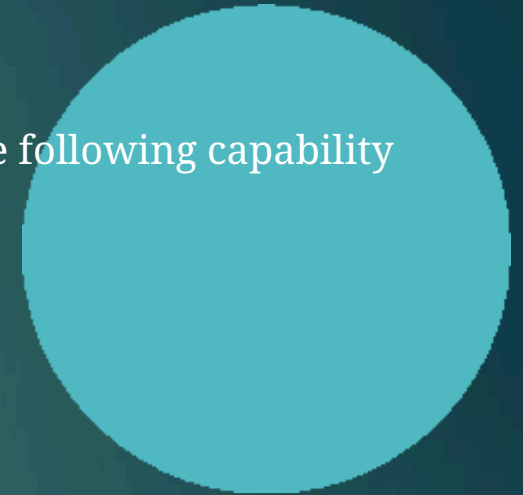
THE CAPABILITY MATURITY MODEL INTEGRATION (CMMI):

- ▶ The CMMI represents a process meta-model in two different ways:
 - ▶ As a continuous model
 - ▶ As a staged model.

Each process area is formally assessed and is rated according to the following capability levels.

- ▶ **Level 0: Incomplete**
- ▶ **Level 1: Performed**
- ▶ **Level 2: Managed**
- ▶ **Level 3: Defined**
- ▶ **Level 4: Quantitatively managed**
- ▶ **Level 5: Optimized**

The CMMI defines each process area in terms of “specific goals” and the “specific practices” required to achieve these goals.



The **specific goals (SG)** and the associated **specific practices(SP)** defined for project planning

SG 1 Establish estimates

- ▶ SP 1.1 Estimate the scope of the project
- ▶ SP 1.2 Establish estimates of work product and task attributes
- ▶ SP 1.3 Define project life cycle
- ▶ SP 1.4 Determine estimates of effort and cost

SG 2 Develop a Project Plan

- ▶ SP 2.1 Establish the budget and schedule
- ▶ SP 2.2 Identify project risks
- ▶ SP 2.3 Plan for data management
- ▶ SP 2.4 Plan for needed knowledge and skills
- ▶ SP 2.5 Plan stakeholder involvement
- ▶ SP 2.6 Establish the project plan

SG 3 Obtain commitment to the plan

- ▶ SP 3.1 Review plans that affect the project
- ▶ SP 3.2 Reconcile work and resource levels
- ▶ SP 3.3 Obtain plan commitment



- ▶ In addition to specific goals and practices, the CMMI also defines a set of five **generic goals** and **related practices** for each process area.
- ▶ Each of the five generic goals corresponds to one of the five capability levels.

GG 1 Achieve specific goals

- ▶ GP 1.1 Perform base practices

GG 2 Institutionalize a managed process

- ▶ GP 2.1 Establish and organizational policy
- ▶ GP 2.2 Plan the process
- ▶ GP 2.3 Provide resources
- ▶ GP 2.4 Assign responsibility
- ▶ GP 2.5 Train people
- ▶ GP 2.6 Manage configurations
- ▶ GP 2.7 Identify and involve relevant stakeholders
- ▶ GP 2.8 Monitor and control the process
- ▶ GP 2.9 Objectively evaluate adherence
- ▶ GP 2.10 Review status with higher level management



GG 3 Institutionalize a defined process

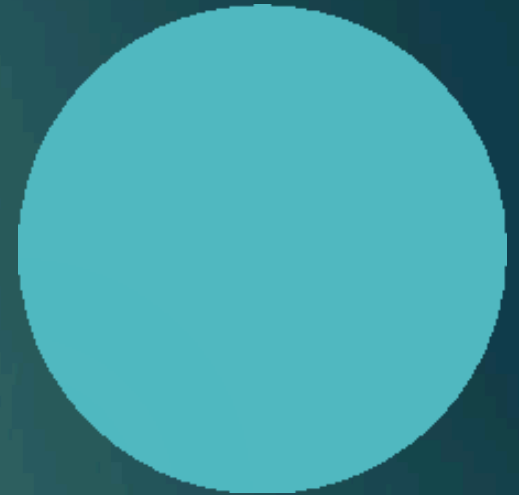
- ▶ GP 3.1 Establish a defined process
- ▶ GP 3.2 Collect improvement information

GG 4 Institutionalize a quantitatively managed process

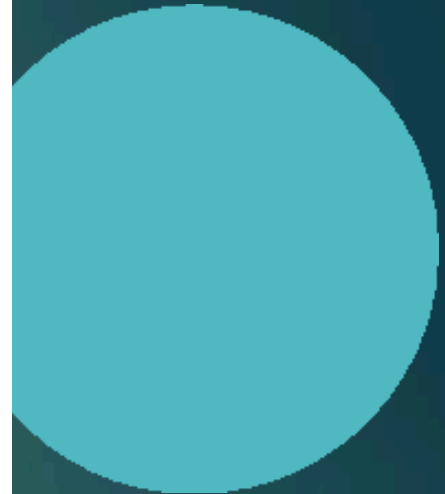
- ▶ GP 4.1 Establish quantitative objectives for the process
- ▶ GP 4.2 Stabilize sub process performance

GG 5 Institutionalize and optimizing process

- ▶ GP 5.1 Ensure continuous process improvement
- ▶ GP 5.2 Correct root causes of problems

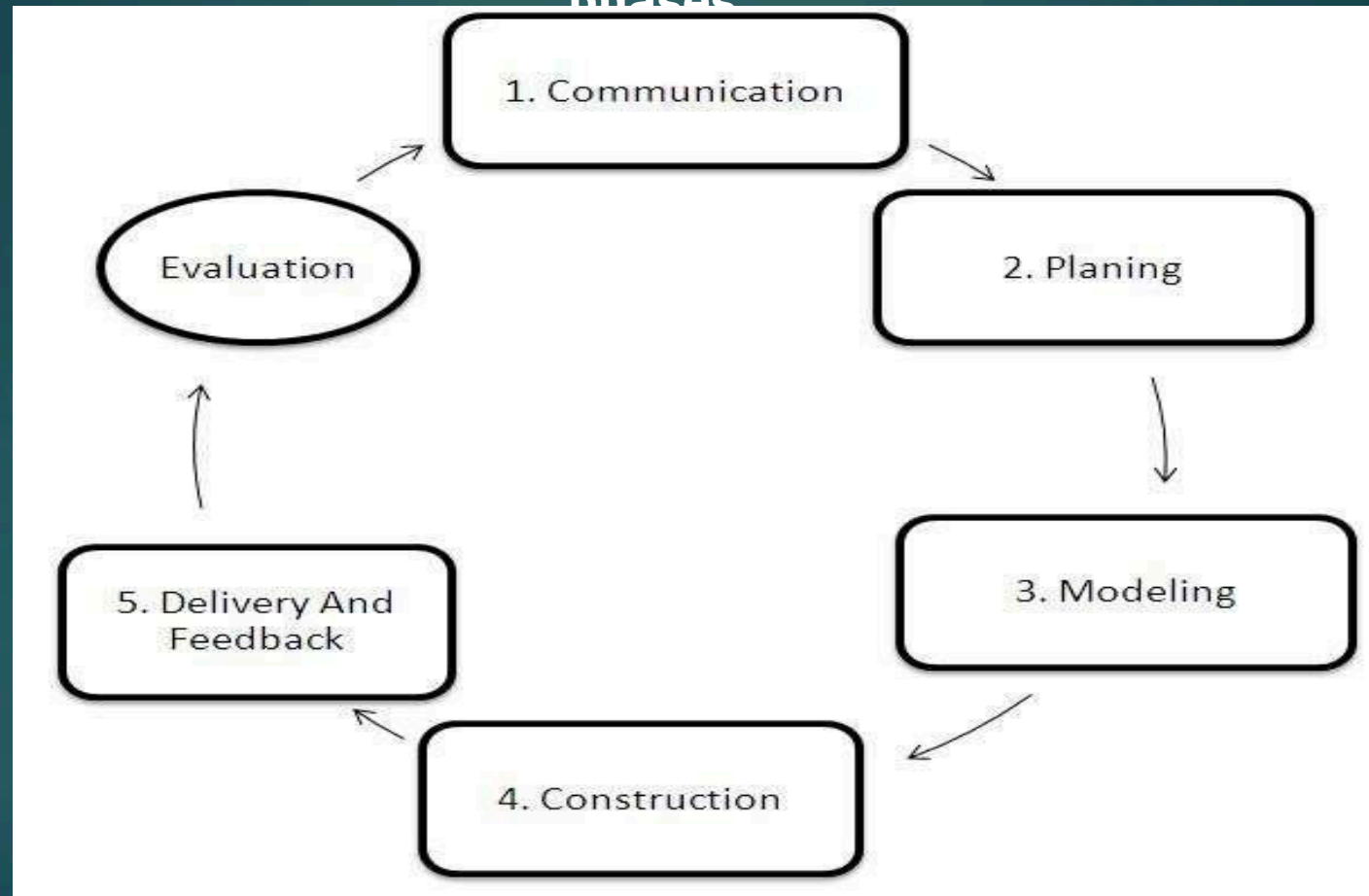


- ering.



- ▶ Number of different approaches to software process assessment have been proposed ,
- ▶ Standards CMMI Assessment Method for Process Improvement (SCAMPI)
 - ▶ CMM Based Appraisal for Internal Process Improvement (CBA IPI)
 - ▶ SPICE (ISO/IEC15504)
 - ▶ ISO 9001:2000 for Software.

SDLC phases



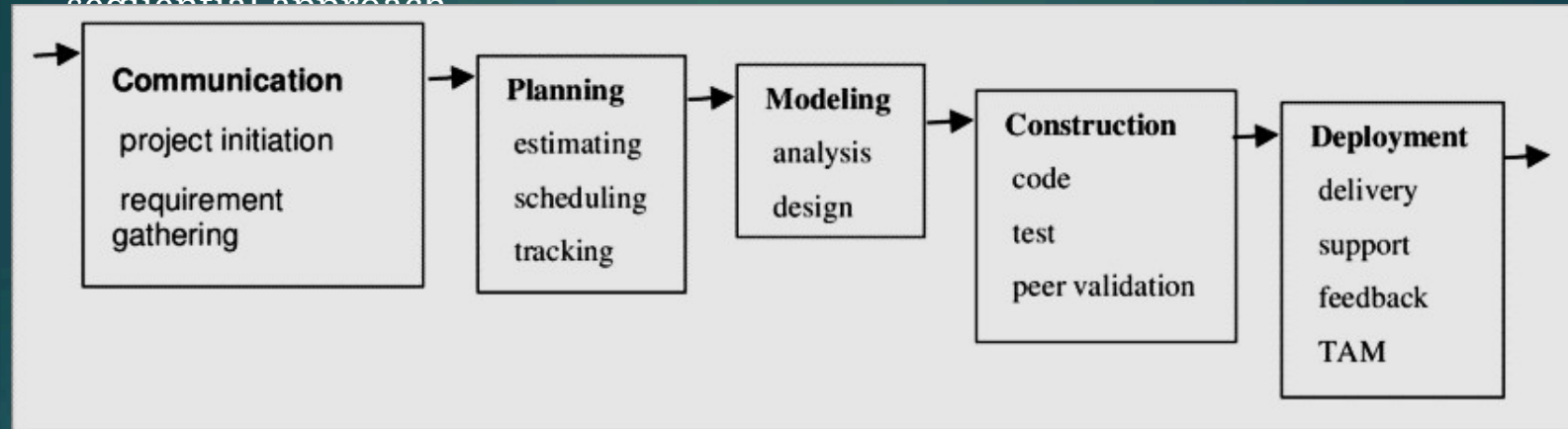
PROCESS MODELS

1. Waterfall model
2. Incremental process model
 1. The Incremental Model
 2. RAD Model
 3. Evolutionary Process Model
 3. Prototyping
 4. Spiral Model
 5. Concurrent Development Model
4. Specialized Process Models
 2. Component Based Development
 3. The Formal Method Model
 4. Aspect – Oriented Software Development
5. The Unified Process
 2. Brief History
 3. Phases of Unified Process
 4. unified Process Work Products



THE WATERFALL

MODEL: Sometimes called the *classic life cycle*, suggests a systematic sequential approach



Advantages:

- ▶ useful process model where requirements are fixed and work is to proceed to complete in a linear manner.
- ▶ Simple and easy to understand.
- ▶ Phases do not overlap
- ▶ Used for small projects

Disadvantag

es] changes can cause confusion as the project team proceeds.

- It is often difficult for the customer to state all requirements explicitly.
- A working version of the programs will not be available until late in the project time-span.
- Testing done at the end.
- Not suitable for long object oriented projects and complex projects.
- Fixing bugs can cause lot of expenses.

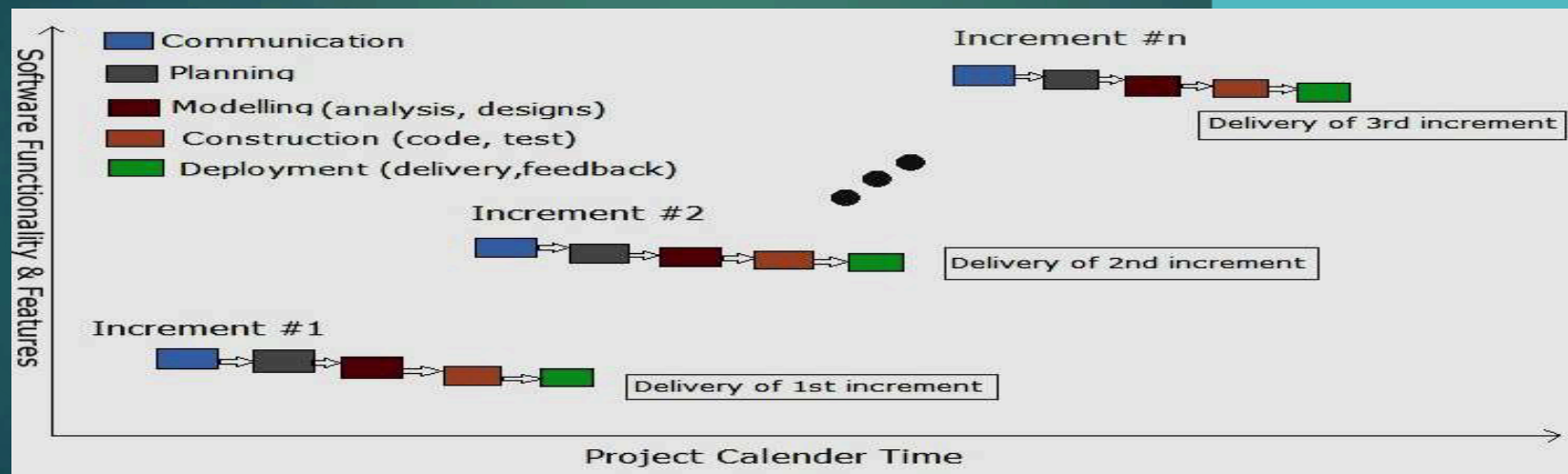
INCREMENTAL PROCESS

Models: The drawback of waterfall model is overcome by this model.

- ▶ The incremental model
- ▶ The RAD model

THE INCREMENTAL MODEL:

- ▶ Incremental development is particularly useful when staffing is unavailable for a complete implementation by the business deadline that has been established for the project.



Advantage

S :

- It is easier to test and debug during a smaller iteration.
- In this model customer can respond to each built.
- Generates working software quickly and early during the software life cycle.

Disadvantages:

- Needs good planning and design.
- Needs a clear and complete definition of the whole system before it can be built incrementally.
- Total cost is higher than waterfall.

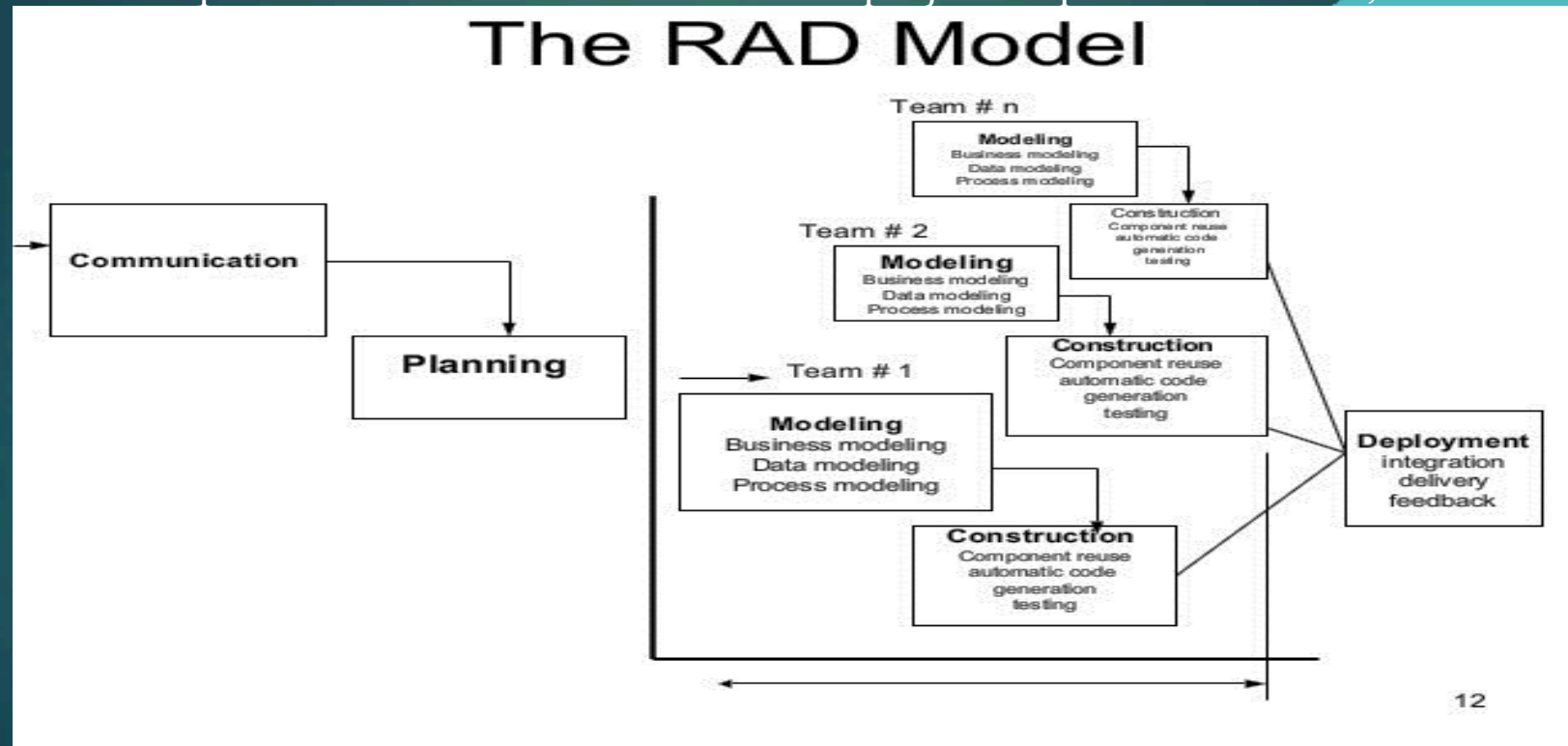


broken down
and

THE RAD

MODEL: a “high-speed” adaption of the waterfall model, in which rapid development is achieved by using a component base construction approach.

- If requirements are well understood and project scope is constrained, the RAD



Drawback

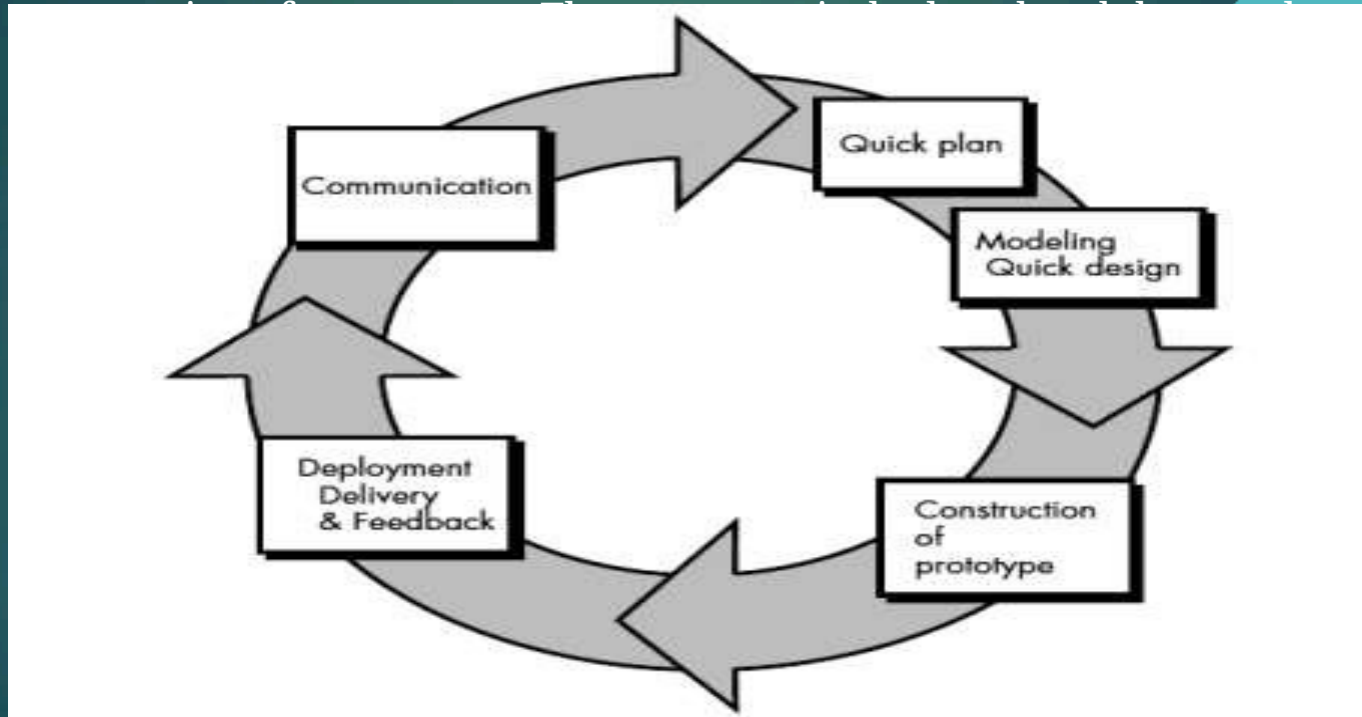
s :

- It can't be used for smaller projects.
- When technical risk is high, it is not suitable.
- If developers are not committed to delivering software on time, RAD projects can fail.
- If the system cannot be modularized properly, building the components necessary for RAD will be problematic.

EVOLUTIONARY PROCESS MODELS:

PROTOTYPING:

- ▶ can be implemented within the context of anyone of the process model.
- ▶ Prototyping iteration is planned quickly and modeling occurs. The quick design leads to the



Advantages:

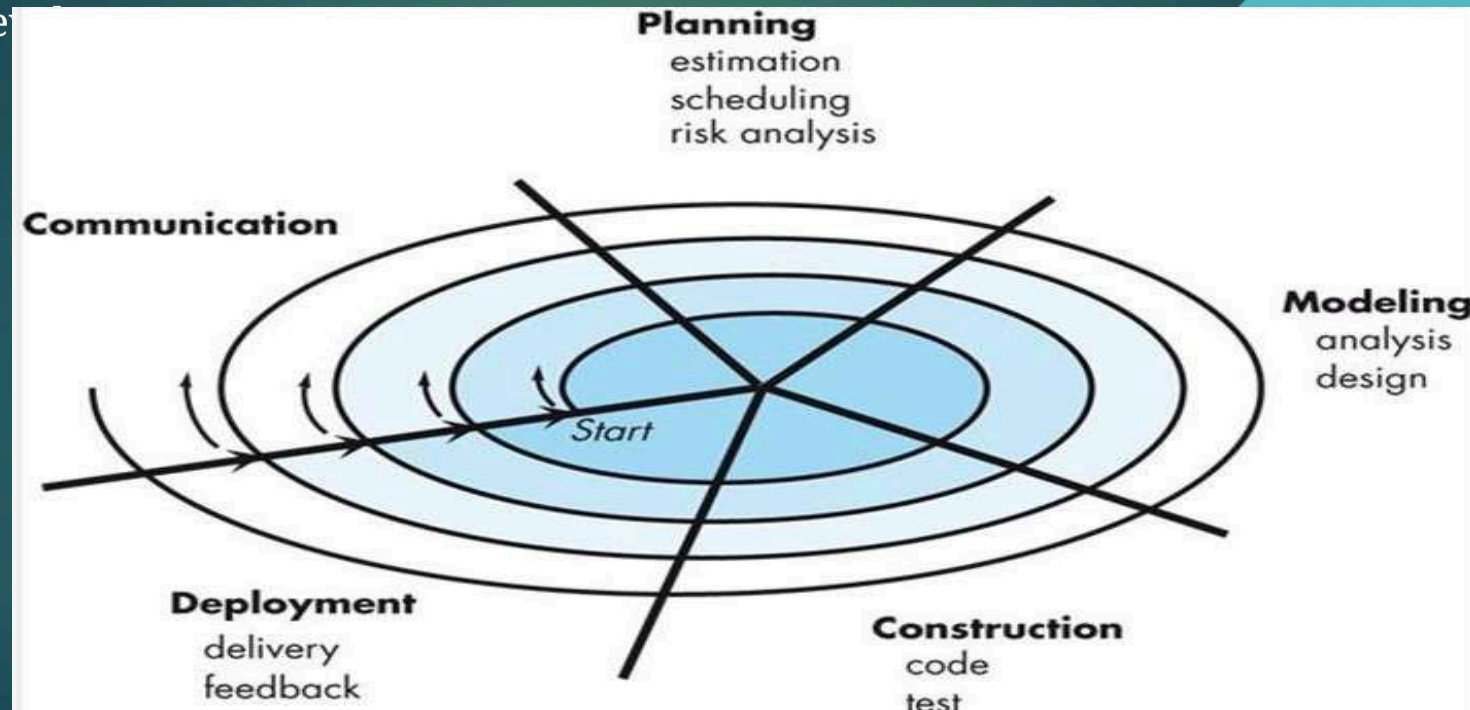
- New requirements can be easily accommodated.
- It ensures a great level of customers satisfaction and comfort.
- It helps developers and users both understand the system better.
- It serves as a mechanism for identifying requirements.

Disadvantages:

- If not managed properly iterative process continuous for long time.
- Poor documentation because requirements are changing continuously.
- There may be too much variations in requirements.
- Unaware that in the rush to get it working we have not considered overall software quality or long term maintainability.
- Developers often compromise on implementation.

THE SPIRAL MODEL

- ▶ Couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.
- ▶ Adapted to apply throughout the entire life cycle of an application, from concept development to deployment.



Advantag

es:

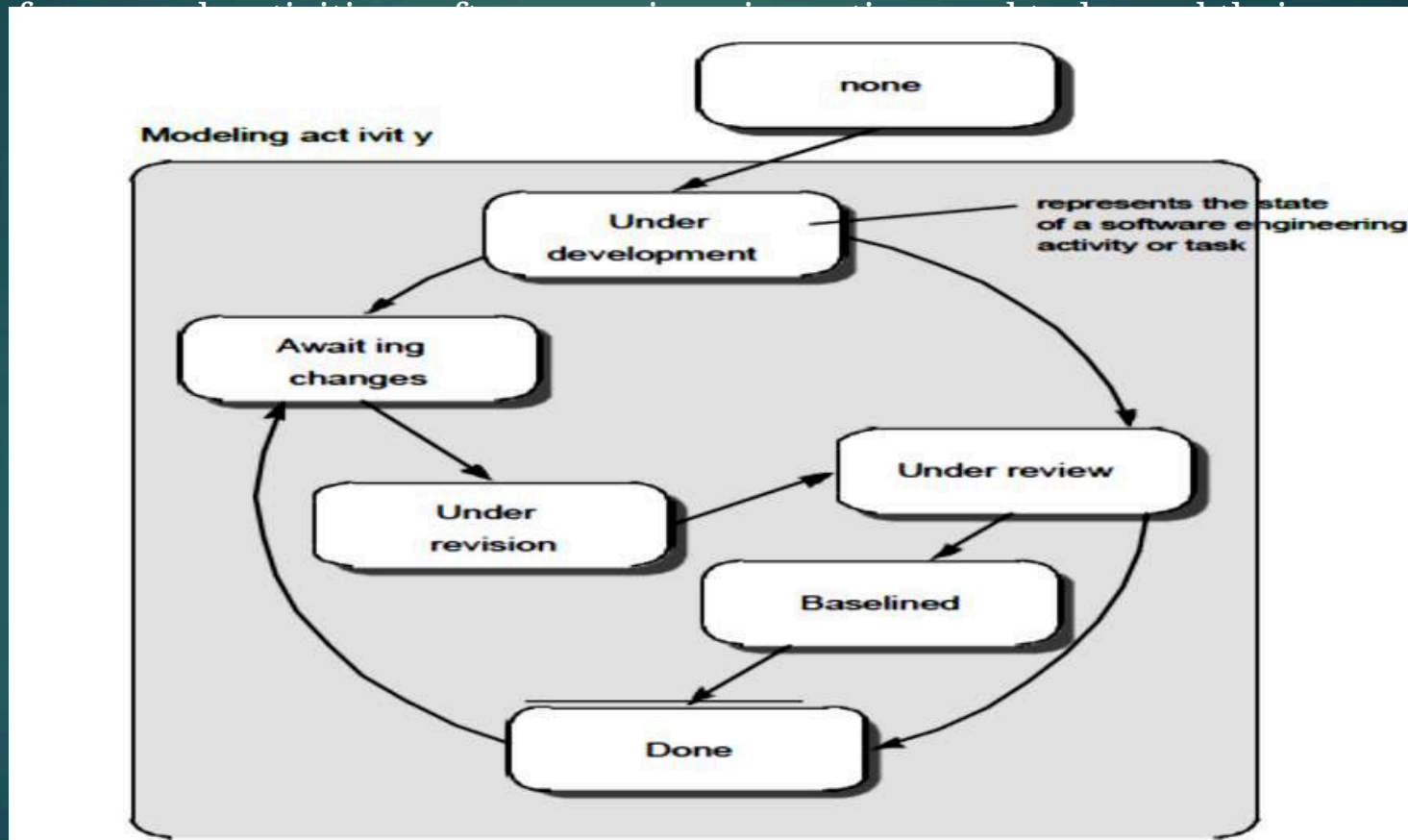
- It is realistic approach to the development of large scale systems and soft wares.
- Provides easily and frequent feedbacks of the customer, additional functionalities can be added at later.
- Continuous involvement of the customer so more satisfaction to the customer.
- Suitable for high risk projects where business needs may be unstable.
- Risk handling (Risk analysis and Risk handling) is the major advantage.

Disadvantages:

- Complex to use.
- Risk analysis requires highly skilled experts.
- Not suitable for low risk and small projects.

THE CONCURRENT DEVELOPMENT MODEL:

- sometimes called *concurrent engineering*, can be represented schematically as a series of

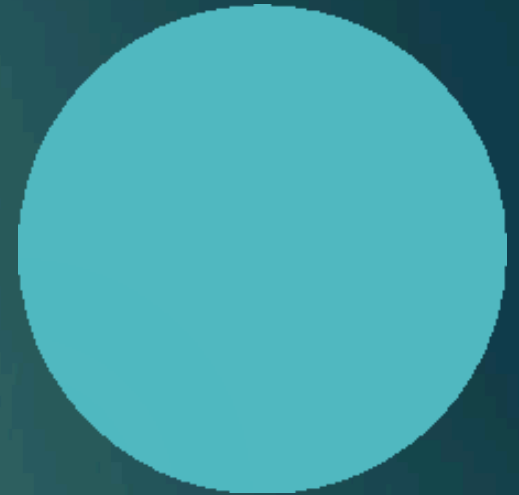


Advantages:

- Applicable to all types of software development.
- Provides an accurate picture of the current state of a project.
- It defines network of activities.

Disadvantages:

- There is a high amounts of risk and un certainty.
- No working software is produced till late in the life cycle.



THE UNIFIED PROCESS:

- ▶ Implements many of the best principles of agile software development.
- ▶ Recognizes the importance of customer communication and streamlined methods for describing the customer's view of a system.

HISTORY

- During 1980's and early 1990's
- During early 1990's (James Rumbaugh, Grady Booch and Ivar Jacobson)
- In 1997 UML

PHASES OF THE UNIFIED PROCESS:

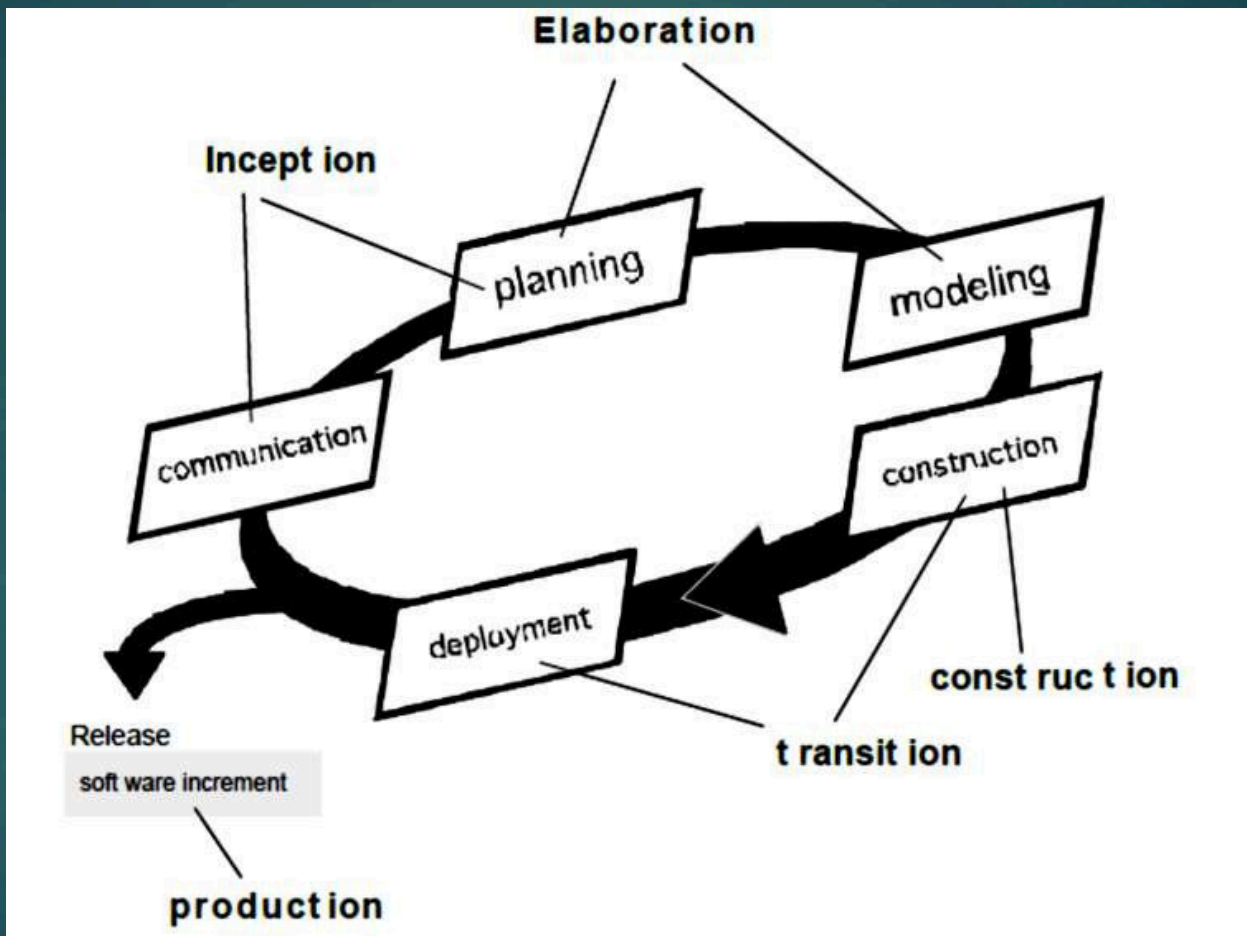
The ***inception*** phase - encompasses both customer communication and planning activities.

The ***elaboration*** phase - encompasses the customer communication and modeling activities of the generic process model.

The ***construction*** phase - identical to the construction activity defined for the generic software process.

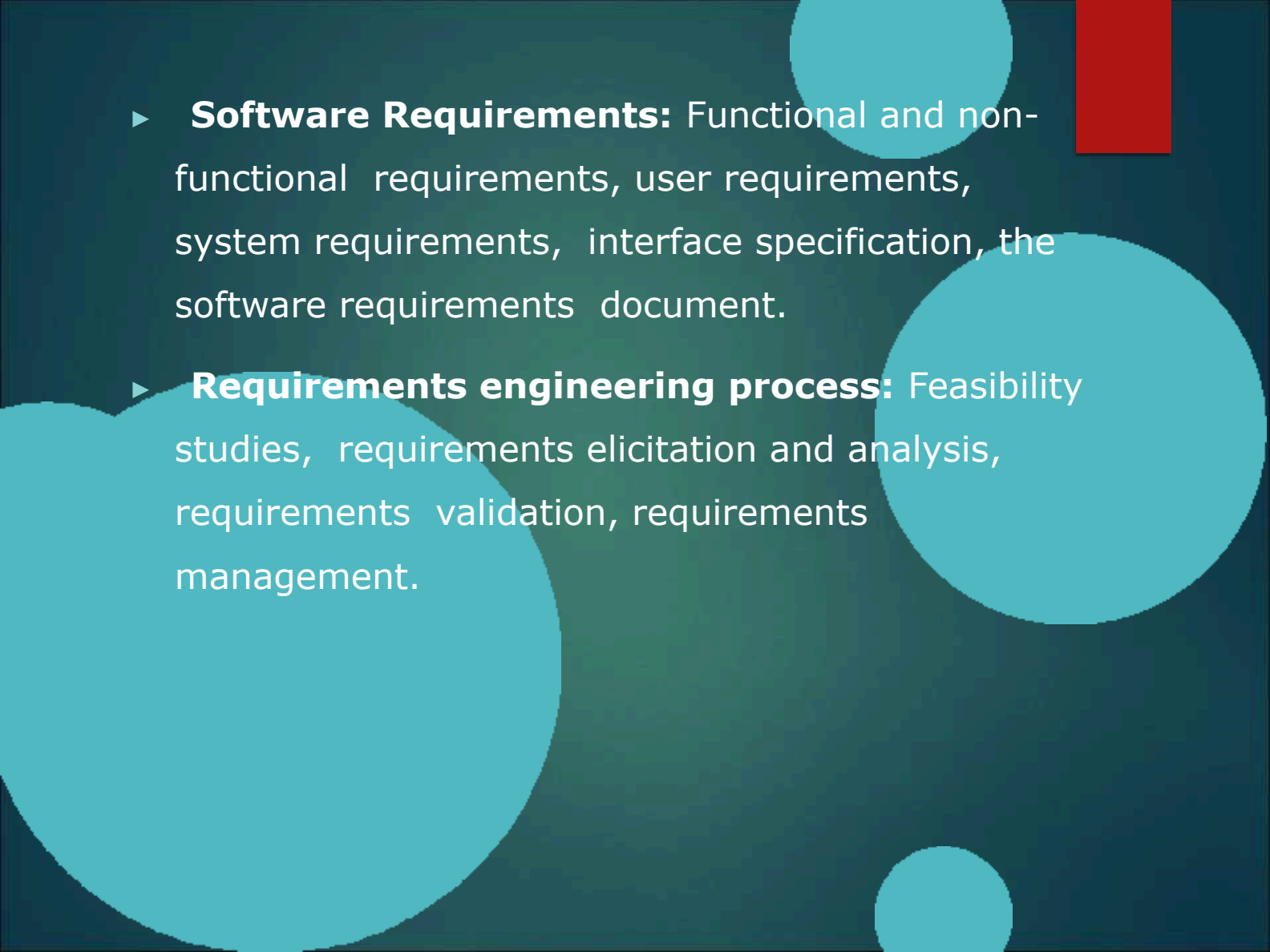
The ***transition*** phase - encompasses the latter stages of the generic construction activity and the first part of the generic deployment activity.

The ***production*** phase - coincides with the deployment activity of the generic process.



UNIFIED PROCESS WORK PRODUCTS

- **INCEPTION PHASE** : Use case Model (10% to 20 %)
- **ELABORATION PHASE** : Set of work products. (80 to 90 % of model has been created.) Risk and best project plans.
- **MODELING PHASE** : Work product is developed.
- **CONSTRUCTION PHASE** : translates design classes into software components into physical computing environment.
- **TRANSITION PHASE** : Feed back from the beta testing.

- 
- ▶ **Software Requirements:** Functional and non-functional requirements, user requirements, system requirements, interface specification, the software requirements document.
 - ▶ **Requirements engineering process:** Feasibility studies, requirements elicitation and analysis, requirements validation, requirements management.

SOFTWARE REQUIREMENTS

Software requirements are necessary

- ▶ To introduce the concepts of user and system requirements
- ▶ To describe functional and non-functional requirements
- ▶ To explain how software requirements may be organized

What is a requirement?

- ▶ The description of the services provided by the system and its operational constraints
- ▶ It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification

Requirements engineering:

- ▶ The process of finding out, analysing documenting and checking these services and constraints.
- ▶ The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process

Types of requirement:

User requirements

- ▶ Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

System requirements

- ▶ A structured document setting out detailed descriptions of the system's functions, services and operational constraints.
- ▶ Defines what should be implemented so may be part of a contract between client and contractor.

Functional and non-functional requirements:

► Functional requirements

- Statements of services the system should provide how the system should react to particular inputs and how the system should behave in particular situations.

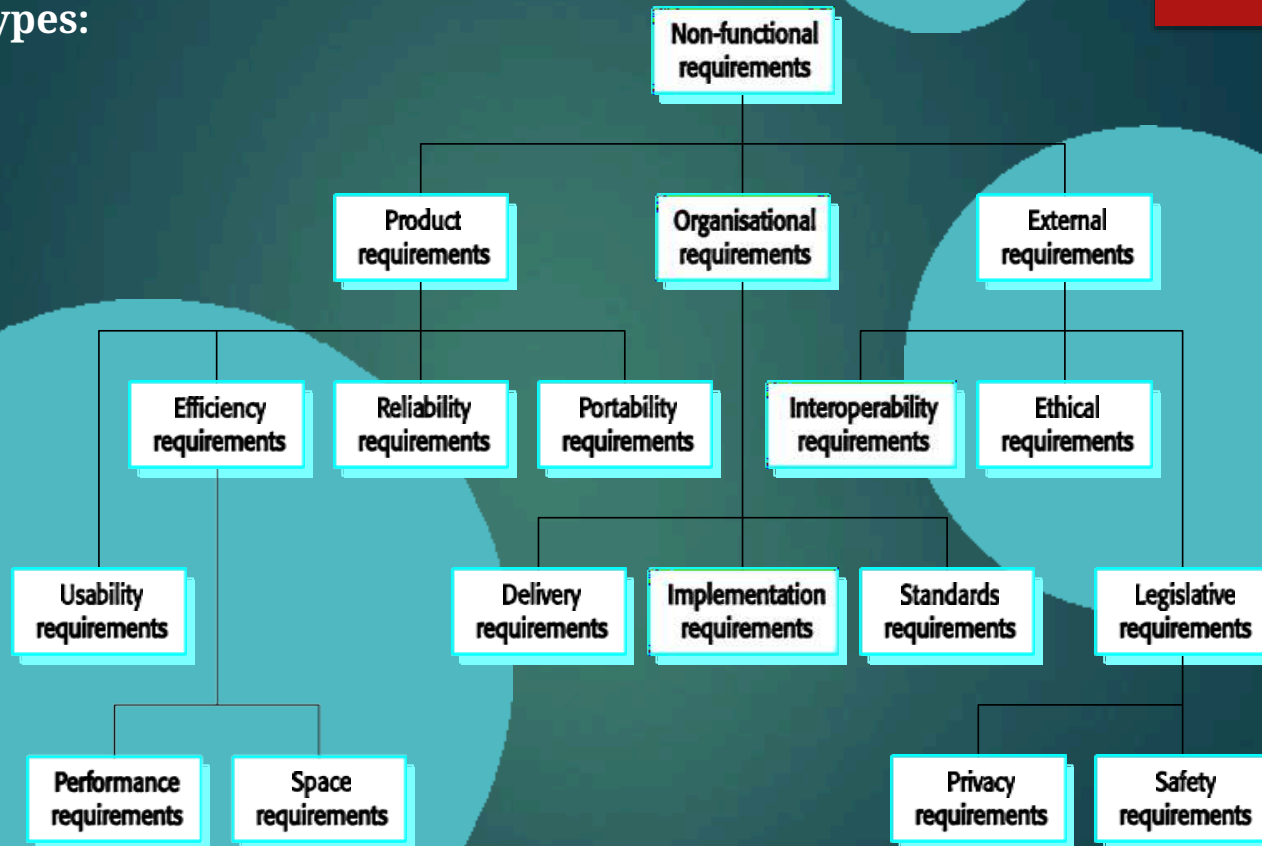
Examples of functional requirements

- The user shall be able to search either all of the initial set of databases or select a subset from it.
- The system shall provide appropriate viewers for the user to read documents in the document store.
- Every order shall be allocated a unique identifier (ORDER_ID) which the user shall be able to copy to the account's permanent storage area.

► Non-functional requirements

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, reliability, storage requirements and standards, etc.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system is useless.

Non-functional requirement types:



Requirements

measures

Property	Measure
Speed	Processed transactions/second User/ Event response time Screen refresh time
Size	M Bytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

USER REQUIREMENTS

User requirements are defined using natural language, tables and diagrams as these can be understood by all users.

Problems with natural language

- ▶ Lack of clarity
 - ▶ Precision is difficult without making the document difficult to read.
- ▶ Requirements confusion
 - ▶ Functional and non-functional requirements tend to be mixed-up.
- ▶ Requirements amalgamation
 - ▶ Several different requirements may be expressed together.

▶ Guidelines for writing requirements

- ▶ Invent a standard format and use it for all requirements.
- ▶ Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- ▶ Use text highlighting to identify key parts of the requirement.

▶

SYSTEM REQUIREMENTS

- ▶ More detailed specifications of system functions, services and constraints than user requirements.
- ▶ They are intended to be a basis for designing the system.
- ▶ They may be incorporated into the system contract.
- ▶ System requirements may be defined or illustrated using system models.

Problems with NL (natural language) specification

- ▶ Ambiguity
 - ▶ The readers and writers of the requirement must interpret the same words in the same way. NL is naturally ambiguous so this is very difficult.
- ▶ Over-flexibility
 - ▶ The same thing may be said in a number of different ways in the specification.
- ▶ Lack of modularization.
 - ▶ NL structures are inadequate to structure system requirements.

Alternatives to NL specification:

► **Structured language specifications**

- All requirements are written in a standard way.
- The terminology used in the description may be limited.
- The advantage is that the most of the expressiveness of natural language is maintained but a degree of uniformity is imposed on the specification.

► **Form-based specifications**

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Indication of other entities required.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function.

- ▶ **Tabular specification**

- ▶ Used to supplement natural language.
- ▶ Particularly useful when you have to define a number of possible alternative courses of action.

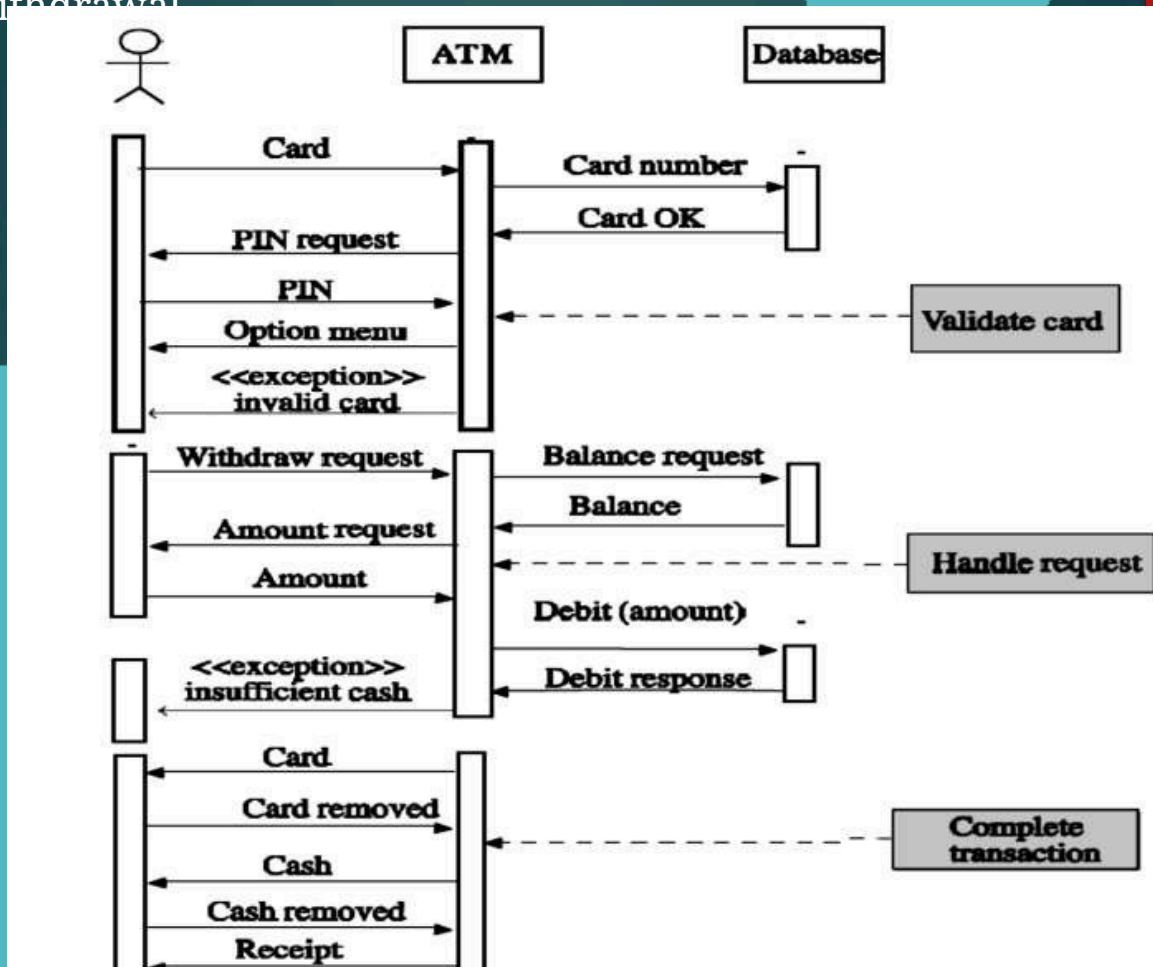
- ▶ **Graphical models**

- ▶ Graphical models are most useful when you need to show how state changes or where you need to describe a sequence of actions.

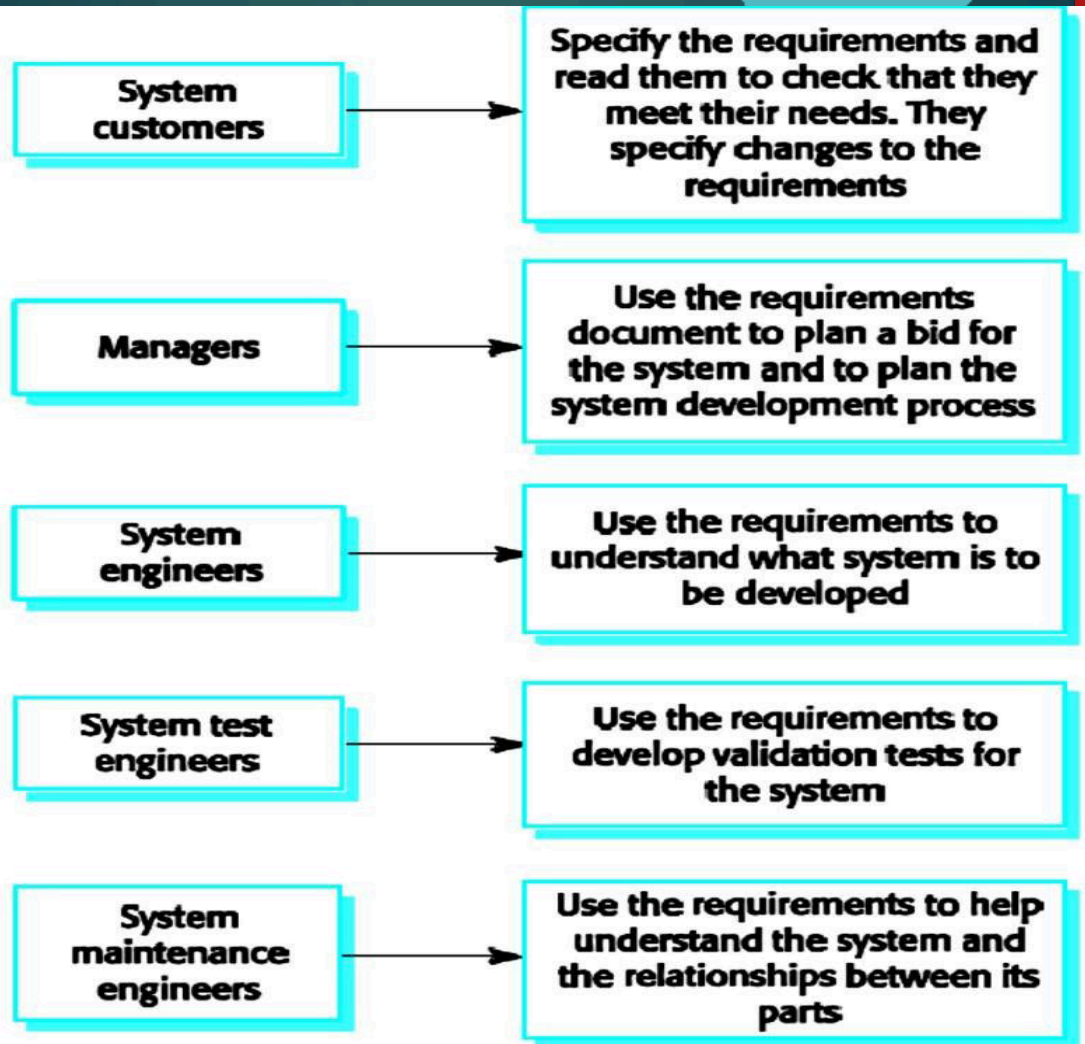
- ▶ **Sequence diagrams**

- ▶ These show the sequence of events that take place during some user interaction with a system.
- ▶ You read them from top to bottom to see the order of the actions that take place.
- ▶ Cash withdrawal from an ATM
 - ▶ Validate card;
 - ▶ Handle request;
 - ▶ Complete transaction.

► Sequence diagram of ATM withdrawal



THE SOFTWARE REQUIREMENTS DOCUMENT



- **IEEE requirements standard** defines a generic structure for a requirements document that must be instantiated for each specific system.

1. Introduction.

i) Purpose of the requirements document

iii) Definitions, acronyms and abbreviations

ii) Scope of the project

iv) References

v) Overview of the remainder of the document
i) Product perspective

ii) Product functions

2. General description.

iii) User characteristics
vi) Assumptions and dependencies

iv) General constraints

- 3. **Specific requirements** cover functional and non-functional requirements. The requirements may document external interfaces, describe system functionality and performance, specify logical database requirements, design constraints, emergent system properties and quality characteristics.

4. Appendices.

5. Index.

REQUIREMENTS ENGINEERING PROCESS

▶ To create and maintain a system requirement document

▶ The overall process includes four high level requirements engineering sub-processes:

▶ 1. Feasibility study

--Concerned with assessing whether the system is useful to the business

▶ 2. Elicitation and analysis

--Discovering requirements

▶ 3. Specifications

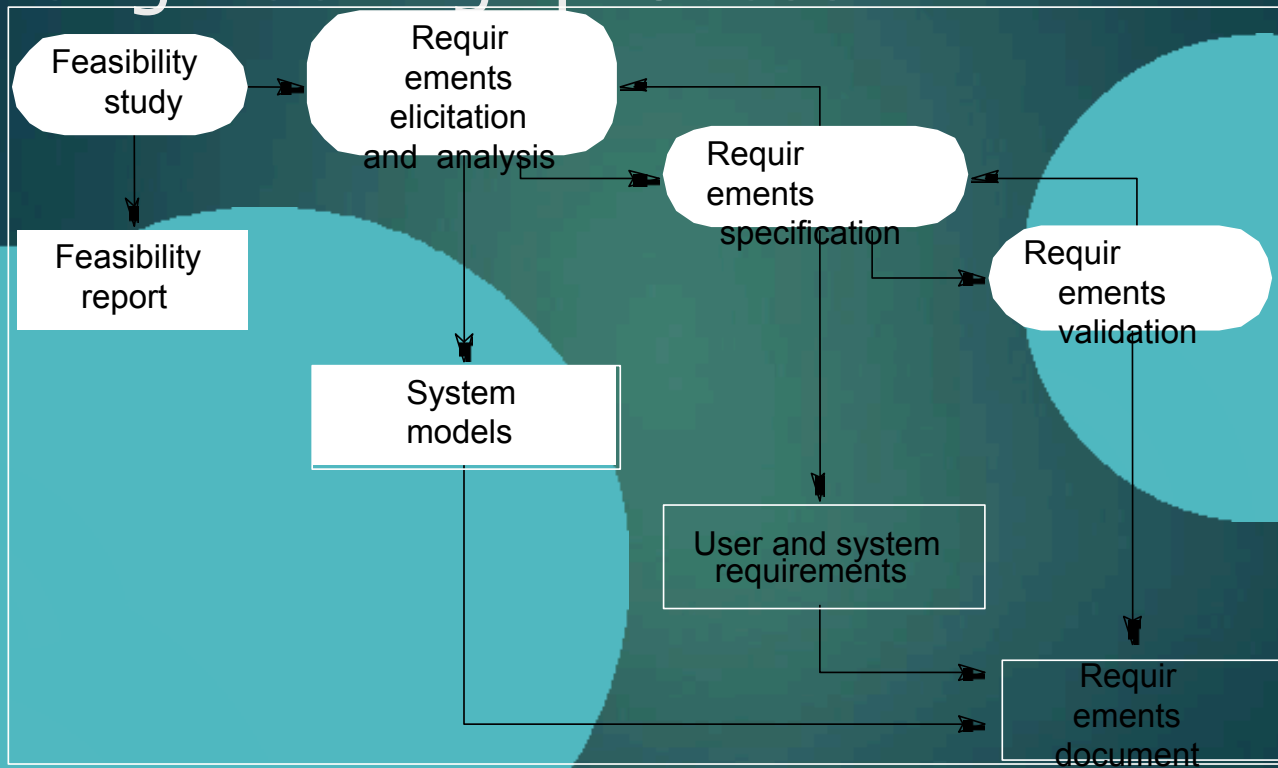
--Converting the requirements into a standard form

▶ 4. Validation
--Checking that the requirements actually define the system that the customer wants

□ 5. Requirements Management

--Managing the changes in the requirements.

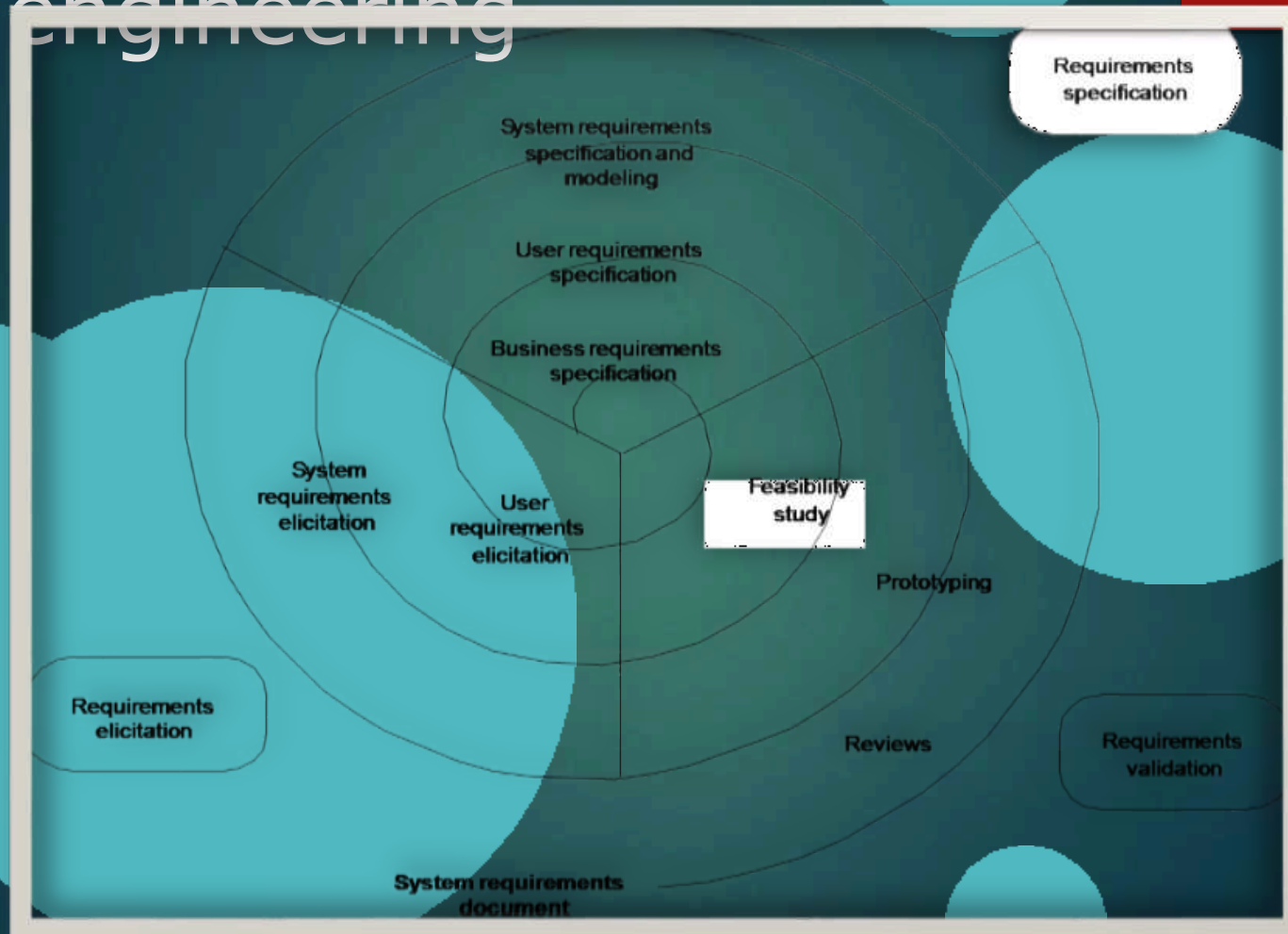
The requirements engineering process



SPIRAL REPRESENTATION OF REQUIREMENTS ENGINEERING PROCESS

- ▶ Process represented as three stage activity
- ▶ Activities are organized as an iterative process around a spiral.
- ▶ Early in the process, most effort will be spent on understanding high-level business and the user requirement.
- ▶ Later in the outer rings, more effort will be devoted to system requirements engineering and system modeling
- ▶ Three level process consists of:
 - ▶ 1.Requirements elicitation
 - ▶ 2. Requirements specification
 - ▶ 3. Requirements validation

Requirements engineering



FEASIBILITY STUDIES

- ▶ Starting point of the requirements engineering process
- ▶ Input: Set of preliminary business requirements, an outline description of the system and how the system is intended to support business processes
- ▶ Output: Feasibility report that recommends whether or not it is worth carrying out further
- ▶ Feasibility report answers a number of questions:
 - ▶ 1.Does the system contribute to the overall objective
 - ▶ 2.Can the system be implemented using the current technology and within given cost and schedule
 - ▶ 2.Can the system be integrated with other system which are already in place.

REQUIREMENTS ELICITATION

ANALYSIS involves a number of people in an organization.

- ▶ Stakeholder definition

Refers to any person or group who will be affected by the system directly or indirectly i.e. End users, Engineers, business managers, domain experts.

- ▶ Reasons why eliciting is difficult

- ▶ 1.Stakeholder often don't know what they want from the computer system.

- ▶ 2.Stakeholder expression of requirements in natural language is sometimes difficult to understand.

- ▶ 3.Different stakeholders express requirements differently .

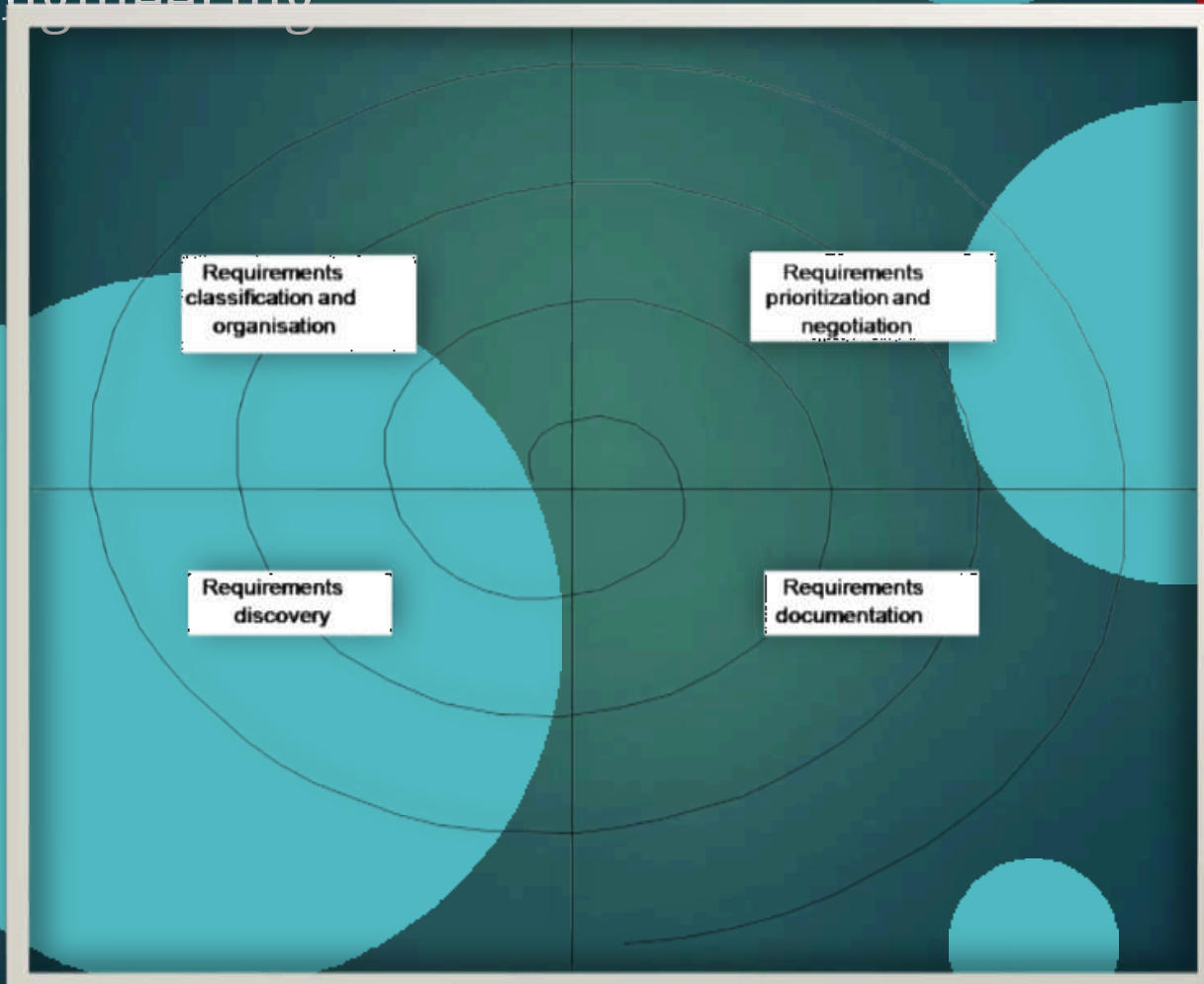
- ▶ Change in requirements due to dynamic environments.

REQUIREMENTS ELICITATION PROCESS

Process activities

- ▶ 1.Requirement Discovery
 - ▶ Interaction with stakeholder to collect their requirements including domain and documentation
- ▶ 2.Requirements classification and organization
 - ▶ Coherent clustering of requirements from unstructured collection of requirements
- ▶ 3. Requirements prioritization and negotiation
 - ▶ Assigning priority to requirements
 - ▶ Resolves conflicting requirements through negotiation
- ▶ 4. Requirements documentation
 - ▶ Requirements be documented and placed in the next round of spiral

The spiral representation of Requirements Engineering



REQUIREMENTS DISCOVERY TECHNIQUES

1. Viewpoints

- Based on the viewpoints expressed by the stake holder

- Recognizes multiple perspectives and provides a framework for discovering conflicts in the requirements proposed by different stakeholders

Three Generic types of viewpoints 1. Interactor viewpoint

- Represents people or other system that interact directly with the system

2. Indirect viewpoint

- Stakeholders who influence the requirements, but don't use the system

3. Domain viewpoint

- Requirements domain characteristics and constraints that influence the requirements.

REQUIREMENTS DISCOVERY TECHNIQUES

2.

Interviewing

g --Puts questions to stakeholders about the system to be developed.

--Requirements are derived from the answers

that they use
and the

❖ Two types of interview

where the stakeholders answer a pre-defined set of questions.

► Open interviews :-

discuss a range of issues with the stakeholders for better understanding their needs.

REQUIREMENTS DISCOVERY TECHNIQUES

3. Scenarios

- Easier to relate to real life examples than to abstract description

- Starts with an outline of the interaction and during elicitation, details are added to create a complete description of that interaction

- Scenario includes:

1. Description at the start of the scenario
2. Description of normal flow of the event
3. Description of what can go wrong and how this is handled
4. Information about other activities parallel to the scenario
5. Description of the system state when the scenario finishes

LIBSYS

Scenario

- ▶ **Initial assumption:** The user has logged on to the LIBSYS system and has located the journal containing the copy of the article.
- ▶ **Normal:** The user selects the article to be copied. He or she is then prompted by the system to either provide subscriber information for the journal or to indicate how they will pay for the article. Alternative payment methods are by credit card or by quoting an organisational account number.
- ▶ The user is then asked to fill in a copyright form that maintains details of the transaction and they then submit this to the LIBSYS system.
- ▶ The copyright form is checked and, if OK, the PDF version of the article is downloaded to the LIBSYS working area on the user's computer and the user is informed that it is available. The user is asked to select a printer and a copy of the article is printed

LIBSYS

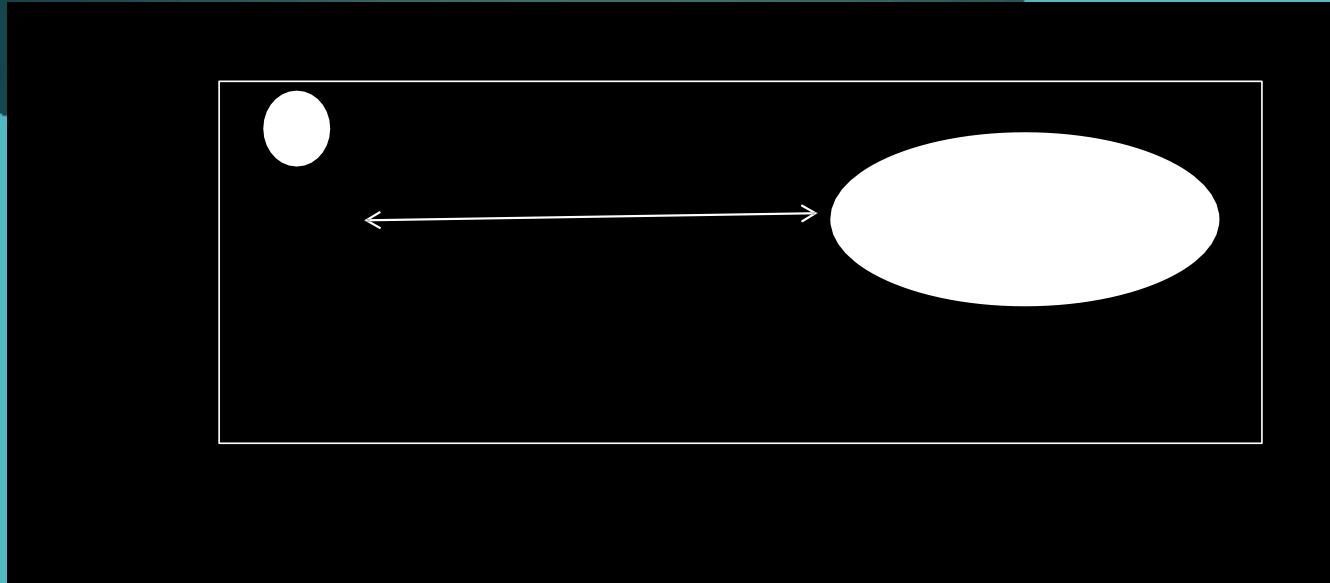
- ▶ **What can go wrong:** The user may fail to fill in the copyright form correctly. In this case, the form should be re-presented to the user for correction. If the resubmitted form is still incorrect then the user's request for the article is rejected.
- ▶ The payment may be rejected by the system. The user's request for the article is rejected.
- ▶ The article download may fail. Retry until successful or the user terminates the session..
- ▶ **Other activities:** Simultaneous downloads of other articles.
- ▶ **System state on completion:** User is logged on. The downloaded article has been deleted from LIBSYS workspace if it has been flagged as print- only.

REQUIREMENTS DISCOVERY TECHNIQUES

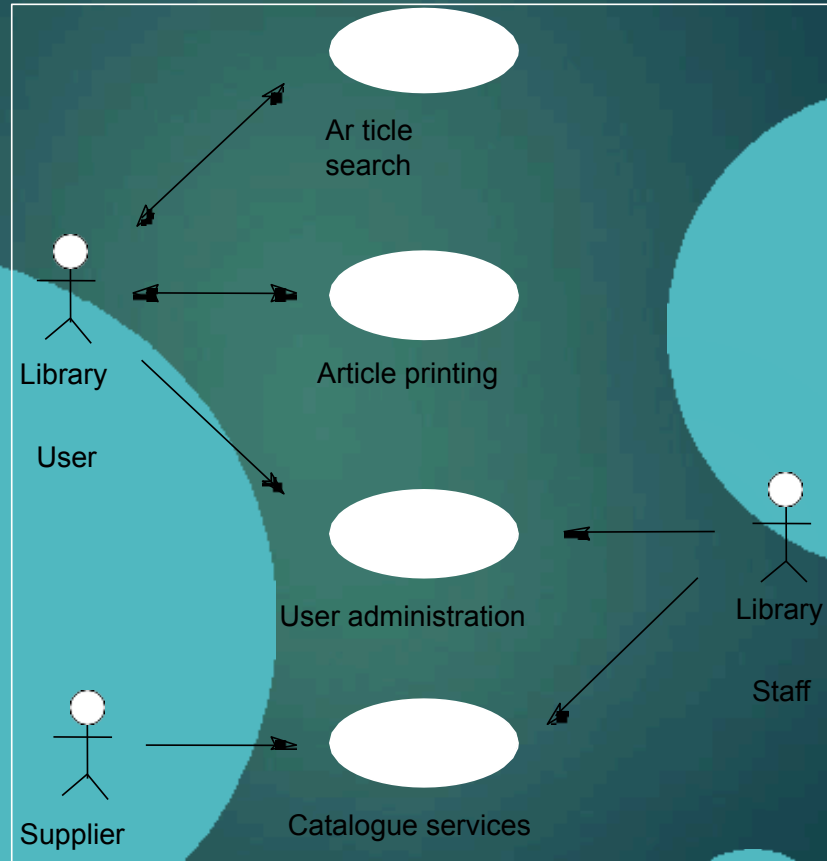
4. Use cases

- scenario based technique for requirement elicitation
- A fundamental feature of UML, notation for describing object-oriented system models
- Identifies a type of interaction and the actors involved
- Sequence diagrams are used to add information to a Use case

Article printing use-case



LIBSYS use cases



REQUIREMENTS DISCOVERY TECHNIQUES

30

5.Ethnography:

...social scientists spend a considerable time for observing and analyzing how people actually work.
....People don't have to explain or articulate their work.

Scope of Ethnography:

Requirements derived from cooperation and awareness of other peoples's activities.

REQUIREMENTS

VALIDATION

Concerned with showing that the requirements define the system that the customer wants

- ▶ Important because errors in requirements can lead to extensive rework cost
- ▶ Validation checks
 1. Validity checks
 - Verification that the system performs the intended function by the user
 2. Consistency check
 - Requirements should not conflict
 3. Completeness checks
 - Includes requirements which define all functions and intended by the system user
 4. Realism checks
 - Ensures that the requirements can be actually implemented
 5. Verifiability
 - Testable to avoid disputes between customer and developer.

constraints

VALIDATION TECHNIQUES

1. REQUIREMENTS REVIEWS

--Reviewers check the following:

- (a) Verifiability: Testable
- (b) Comprehensibility: understood
- (c) Traceability: origin of the requirement clearly stated
- (d) Adaptability: requirement can be changed

2. PROTOTYPING

3. TEST-CASE GENERATION

Requirements management

Requirements are likely to change for large software systems and as such requirements management process is required to handle changes.

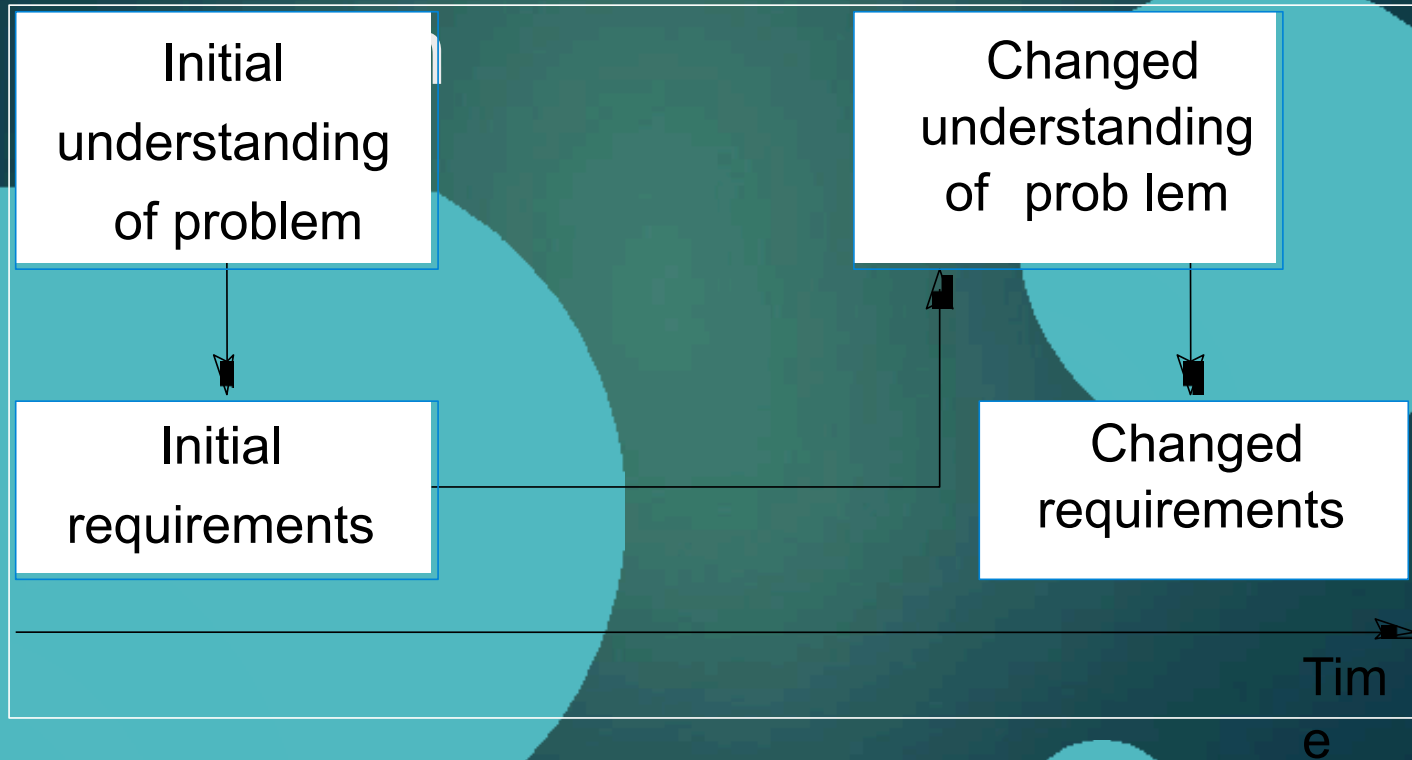
Reasons for requirements changes

- (a) Diverse Users community where users have different requirements and priorities
- (b) System customers and end users are different
- (c) Change in the business and technical environment after installation

Two classes of requirements

- (a) Enduring requirements: Relatively stable requirements
- (b) Volatile requirements: Likely to change during system development process or during operation

Requirements



Requirements management planning

An essential first stage in requirement management process Planning process consists of the following

- 1.Requirements identification
 - Each requirement must have unique traceability tag for cross reference and
- 2.Change management process
 - Set of activities that assess the impact and cost of changes
- 3.Traceability policy
 - A matrix showing links between requirements and other elements of software development
- 4.CASE tool support
 - Automatic tool to improve efficiency of change management process. Automated tools are required for requirements storage, change management and traceability management

Traceability

Maintains three types of traceability information.

1. Source traceability

- Links the requirements to the stakeholders

2. Requirements traceability

- Links dependent requirements within the requirements document

3. Design traceability

- Links from the requirements to the design module

Requirements change management

Consists of three principal stages:

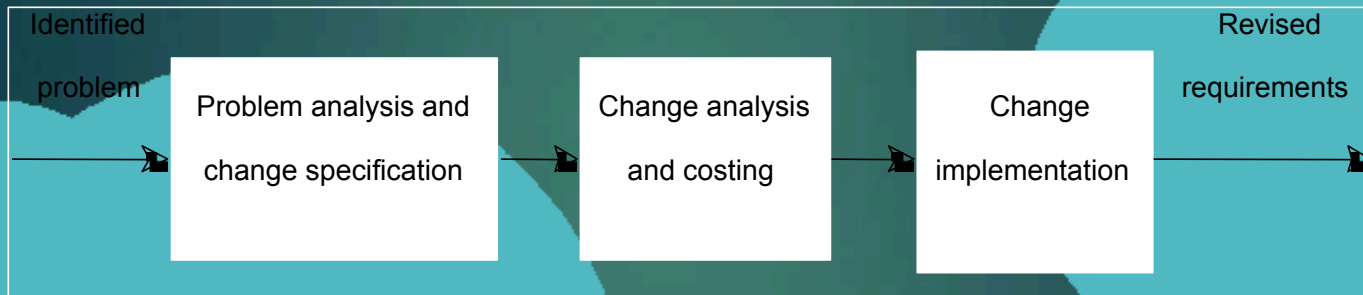
1. Problem analysis and change specification

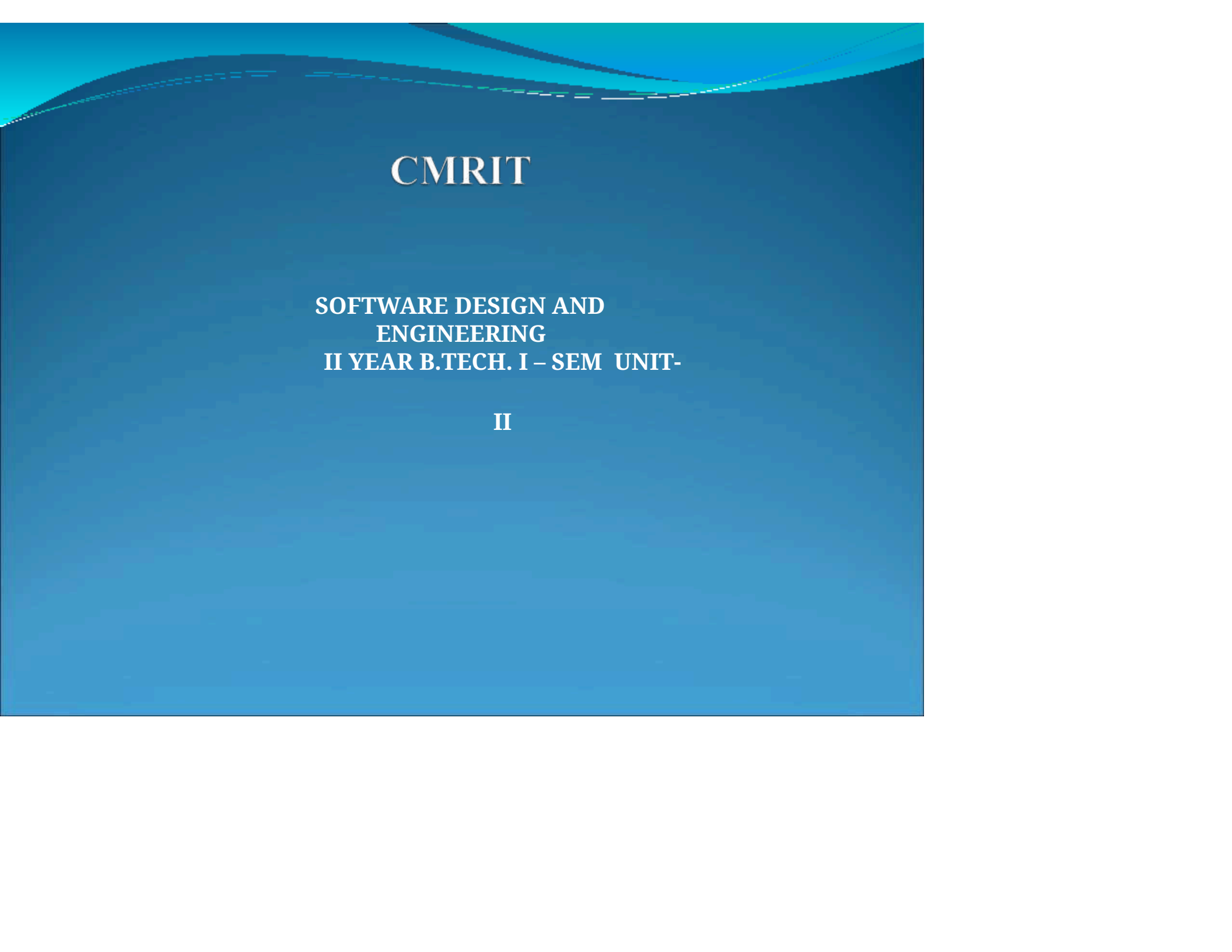
--Process starts with a specific change proposal and analysed to verify that it is valid. Impact analysis in terms of cost, time and risks

2. Change analysis and costing
3. Change implementation

--Carrying out the changes in requirements document, system design and its implementation

Change management





CMRIT

SOFTWARE DESIGN AND
ENGINEERING
II YEAR B.TECH. I – SEM UNIT-

II

- **Design Engineering** : Design process and Design quality, Design concepts, the design model.

- **Creating an architectural design** :
Software architecture, Data design,
Architectural styles and patterns,
Architectural Design.
- **Modeling component level design & performing user interface design:**
Designing class based components,
conducting component level design ,
Golden rules, user interface analysis and
design.

Design Engineering

- **Design engineering** encompasses the **set of principles, concepts, and practices** that leads to the development of a high- quality system or product.

What is design?

- Design is what virtually every engineer wants to do.
- It is the place where creativity rules – customer's requirements, business needs, and technical considerations all come together in the formulation of a product or a system.

Why is it important?

- Design allows a software engineer to model the system or product that is to be built.
- Design is the place where software **quality is established.**

DESIGN PROCESS AND DESIGN QUALITY:

Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software.

Goals of design:

- The design must implement all of the explicit requirements contained in the analysis model .
- The design must be a readable, understandable guide.
- The design should provide a complete picture of the software

Quality guidelines:

- A design should exhibit an architecture that ,
 - a. has been created using recognizable architectural styles or patterns
 - b. is composed of components that exhibit good design characteristics and
 - c. can be implemented in an evolutionary fashion, thereby facilitating implementation and testing.
- A design should be modular.
- A design should contain distinct representation of data, architecture, interfaces and components.

- lead to data structures that are appropriate for the classes to be implemented .
- lead to components that exhibit independent functional characteristics.
- lead to interface that reduce the complexity of connections between components and with the external environment.
- should be represented using a notation that effectively communicates its meaning.

Quality attributes: (FURPS)

- ***Functionality***
- ***Usability***
- ***Reliability***
- ***Performance***
- ***Supportability***

DESIGN CONCEPTS:

Fundamental software design concepts

provide the necessary framework for

1. **Abstraction:** Many levels of abstraction are there,

- **At the highest level of abstraction**, a solution is stated in broad terms using the language of the problem environment.
- **At lower levels of abstraction**, a more detailed description of the solution is provided.
- A ***procedural abstraction*** refers to a sequence of instructions that have a specific and limited function.
- A ***data abstraction*** is a named collection of data that describes a data object.

2.

Architecture

the overall structure of the software and the ways in which

that structure provides conceptual integrity for a system

- **Structured models** - organized collection of program components.
- **Framework models** - increase the level of design abstraction.
- **Dynamic models** - address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function external events.
- **Process models** - focus on the design of the business or technical process that the system must accommodate.
- **Functional models** - can be used to represent the functional hierarchy of a system.

3. Patterns:

- a design pattern describes a design structure that solves a particular design within a specific context and amid “forces” that may have an impact on the manner in which the pattern is applied and used.

- The intent of each design pattern is to provide a description that enables a designer to determine
 - 1) Whether the pattern is capable to the current work,
 - 2) Whether the pattern can be reused,
 - 3) Whether the pattern can serve as a guide for developing a similar, but functionally or structurally different pattern.

4.

- **Modularity** is divided into separately named and addressable components, sometimes called ***modules*** that are integrated to satisfy problem requirements.

5. Information Hiding:

- information contained within a module is inaccessible to other modules that have no need for such information.

6. Functional Independence:

- direct outgrowth of modularity and the concepts of abstraction and information hiding.
- Independence is assessed using two qualitative criteria: cohesion and coupling.
 - *Cohesion* is an indication of the relative functional strength of a module.
 - *Coupling* is an indication of the relative interdependence among modules.

7.

- **Refinement:** is development by successively refining levels of procedural detail.
- Refinement is actually a process of elaboration.

8. Refactoring:

- Refactoring is a reorganization technique that simplifies the design of a component without changing its function or behavior.

9. Design classes:

- The software team must define a set of design classes that ,
 - 1. Refine the analysis classes by providing design detail that will enable the classes to be implemented, and
 - 2. Create a new set of design classes that implement a software infrastructure to support the design solution.

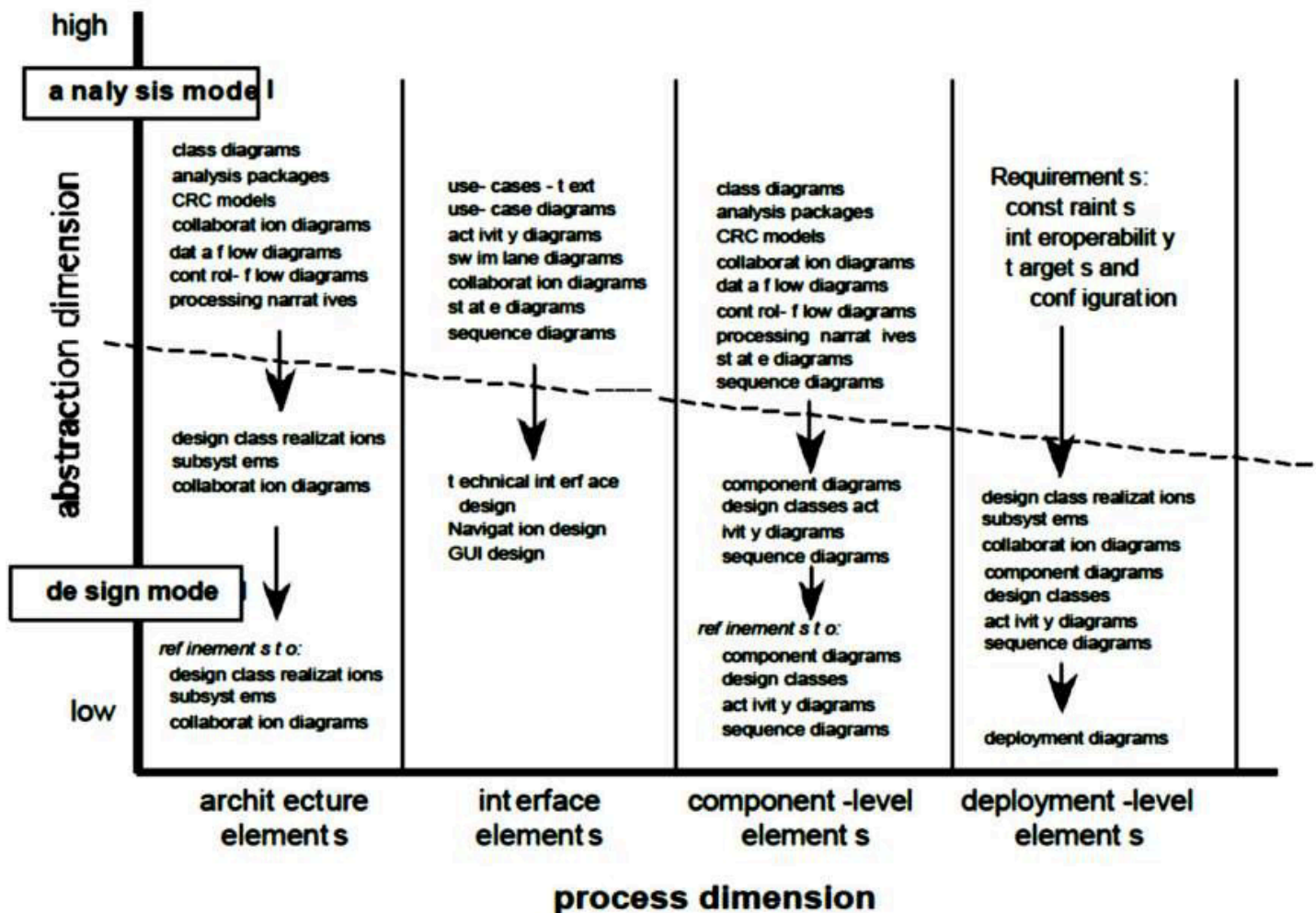
Five different types of design

- **classes interface classes:** define all abstractions that are necessary for human computer interaction.
- **Business domain classes:** are often refinements of the analysis classes defined earlier.
- **Process classes** implement lower – level business abstractions required to fully manage the business domain classes.
- **Persistent classes** represent data stores that will persist beyond the execution of the software.
- **System classes** implement software management and control functions that enable the system to operate and communicate within its computing environment and with the outside world.

Four characteristics of a well- formed design class,

- Complete and sufficient
- Primitiveness
- High cohesion
- Low coupling

THE DESIGN MODEL:



i. Data design elements:

- creates a model of data and/or information that is represented at a high level of abstraction.
- The structure of data has always been an important part of software design.
 - At the **program component level**
 - At **the application level**
 - At the **business level**

ii. Architectural design elements:

- The *architectural design* for software is the equivalent to the floor plan of a house. The architectural model is derived from three sources.
 - a. Information about the application domain for the software to be built.
 - b. Specific analysis model elements, and
 - c. The availability of architectural patterns.

iii. Interface design

- **Interface design** is equivalent to a set of detailed drawings for the doors, windows, and external utilities of a house.
- There are 3 important elements of interface design:
 - a. The user interface(UI);
 - b. External interfaces to other systems, devices, networks, or other produces or consumers of information; and
 - c. Internal interfaces between various design components.

iv. Component- level design elements:

- It is equivalent to a set of detailed drawings.
- The component-level design for software fully describes the internal detail of each software component.

v. Deployment-level design elements:

- It indicates how software functionality and subsystems will be allocated within the physical computing environment that will support the software.

ARCHITECTURAL

DESIGN

What Is Architecture?

SOFTWARE ARCHITECTURE:

- Architectural design represents the structure of data and program components that are required to build a computer- based system.
- The design of software architecture considers two levels of the design pyramid
 - - data design
 - - architectural design.

Why Is Architecture Important?

- Representations of software architecture are an enabler for communication between all parties (stakeholders) interested in the development of a computer-based system.
- Architecture “constitutes a relatively small, intellectually graspable model of how the system is structured and how its components work together” .

DATA DESIGN:

- translates data objects as part of the analysis model into data structures at the software component level and, when necessary, a database architecture at the application level.

1. Data design at the Architectural Level:

- The challenge for a business has been to extract useful information from this data environment, particularly when the information desired is cross functional.

2. Data design at the Component Level:

- focuses on the representation of data structures that are directly accessed by one or more software components.

The following set of principles for data specification. The systematic analysis principles applied to function and behavior should also be applied to data structures and the operations performed on each.

3. A data dictionary should be established and used to define both data and program design.
4. Low-level data design decisions should be deferred until late in the design process.
5. The representation of data structure should be known only to those modules that must make direct use of the data contained within the structure.
6. A library of useful data structures and the operations that may be applied to them should be developed.
7. A software design and programming language should support the specification and realization of abstract data types.

ARCHITECTURAL STYLES AND

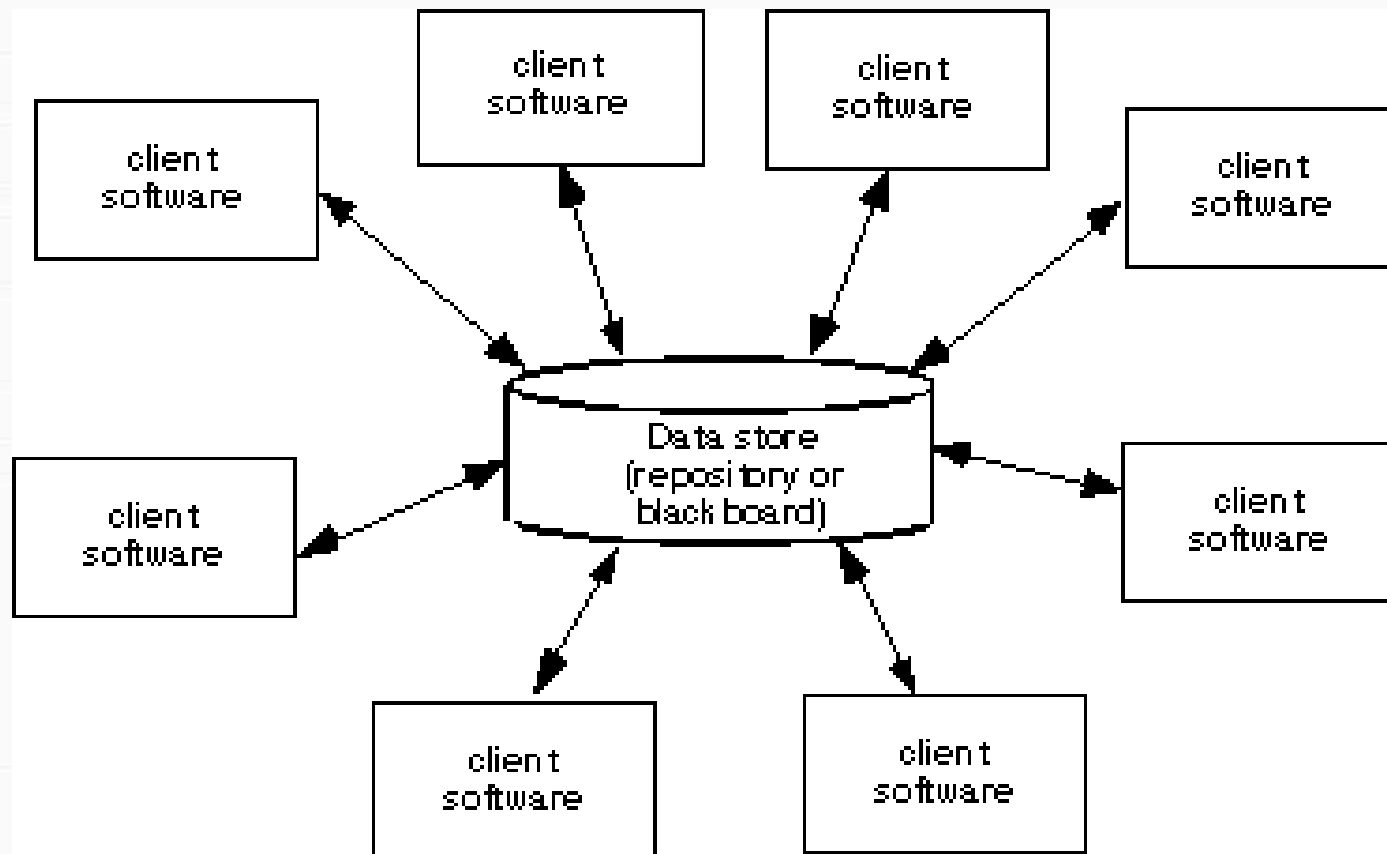
- **PATTERNS:** are that is built for computer-based systems also exhibits one of many architectural styles.
- Each style describes a system category that encompasses
 - (1) A set of *components*
 - (2) A set of *connectors*
 - (3) *Constraints*
 - (4) *Semantic models*

An *architectural pattern*, imposes a transformation the design of architecture. However, a pattern differs from a style in a number of fundamental ways:

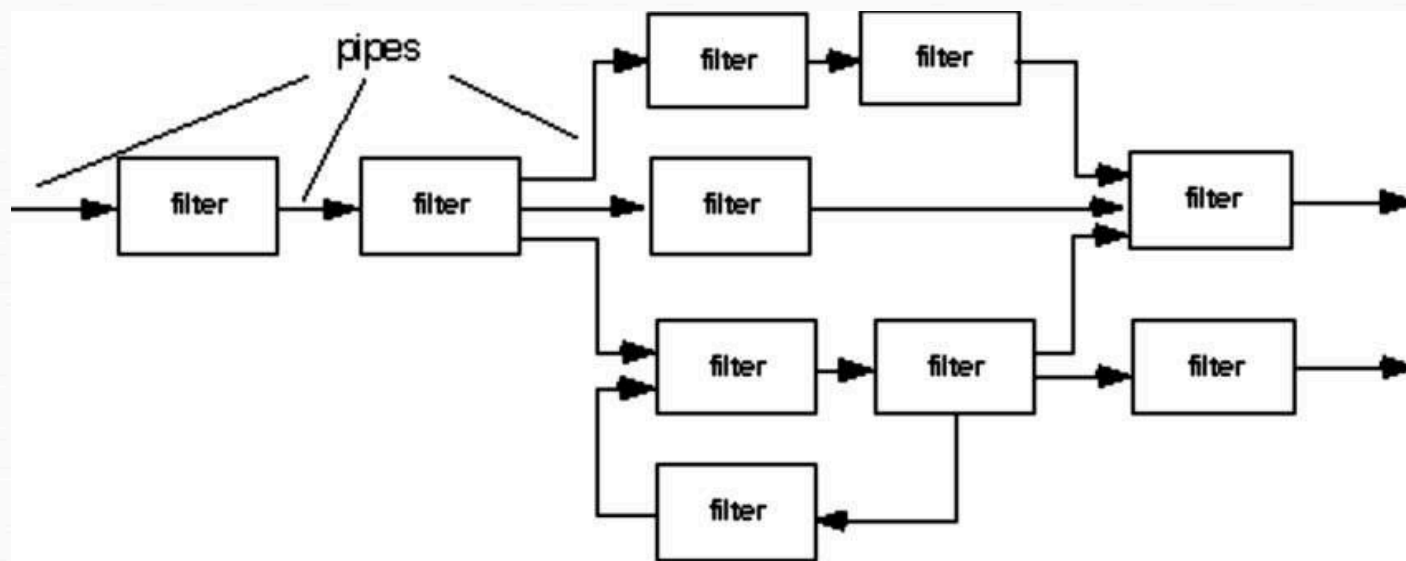
- (1) The scope of a pattern is less broad, focusing on one aspect of the architecture rather than the architecture in its entirety.
- (2) A pattern imposes a rule on the architecture, describing how the software will handle some aspect of its functionality at the infrastructure level.
- (3) Architectural patterns tend to address specific behavioral issues within the context of the architectural

1. A Brief Taxonomy of Styles and Patterns

Data-centered



Data-flow



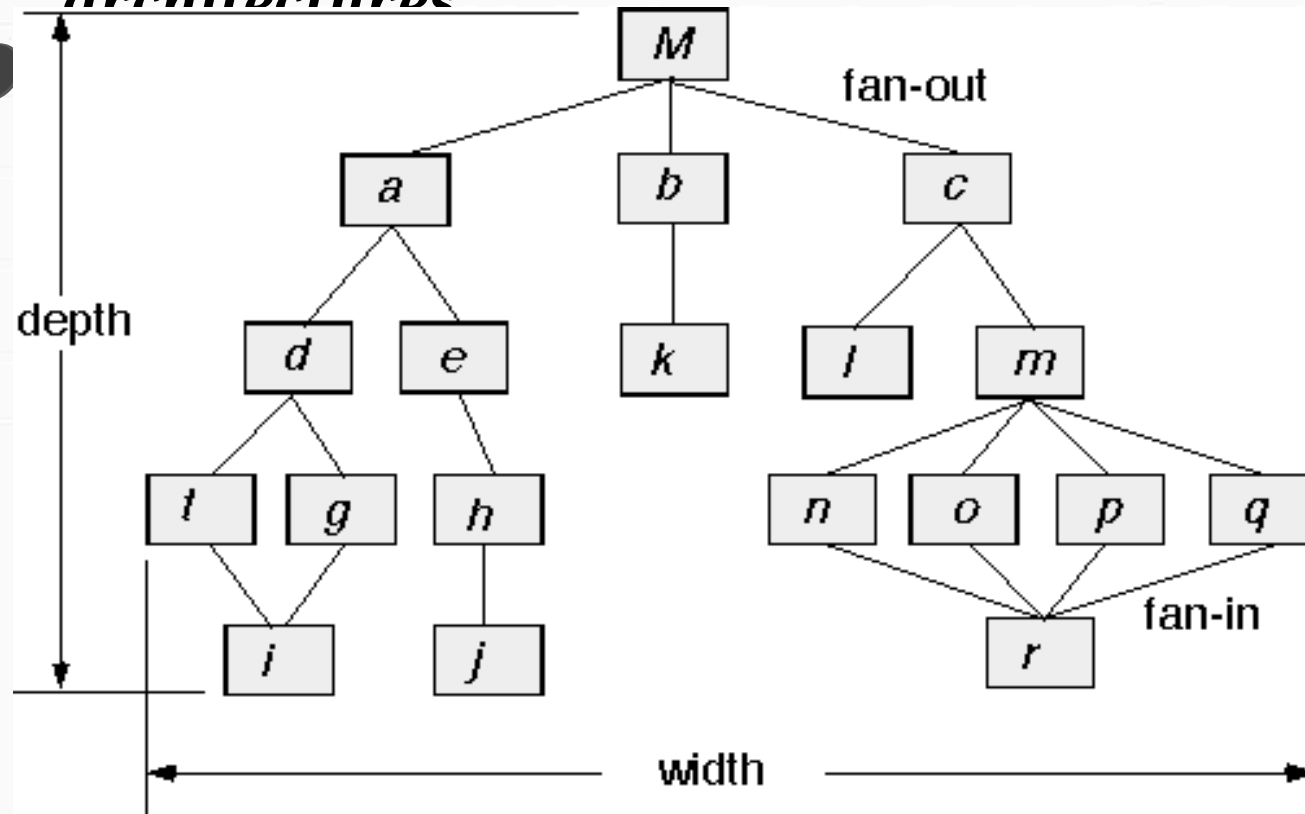
(a) pipes and filters



(b) batch sequential

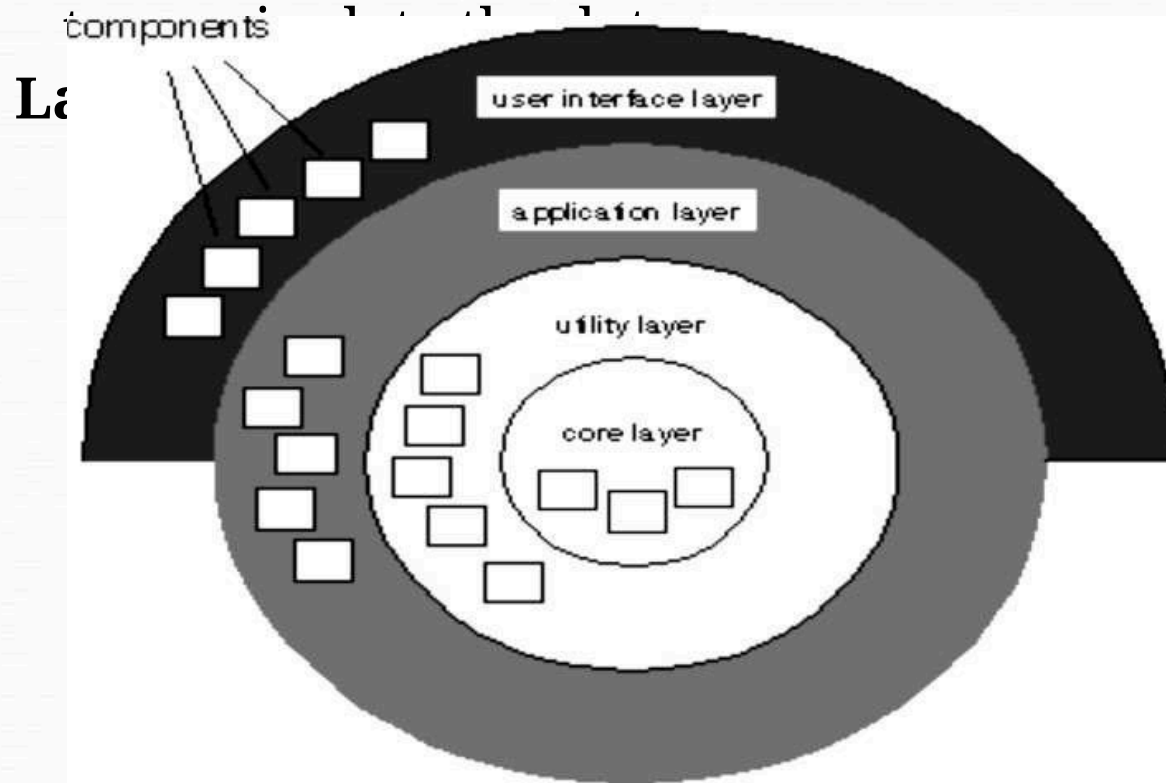
Call and return architectures:

- *Main program/subprogram architectures*



Object-oriented architectures:

- The components of a system encapsulate data and the operations that must be applied



2. Architectural Patterns:

- The architectural patterns for software define a specific approach for handling some behavioral characteristics of the system,
 - **Concurrency**
 - *operating system process management* pattern
 - *task scheduler* pattern
 - **Persistence**
 - a *database management system* pattern
 - an *application level persistence* pattern
 - **Distribution**
- **Organization and Refinement:**
 - it is important to establish a set of design criteria that can be used to assess an architectural design that is derived.

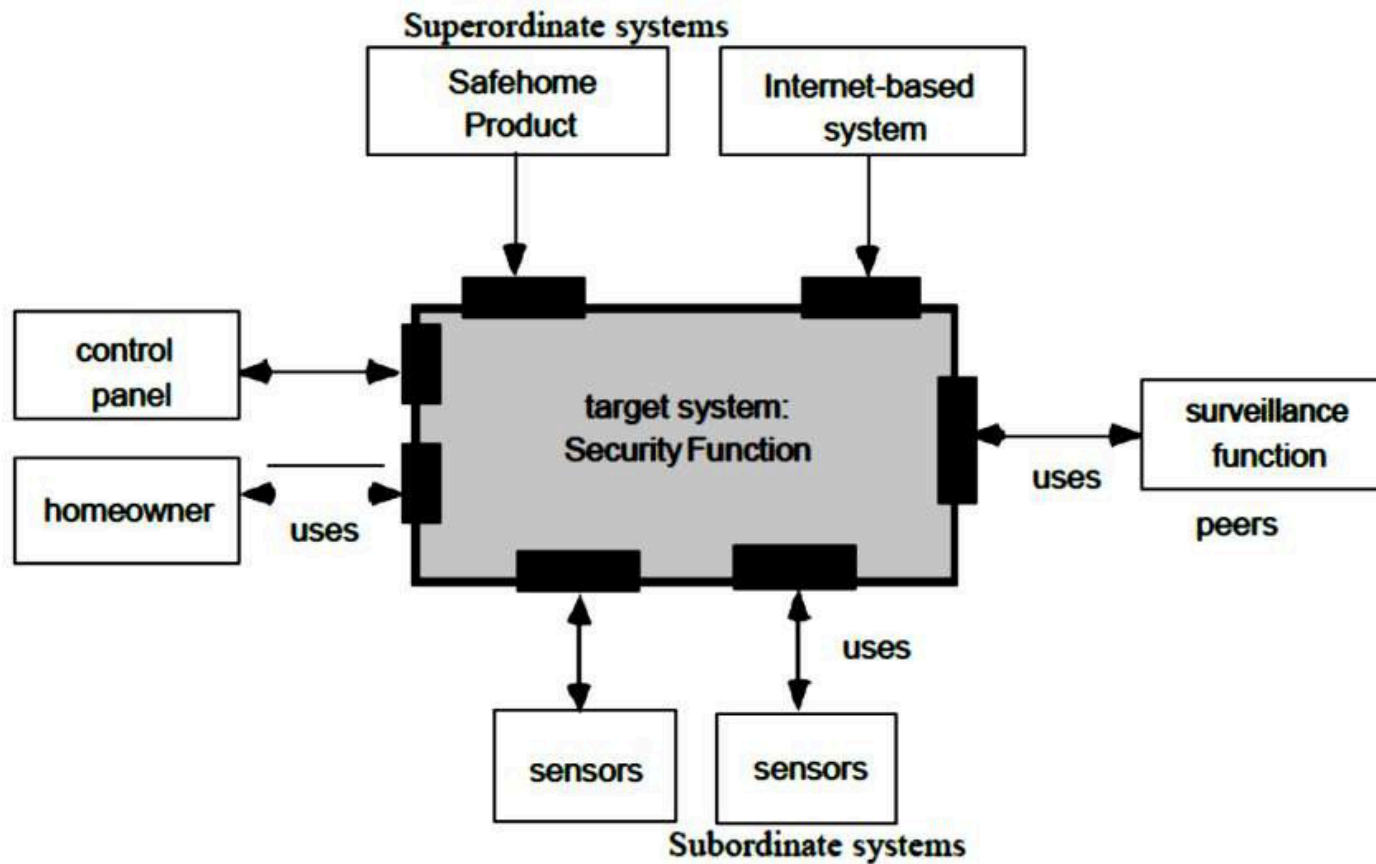
Contr

- **ol** How is control managed within the architecture?
- Does a distinct control hierarchy exist, and if so what is the role of components within this control hierarchy?
- How do components transfer control within the system?
- How is control shared among components?

Data

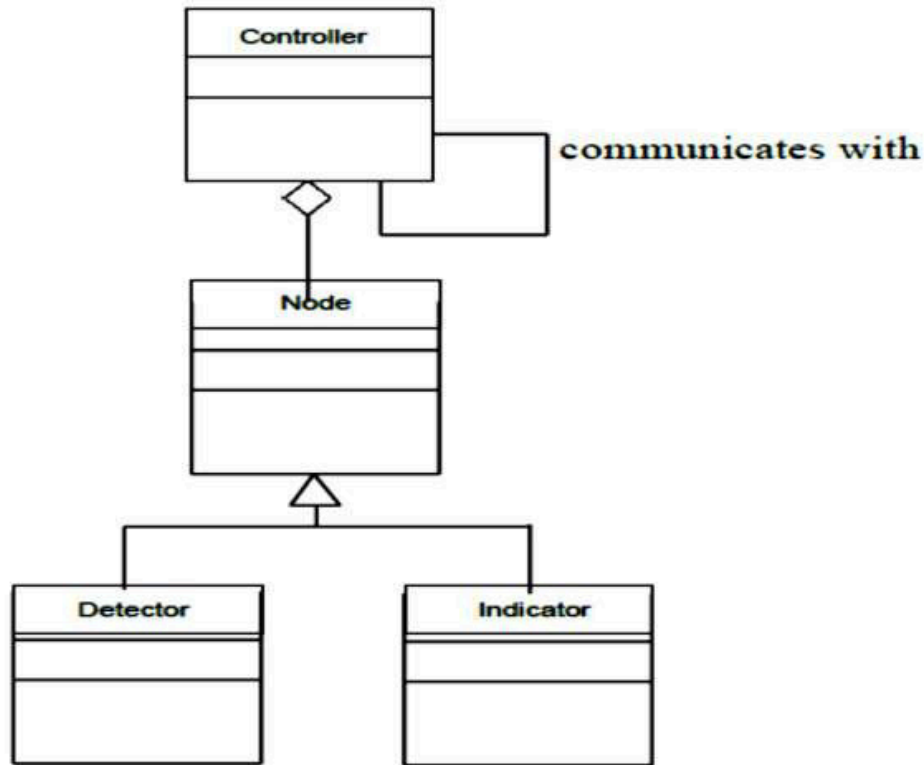
- ...How are data communicated between components?
- ... Is the flow of data continuous, or are data objects passed to the system sporadically?
- ...What is the mode of data transfer (i.e., are data passed from one component to another or are data available globally to be shared among system components)?
- ... Do data components (e.g., a blackboard or repository) exist, and if so, what is their role?
- ...How do functional components interact with data components?
- ...Are data components passive or active (i.e., does the data component actively interact with other components in the system)? How do data and control interact within the system?

ARCHITECTURAL DESIGN: Architectural context diagram



ii. Defining Archetypes:

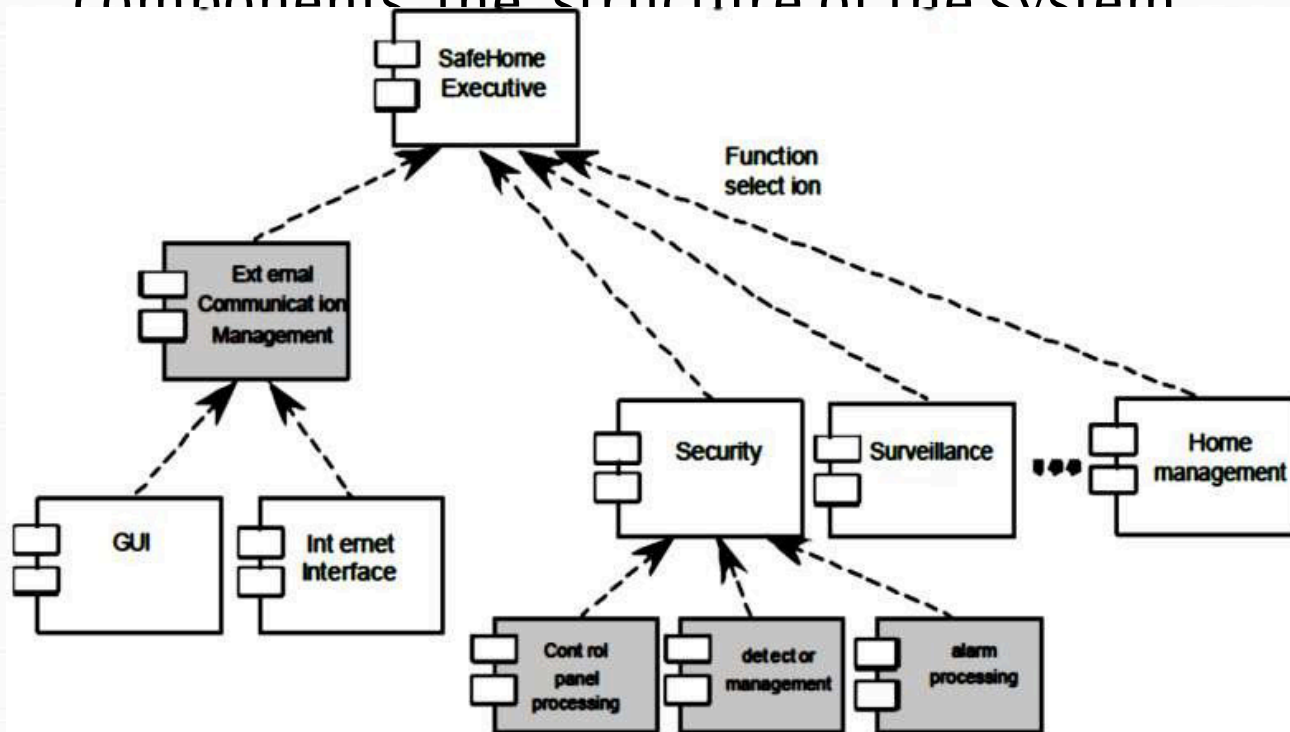
- An archetype is a class or pattern that represents a core abstraction that is critical to the design of architecture.



iii. Refining the Architecture into

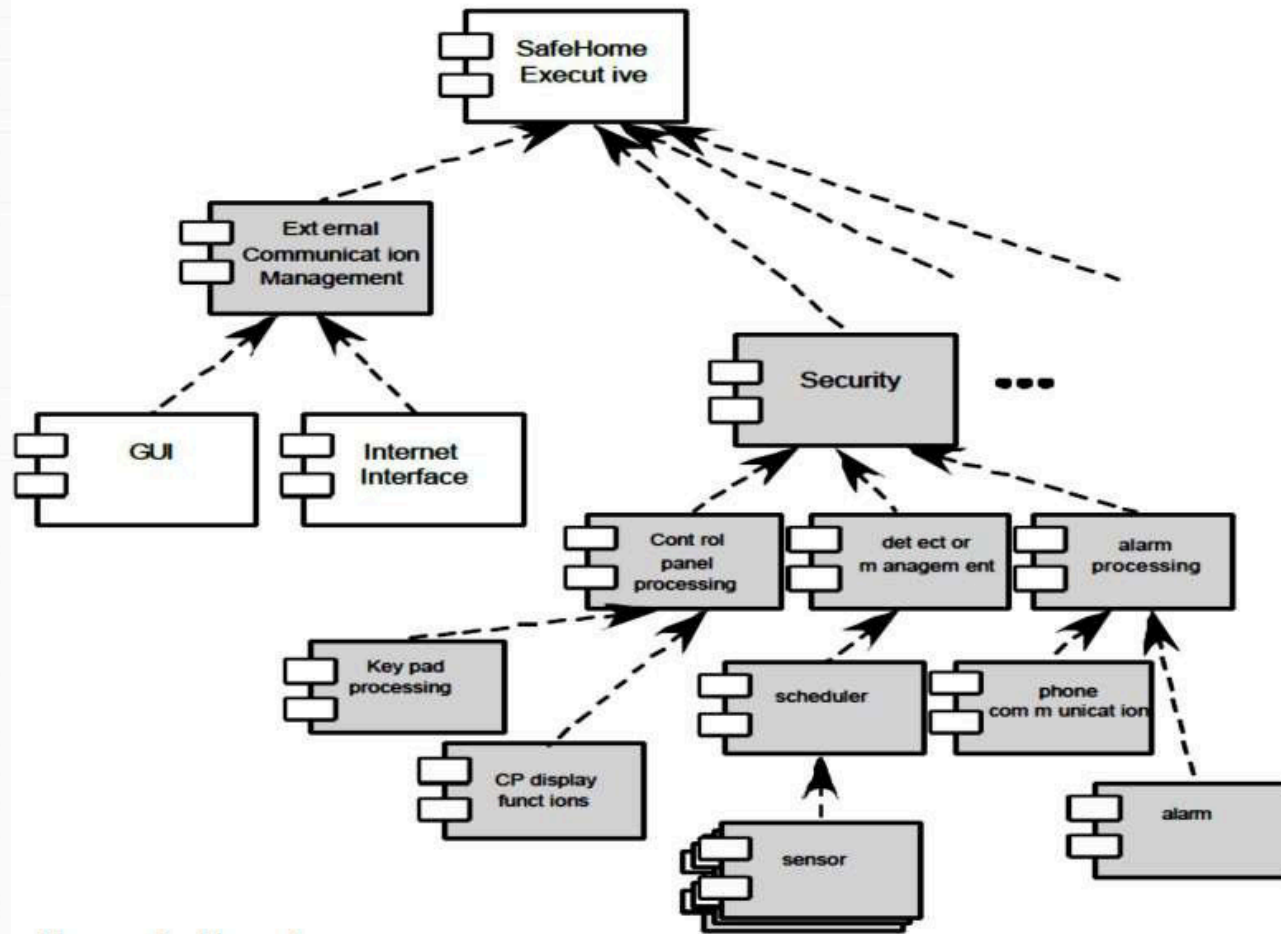
Components:

As the architecture is refined into components, the structure of the system



Component Structure

iv. Describing Instantiations of the



Designing class based components

Component:

Characteristics of components: 1.Reusability
2.Replacable 3.Extensible 4.Encapsulated
5.Independent

Class?

Class name

Data objects and data member functions.

By using data object we access the members of a class.

Designing class based components

1. Basic design principles
2. Component level design guidelines
3. Cohesion
4. Coupling



Basic design principles

1. **Open-closed principle**
2. **Liskov substitution principle**
3. **Dependency inversion principle**
4. **Interface segregation principle**
5. **Release reuse equivalency principle**
6. **Common closure principle**
7. **Common reuse principle**

2.Component-Level Design Guidelines

Components – Establish naming conventions for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model .

- Obtain architectural component names from the problem domain and ensure that they have meaning to all stakeholders who view the architectural model
- Use infrastructure component names that reflect their implementation-specific meaning
- **Dependencies and inheritance in UML**

3.Cohesion

The kinds of cohesion can be ranked in order from highest (best) to lowest (worst)

1. Functional
2. Layer
3. Communicational
4. Sequential
5. Procedural
6. Temporal
7. Utility

4. Coupling

The kinds of coupling can be ranked in order from lowest (best) to highest (worst)

1. Data coupling
2. Stamp coupling
3. Control coupling
4. Common coupling
5. Content coupling
6. Subroutine call coupling
7. Type use coupling
8. Inclusion or import coupling
9. External coupling

Conducting Component-Level Design

1. Identify all design classes that correspond to the problem domain as defined in the analysis model and architectural model
2. Identify all design classes that correspond to the infrastructure domain
3. Elaborate all design classes that are not acquired as reusable
 - a) components
 - Specify message details (i.e., structure) when classes or components collaborate
 - b) Identify appropriate interfaces (e.g., abstract classes) for each component
 - c) Elaborate attributes and define data types and data structures
 - d) Describe processing flow within each operation in detail by means of pseudocode or UML activity diagrams

Conducting Component-Level Design

4. Describe persistent data sources (databases and files) and identify the classes required to manage them
5. Develop and elaborate behavioural representations for a class or component
6. Elaborate deployment diagrams to provide additional implementation detail
7. Factor every component-level design representation and always consider alternatives

USER INTERFACE DESIGN

THE GOLDENRULES

There are three “golden rules”:

1. Place the user in control.
2. Reduce the user’s memory load.
3. Make the interface consistent.

Place the User in Control

- Define interaction modes in a way that does not force a user into unnecessary or undesired actions.
- Provide for flexible interaction.
- Allow user interaction to be interruptible and undoable.
- Streamline interaction as skill levels advance and allow the interaction to be customized.
- Hide technical internals from the casual user.
- Design for direct interaction with objects that appear on the screen.

Man-de-l
Reduces memory load:

- Reduces transitions between components, that enable a n in
- Define shortcuts that are intuitive.
- The visual layout of the interface should be based on a real world metaphor.
- Disclose information in a progressive fashion.

Make the Interface Consistent

Mandel [MAN97] defines a set of design principles that help make the interface consistent:

- Allow the user to put the current task into a meaningful context.
- Maintain consistency across a family of applications.
- If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so.

USER INTERFACE DESIGN

1. Interface Design Models

User Model

Design Model

Mental Model

Implementation Model

2. The User Interface Design Process

User, task, and environment analysis and modeling

Interface design Interface construction Interface

validation

USER INTERFACE DESIGN

1. User

Models can be categorized as

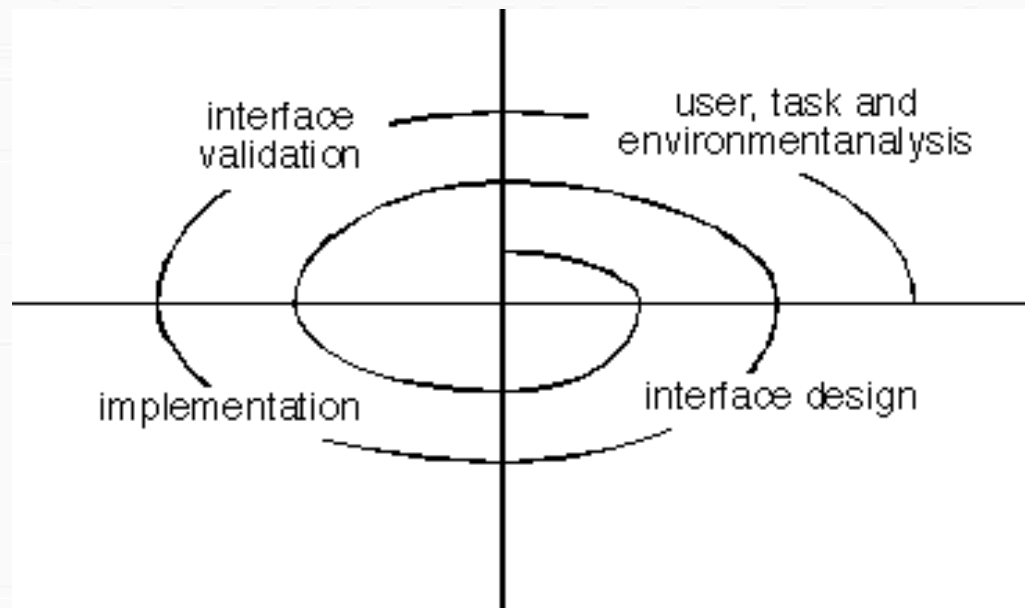
- Novices.
- Knowledgeable, intermittent users.
- Knowledgeable, frequent users.

2. Design Model

3. Mental Model

4. Implementation Model

The User Interface Design Process



The user interface framework activities

User task, and environment analysis and modeling
design processes seen in comparison to user
Interface design Interface construction Interface

dis validation
Process

User, task and environment analysis and modeling

The analysis of the user environment focuses on the physical work environment. Among the questions to be asked are

- Where will the interface be located physically?
- Will the user be sitting, standing, or performing other tasks unrelated to the interface?
- Does the interface hardware accommodate space, light, or noise constraints?



The goal of **interface design** is to define a set of interface objects and actions (and their screen representations) that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system.

Interface Construction(implementation)

The implementation activity normally begins with the creation of a prototype that enables usage scenarios to be evaluated. As the iterative design process continues, a user interface tool kit may be used to complete the construction of the interface.

Interface Validation

Validation focuses on the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements; the degree to which the interface is easy to use and easy to learn; and the users' acceptance of the interface as a useful tool in their work.



CMRIT

**SOFTWARE DESIGN AND
ENGINEERING
II YEAR B.TECH. I – SEM UNIT-**

III

Principles of modelling

First principle of modelling:

The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.

Second principle of modelling:

Every model may be expressed at different levels of precision.

Third principle of modelling:

The best models are connected to reality.

Fourth principle of modelling:

No single model is sufficient. Every non-trivial system is best approached through a small set of nearly independent models.

Conceptual model of UML

1. Things
2. Relationships
3. Diagrams

1. Things

1. Structural things
2. Behavioural things
3. Grouping things
4. Annotational things

1.1 Structural things

Class

Interface

Usecase

Collabarati

on

Componen

t Node

Active

class

Class

Graphical
representation:

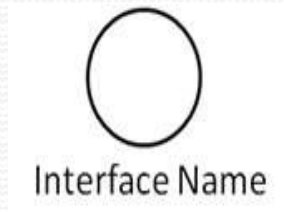
Examp
le:

ClassName
attributes list
operations list

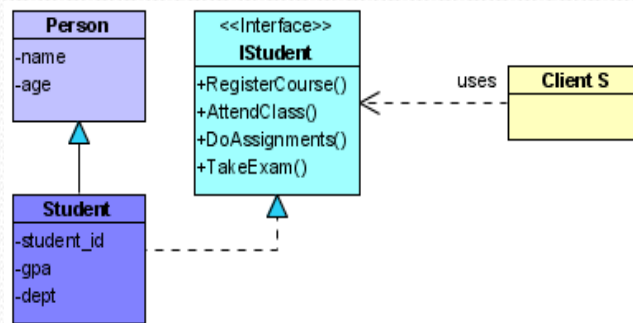
BankAccount
owner : String balance : Dollars = 0
deposit (amount : Dollars) withdrawal (amount : Dollars)

Interface

Graphical
representation:



O
r

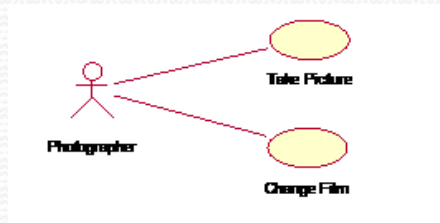


Use case

Graphical
representation:



Examp
le:

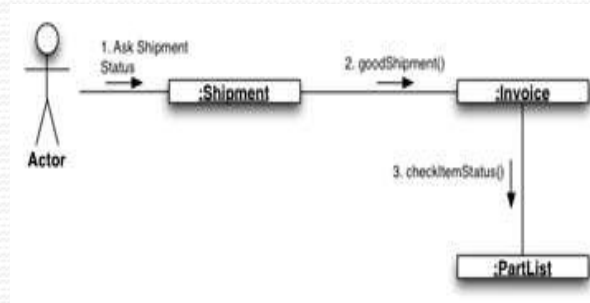


Collaboration

Graphical representation:



Example:

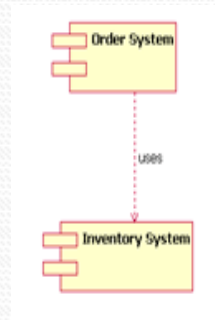


Component

Graphical
representation:

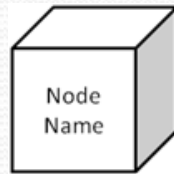


Example:

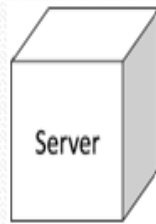


Node

Graphical
representation:



Example:

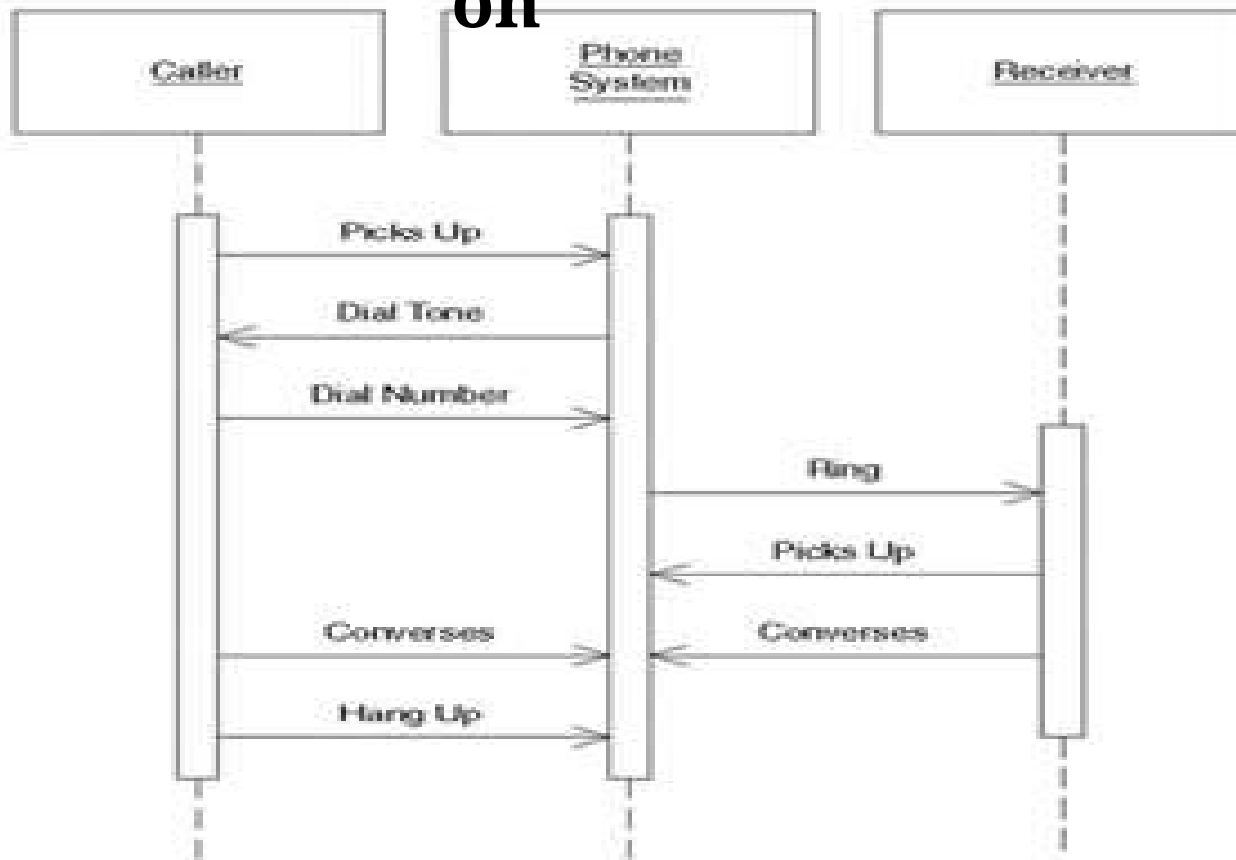




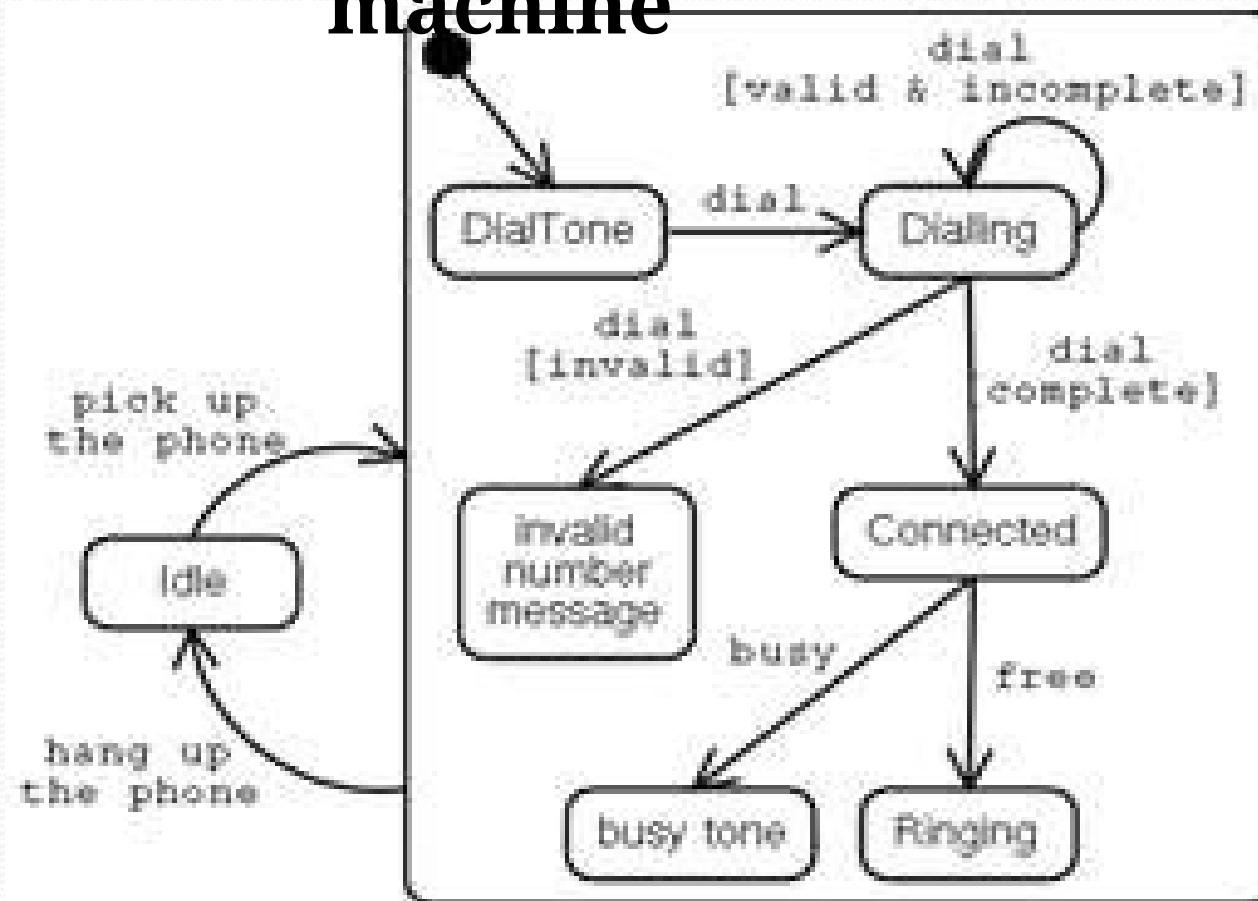
1.2 Behavioural things

Interaction State machine

Interacti on



State machine



1.3 Grouping things

Package

Graphical representation:



Example:



1.4 Annotational things

Not

e

Graphical
representation:

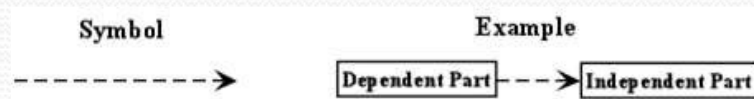


2. Relationships

1. Dependency
2. Association
3. Generalization-Specialization(is-a)
4. Realization

2.1 Dependency

Graphical
representation:



Example:



2.2

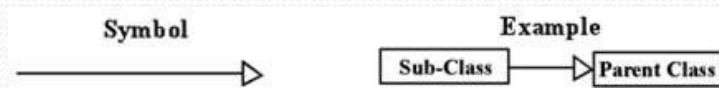
Association

Example.



2.3 Generalization-Specialization(is-a)

Graphical representation:

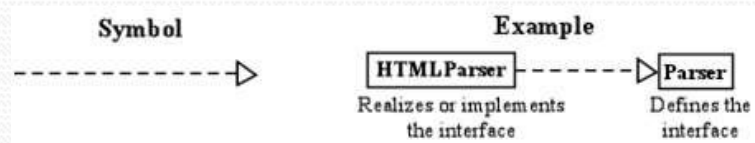


Example:



2.4 Realization

Graphical representation and
Example:



3. Diagrams

- 1) Class diagram
- 2) Object diagram
- 3) Use case diagram
- 4) Component diagram
- 5) Deployment diagram
- 6) Sequence diagram
- 7) Collaboration diagram
- 8) State chart diagram and
- 9) Activity diagram.



Diagrams in UML can be broadly classified as:

Structural Diagrams – Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

Behavior Diagrams – Capture dynamic aspects or behaviour of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

Class diagram

- Class diagram is a static diagram.
- It represents the static view of an application.
- Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- Class diagram describes the attributes and operations of a class and also the constraints imposed on the system.
- Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Purpose of Class diagram

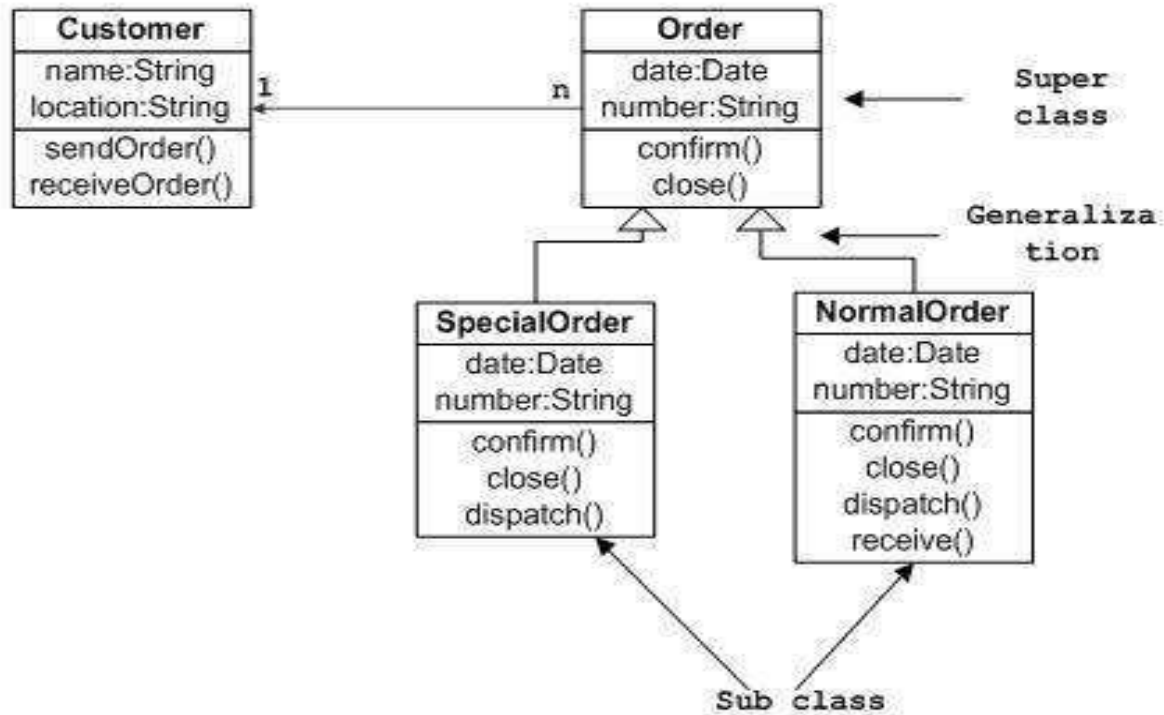
- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering
- A collection of class diagrams represent the whole system.

Class diagram

The following points should be remembered while drawing a class diagram

- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.
- Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

Sample Class Diagram



Object diagram

- ❑ Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams.
- ❑ Object diagrams represent an instance of a class diagram.
- ❑ The basic concepts are similar for class diagrams and object diagrams.
- ❑ Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment.
- ❑ Object diagrams are used to render a set of objects and their relationships as an instance.

Purpose of Object Diagrams

- The purposes of object diagrams are similar to class diagrams.
 - The difference is that a class diagram represents an abstract model (blue print) consisting of classes and their relationships.
 - However, an object diagram represents an instance at a particular moment (real world object), which is concrete in nature.
 - It means the object diagram is closer to the actual system behaviour. The purpose is to capture the static view of a system at a particular moment.
-
1. Forward and reverse engineering.
 2. Object relationships of a system
 3. Static view of an interaction.
 4. Understand object behaviour and their relationship from practical perspective

How to draw an object diagram

- Only those instances are considered, which have an impact on the system.
- it is clear that a single object diagram cannot capture all the necessary

instances or rather cannot specify all the objects of a system.

Hence, the solution is –

First, analyze the system and decide which instances have important data and association.

Second, consider only those instances, which will cover the functionality. Third, make some optimization as the number of instances are unlimited. Before drawing an object diagram, the following things should be remembered and understood clearly

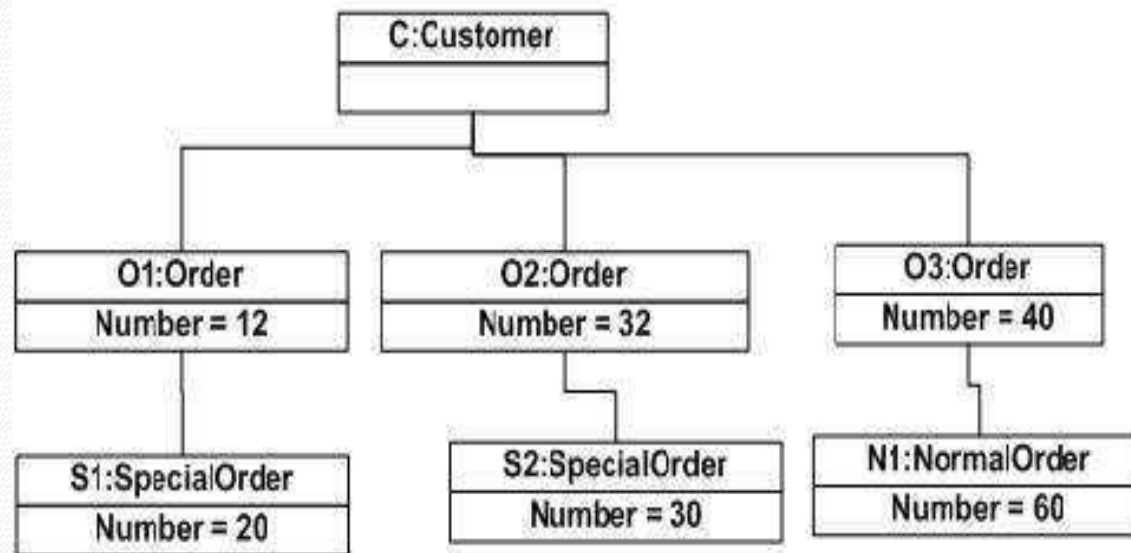
–

- **Object diagrams consist of objects.**
- **The link in object diagram is used to connect objects.**
- **Objects and links are the two elements used to construct an object diagram.**

Following things are to be decided before starting the construction of the diagram

- The object diagram should have a meaningful name to indicate its purpose.
- The most important elements are to be identified.
- The association among objects should be clarified.
- Values of different elements need to be captured to include in the object diagram.
- Add proper notes at points where more clarity is required.

Object diagram of an order management system





Object diagrams are used for

- Making the prototype of a system.
- Reverse engineering.
- Modelling complex data structures.
- Understanding the system from practical perspective.

Interaction Diagrams

From the term Interaction, it is clear that the diagram is used to describe some type of interactions among the different elements in the model. This interaction is a part of dynamic behavior of the system.

This interactive behavior is represented in UML by two diagrams known as **Sequence diagram** and **Collaboration diagram**. The basic purpose of both the diagrams are similar.

Sequence diagram emphasizes on time sequence of messages and collaboration diagram emphasizes on the structural organization of the objects that send and receive messages

Purpose of Interaction Diagrams

The purpose of interaction diagrams is to visualize the interactive behavior of the system. Visualizing the interaction is a difficult task. Hence, the solution is to use different types of models to capture the different aspects of the interaction. Sequence and collaboration diagrams are used to capture the dynamic nature but from a different angle.

The purpose of interaction diagram is –

To capture the dynamic behaviour of a system.

To describe the message flow in the system.

To describe the structural organization of the objects.

To describe the interaction among objects.

How to Draw an Interaction Diagram?

The purpose of interaction diagrams is to capture the dynamic aspect of a system. So to capture the dynamic aspect, we need to understand what a dynamic aspect is and how it is visualized.

Dynamic aspect can be defined as the snapshot of the running system at a particular moment

We have two types of interaction diagrams in UML.

One is the sequence diagram and the other is the collaboration diagram.

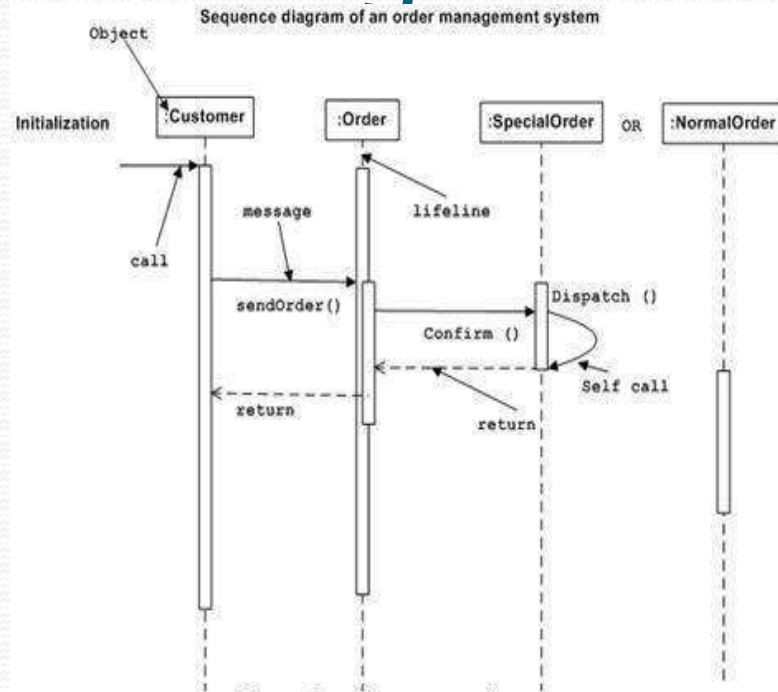
The sequence diagram captures the time sequence of the message flow from one object to another and

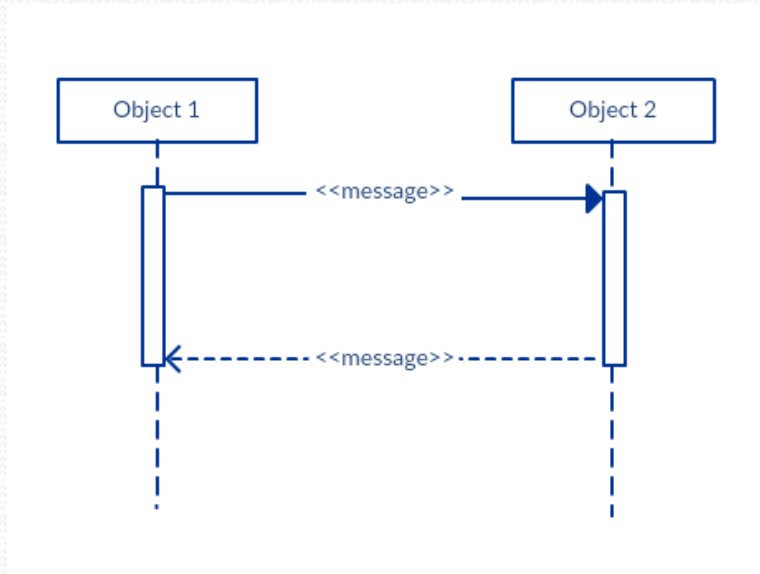
the collaboration diagram describes the organization of objects in a system taking part in the message flow.

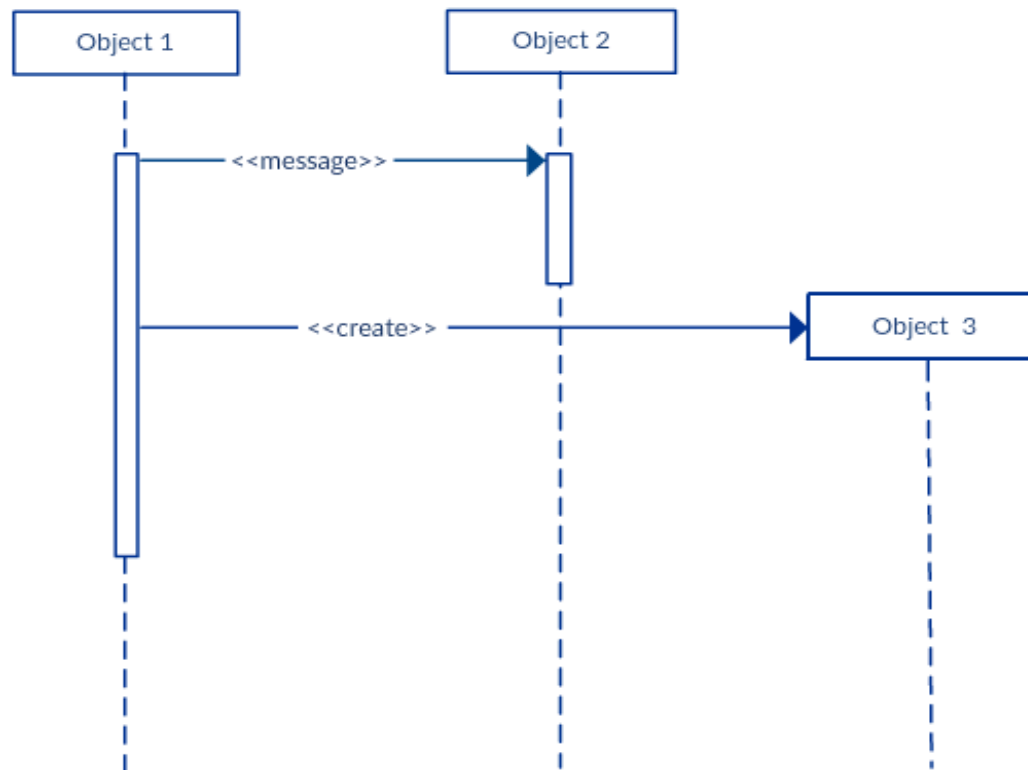
Following things are to be identified clearly before drawing the interaction diagram

- Objects taking part in the interaction.
- Message flows among the objects.
- The sequence in which the messages are flowing.
- Object organization.

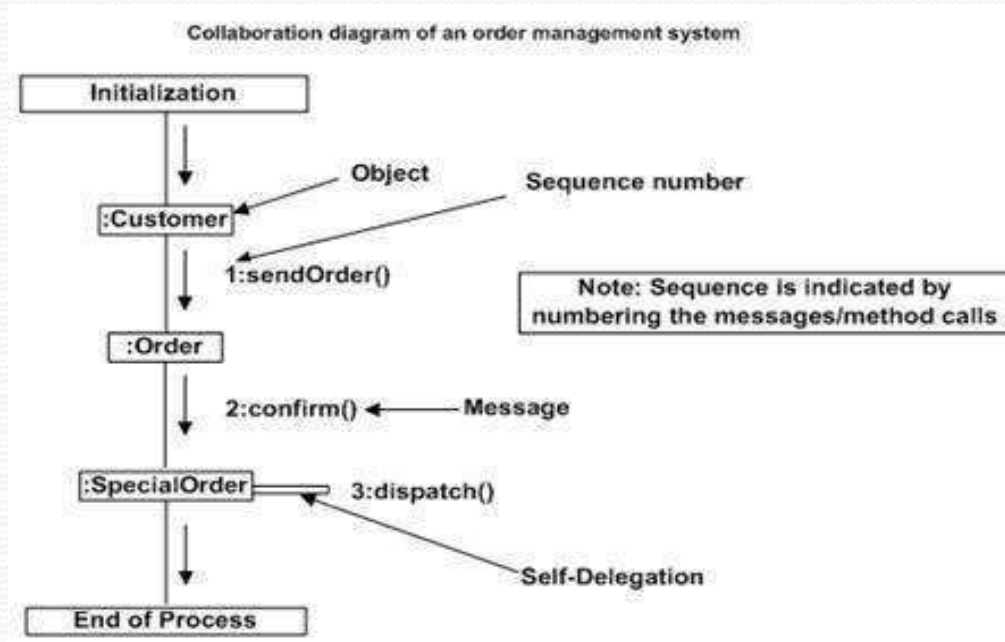
Sequence diagram







Collaboration diagram



Where to Use Interaction Diagrams?

Sequence diagrams are used to capture the order of messages flowing from one object to another. Collaboration diagrams are used to describe the structural organization of the objects taking part in the interaction. A single diagram is not sufficient to describe the dynamic aspect of an entire system, so a set of diagrams are used to capture it as a whole. **Interaction diagrams** are used when we want to understand the message flow and the structural organization. Message flow means the sequence of control flow from one object to another. Structural organization means the visual organization of the elements in a system.

Interaction diagrams can be used –

- To model the flow of control by time sequence.
- To model the flow of control by structural organizations.
- For forward engineering.
- For reverse engineering.

Use Case Diagrams

- To model a system, the most important aspect is to capture the dynamic behavior. Dynamic behavior means the behavior of the system when it is running/operating.
- Only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML, there are five diagrams available to model the dynamic nature and use case diagram is one of them. The use case diagram is dynamic in nature, there should be some **internal or external factors** for making the interaction.
- These internal and external agents are known as actors. Use case diagrams consists of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A single use case diagram captures a particular functionality of a system.
- Hence to model the entire system, a number of use case diagrams are used.

Purpose of Use Case Diagrams

- The purpose of use case diagram is to capture the dynamic aspect of a system.
- Use case diagrams are used to gather the requirements of a system including internal and external influences.
- These requirements are mostly design requirements.
- Hence, when a system is analyzed to gather its functionalities, use cases are prepared and actors are identified.
- When the initial task is complete, use case diagrams are modelled to present the outside view.

The purposes of use case diagrams can be said to be as follows –

- Used to gather the requirements of a system.
- Used to get an outside view of a system.
- Identify the external and internal factors influencing the system.
- Show the interaction among the requirements and actors.

How to Draw a Use Case Diagram?

Use case diagrams are considered for high level requirement analysis of a system.

When the requirements of a system are analyzed, the functionalities are captured in use cases.

We can say that use cases are nothing but the system functionalities written in an organized manner.

The second thing which is relevant to use cases are the actors.

Actors can be defined as something that interacts with the system. Actors can be a human user, some internal applications, or may be some external applications.

When we are planning to draw a use case diagram, we should have the following items identified.

- Functionalities to be represented as use case
- Actors
- Relationships among the use cases and actors.

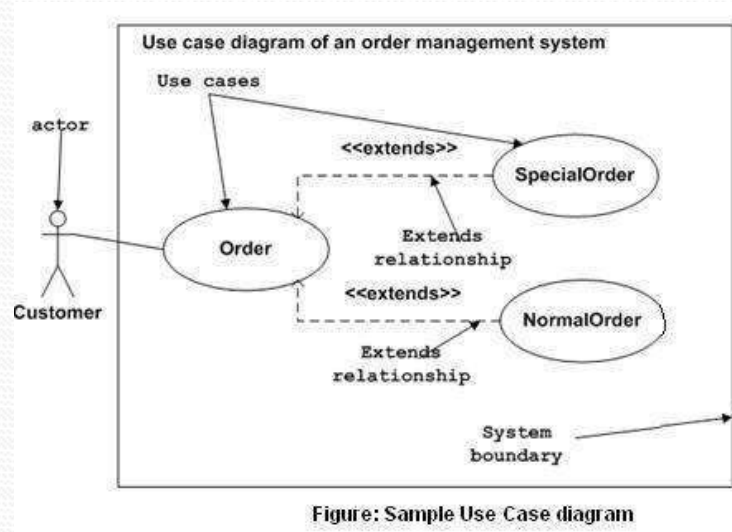
How to Draw a Use Case Diagram?

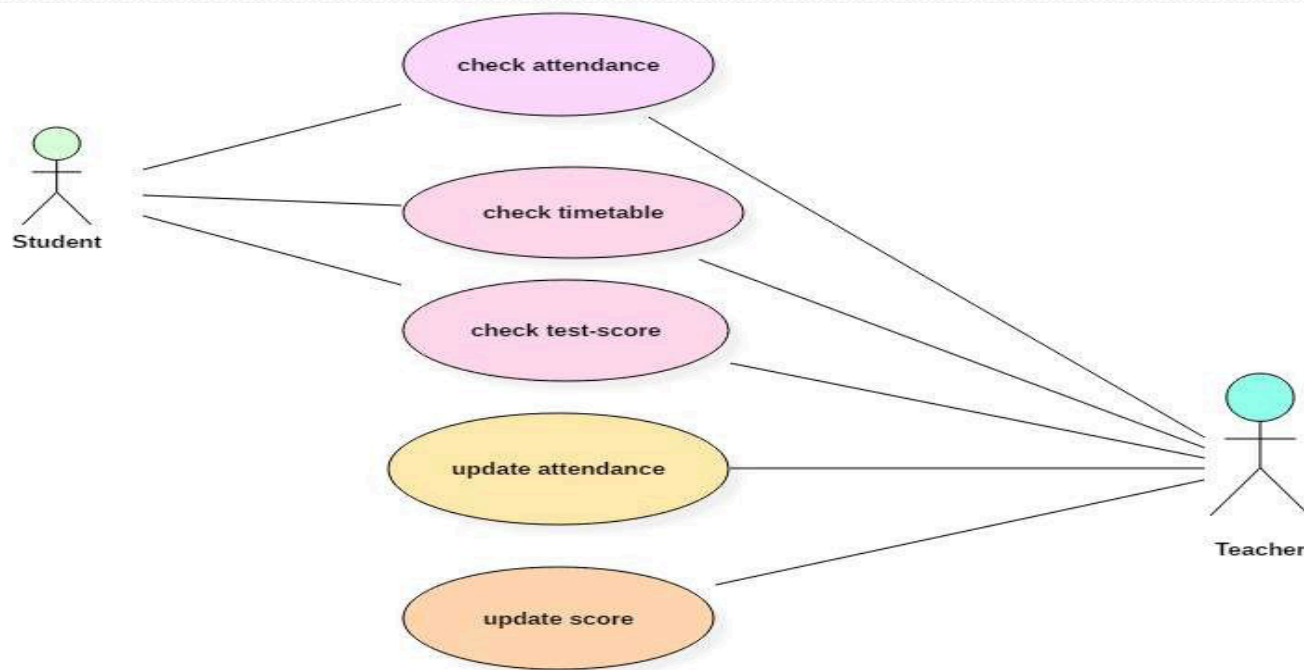
Use case diagrams are drawn to capture the functional requirements of a system.,

we have to use the following guidelines to draw an efficient use case diagram

- The name of a use case is very important. The name should be chosen in such a way so that it can identify the functionalities performed.
- Give a suitable name for actors.
- Show relationships and dependencies clearly in the diagram.
- Do not try to include all types of relationships, as the main purpose of the diagram is to identify the requirements.
- Use notes whenever required to clarify some important points.

Sample use case diagram





Where to Use a Use Case Diagram?

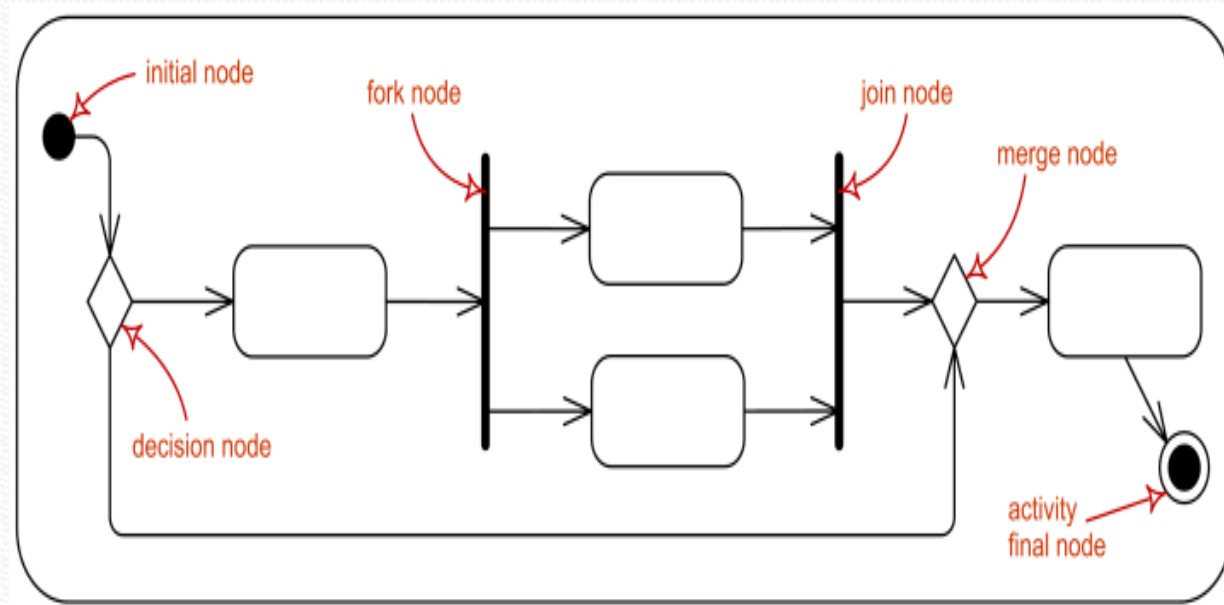
- Use case diagrams specify the events of a system and their flows. But use case diagram never describes how they are implemented. Use case diagram can be imagined as a black box where only the input, output, and the function of the black box is known.
- These diagrams are used at a very high level of design. This high level design is refined again and again to get a complete and practical picture of the system.
- A well-structured use case also describes the pre-condition, post condition, and exceptions. These extra elements are used to make test cases when performing the testing.
- In forward engineering, use case diagrams are used to make test cases and in reverse engineering use cases are used to prepare the requirement details from the existing application.

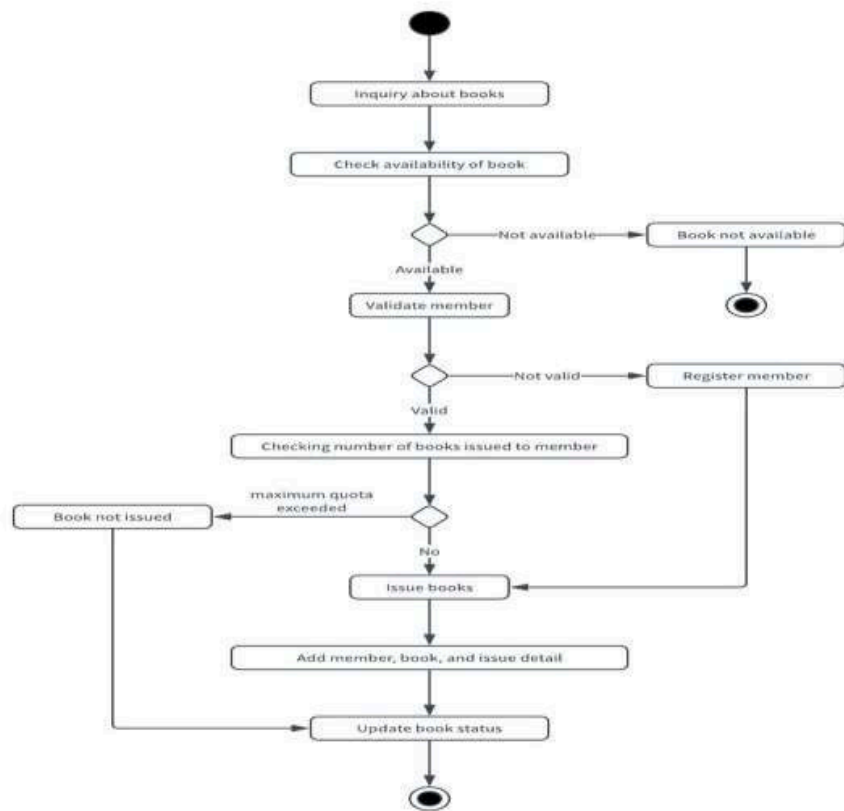
Use case diagrams can be used for –

- Requirement analysis and high level design.
- Model the context of a system.
- Reverse engineering.
- Forward engineering.

Activity Diagrams

- Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.
- Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.
- The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc





Purpose of Activity Diagrams

The basic purposes of activity diagrams is similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another.

Activity is a particular operation of the system.

The only missing thing in the activity diagram is the message part.

It does not show any message flow from one activity to another.

Activity diagram is sometimes considered as the flowchart. Although the diagrams look

like a flowchart, they are not.

It shows different flows such as parallel, branched, concurrent, and single.

The purpose of an activity diagram can be described as -

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

How to Draw an Activity Diagram?

The main element of an activity diagram is the **activity itself**. An activity is a function performed by the system. After identifying the activities, we need to understand how they are associated with constraints and conditions.

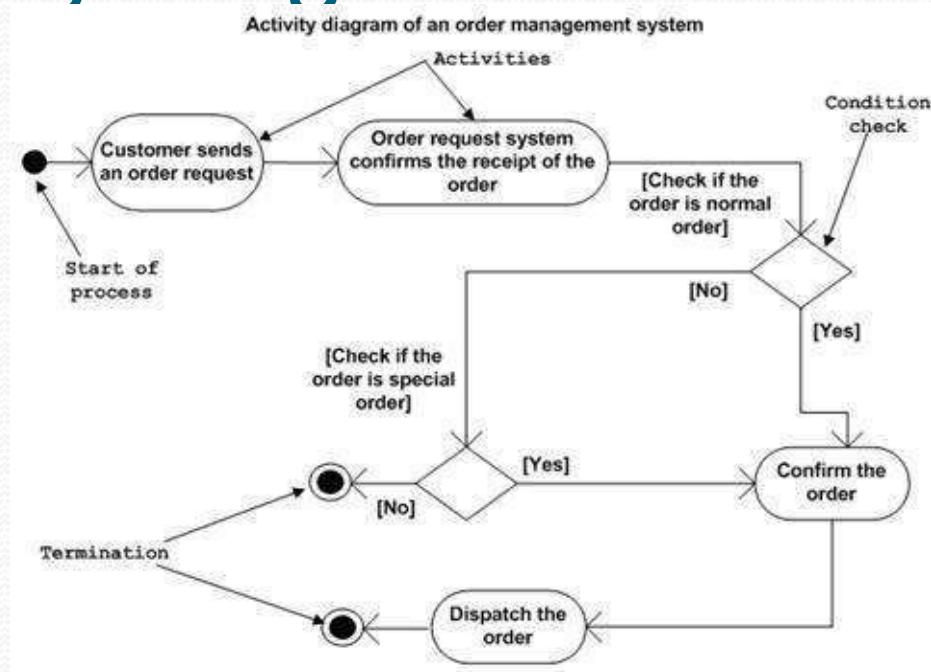
Before drawing an activity diagram, we should identify the following elements

–

- Activities
- Association
- Conditions
- Constraints

Once the above-mentioned parameters are identified, we need to make a mental layout of the entire flow. This mental layout is then transformed into an activity diagram.

Activity diagram



Where to Use Activity Diagrams?

- Activity diagram is suitable for modelling the activity flow of the system.
- An application can have multiple systems. Activity diagram also captures these systems and describes the flow from one system to another. This specific usage is not available in other diagrams. These systems can be database, external queues, or any other system.
- An activity diagram is drawn from a very high level. So it gives high level view of a system. This high level view is mainly for business users or any other person who is not a technical person.
- This diagram is used to model the activities which are nothing but business requirements. The diagram has more impact on business understanding rather than on implementation details.

Activity diagram can be used for –

- Modelling work flow by using activities.
- Modelling business requirements.
- High level understanding of the system's functionalities.
- Investigating business requirements at a later stage.

State chart Diagrams

- The name of the diagram itself clarifies the purpose of the diagram and other details. It describes different states of a component in a system. The states are specific to a component/object of a system.
- A Statechart diagram describes a state machine. State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events.
- It is a special kind of a Statechart diagram. As Statechart diagram defines the states, it is used to model the lifetime of an object.

Purpose of Statechart Diagrams

- ❖ State chart diagrams are useful to model the reactive systems.
Reactive systems
can be defined as a system that responds to external or internal events.
- ❖ Statechart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered.
- ❖ The most important purpose of Statechart diagram is to model lifetime of an object from creation to termination.
- ❖ Statechart diagrams are also used for forward and reverse engineering of a system. However, the main purpose is to model the reactive system.
Following are the main purposes of using Statechart diagrams –
 - To model the dynamic aspect of a system.
 - To model the life time of a reactive system.
 - To describe different states of an object during its life time.
 - Define a state machine to model the states of an object.

How to Draw a Statechart Diagram?

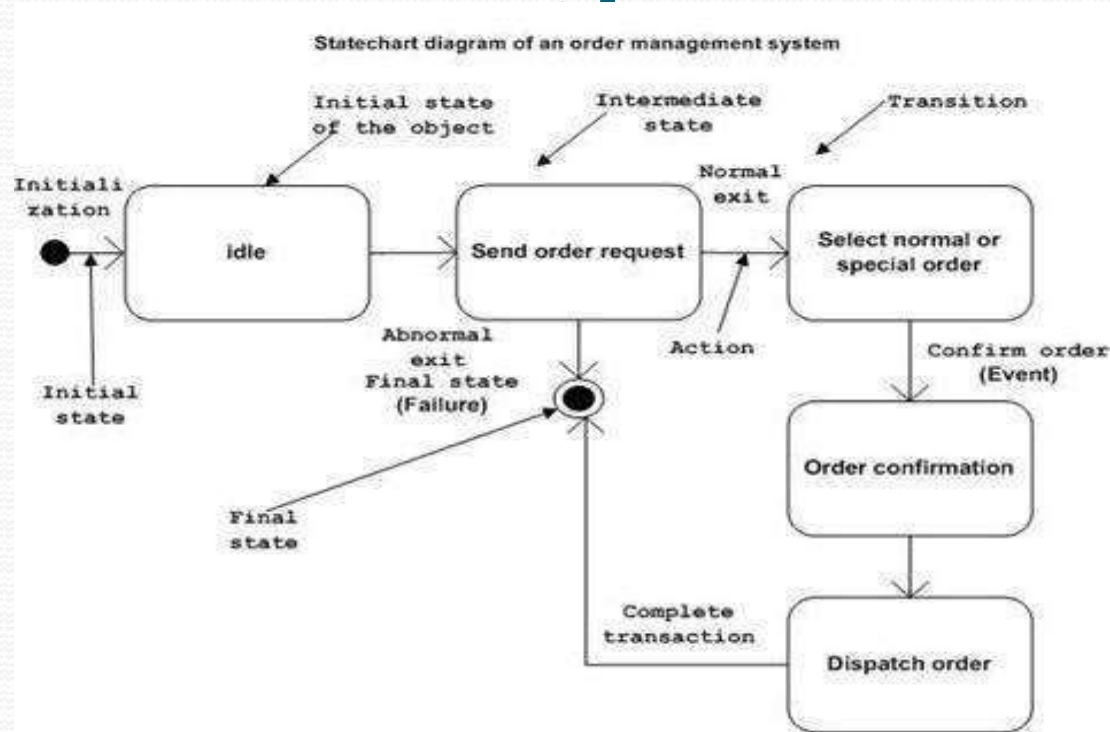
- Statechart diagram is used to describe the states of different objects in its life cycle. Emphasis is placed on the state changes upon some internal or external events. These states of objects are important to analyze and implement them accurately.
- Statechart diagrams are very important for describing the states. States can be identified as the condition of objects when a particular event occurs.

Before drawing a Statechart diagram we should clarify the following points

–

- Identify the important objects to be analyzed.
- Identify the states.
- Identify the events.

Statechart Diagram



Where to Use Statechart Diagrams?

- When implementing a system, it is very important to clarify different states of an object during its life time.
- When these states and events are identified, they are used to model it and these models are used during the implementation of the system.
- If we look into the practical implementation of Statechart diagram, then it is mainly used to analyze the object states influenced by events. This analysis is helpful to understand the system behavior during its execution.

The main usage can be described as –

- To model the object states of a system.
- To model the reactive system. Reactive system consists of reactive objects.
- To identify the events responsible for state changes.
- Forward and reverse engineering.

Component Diagrams

- Component diagrams are different in terms of nature and behavior.
- Component diagrams are used to model the physical aspects of a system. Physical aspects are the elements such as executables, libraries, files, documents, etc. which reside in a node.
- Component diagrams are used to visualize the organization and relationships among components in a system. These diagrams are also used to make executable systems.

Purpose of Component Diagrams

- Component diagram is a special kind of diagram in UML. The purpose is also different from all other diagrams discussed so far. It does not describe the functionality of the system but it describes the components used to make those functionalities.
 - Thus from that point of view, component diagrams are used to visualize the physical components in a system. These components are libraries, packages, files, etc.
 - Component diagrams can also be described as a static implementation view of a system. Static implementation represents the organization of the components at a particular moment.
 - A single component diagram cannot represent the entire system but a collection of diagrams is used to represent the whole.
- The purpose of the component diagram can be summarized as –**
- Visualize the components of a system.
 - Construct executables by using forward and reverse engineering.
 - Describe the organization and relationships of the components.

How to Draw a Component Diagram?

- Component diagrams are used during the implementation phase of an application.

However, it is prepared well in advance to visualize the implementation details.

- Initially, the system is designed using different UML diagrams and then when the artifacts are ready, component diagrams are used to get an idea of the implementation.
- This diagram is very important as without it the application cannot be implemented efficiently. A well-prepared component diagram is also important for other aspects such as application performance, maintenance, etc.



Before drawing a component diagram, the following artifacts are to be identified clearly –

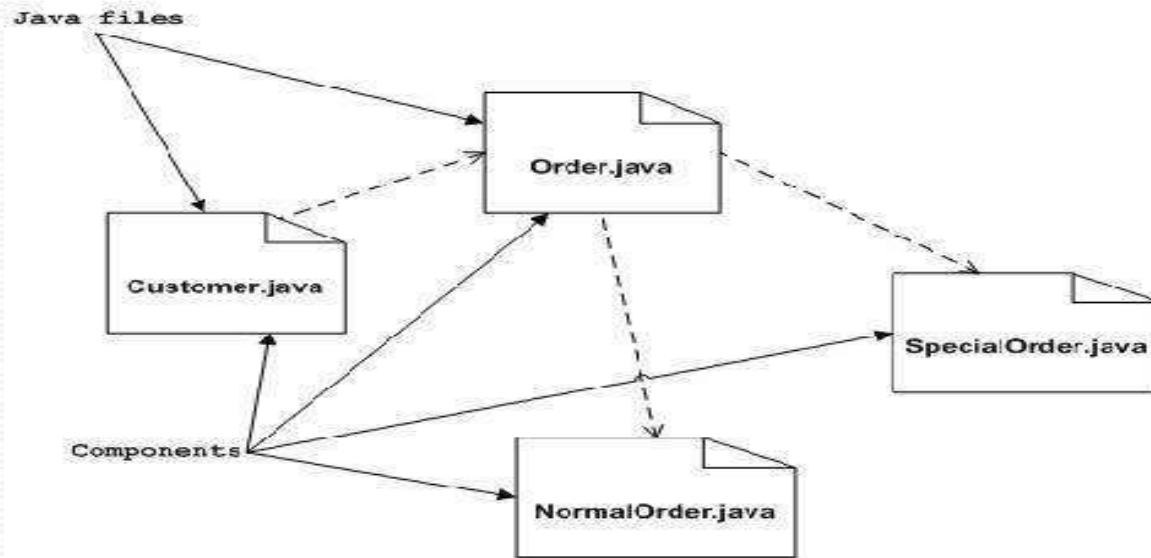
- Files used in the system.
- Libraries and other artifacts relevant to the application.
- Relationships among the artifacts.

After identifying the artifacts, the following points need to be kept in mind.

- Use a meaningful name to identify the component for which the diagram is to be drawn.
- Prepare a mental layout before producing the using tools.
- Use notes for clarifying important points.

Component Diagram

Component diagram of an order management system



Where to Use Component Diagrams?

- Component diagrams are used to visualize the static implementation view of a system.
- Component diagrams are special type of UML diagrams used for different purposes.
- These diagrams show the physical components of a system. To clarify it, we can say that component diagrams describe the organization of the components in a system.
- Organization can be further described as the location of the components in a system.
- These components are organized in a special way to meet the system requirements.
- Those components are libraries, files, executables, etc. Before implementing the application, these components are to be organized.
- This component organization is also designed separately as a part of project execution.

Where to Use Component Diagrams

Component diagrams are very important from implementation perspective. Thus, the implementation team of an application should have a proper knowledge of the component details

Component diagrams can be used to –

- Model the components of a system.
- Model the database schema.
- Model the executables of an application.
- Model the system's source code.

Deployment Diagrams

- Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed.
- Deployment diagrams are used to describe the static deployment view of a system.
- Deployment diagrams consist of nodes and their relationships.

Purpose of Deployment Diagrams

- Deployment diagrams are used for describing the hardware components, where software components are deployed. Component diagrams and deployment diagrams are closely related.
- Component diagrams are used to describe the components and deployment diagrams shows how they are deployed in hardware.
- UML is mainly designed to focus on the software artifacts of a system. However, these two diagrams are special diagrams used to focus on software and hardware components.
- Most of the UML diagrams are used to handle logical components but deployment diagrams are made to focus on the hardware topology of a system. Deployment diagrams are used by the system engineers.

The purpose of deployment diagrams can be described as –

Visualize the hardware topology of a system.

Describe the hardware components used to deploy software components.

Describe the runtime processing nodes.

How to Draw a Deployment Diagram?

- Deployment diagram represents the deployment view of a system. It is related to the component diagram because the components are deployed using the deployment diagrams. A deployment diagram consists of nodes.
- Nodes are nothing but physical hardware used to deploy the application.
- Deployment diagrams are useful for system engineers.

An efficient deployment diagram is very important as it controls the following parameters –

- Performance
- Scalability
- Maintainability
- Portability

How to Draw a Deployment Diagram?

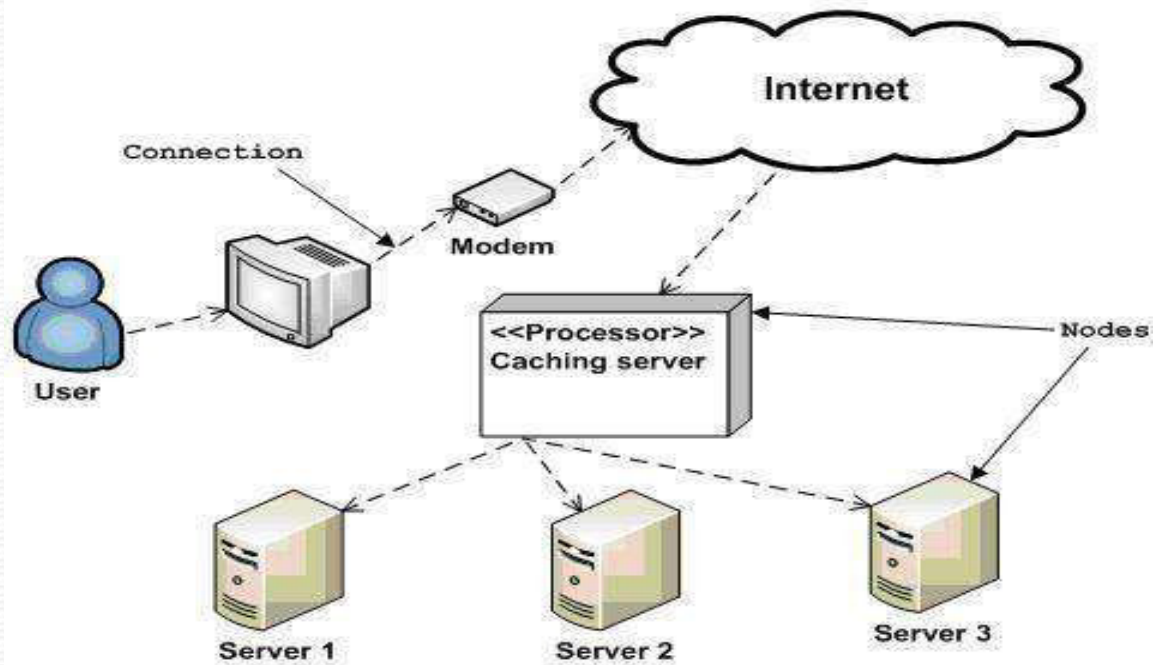
Before drawing a deployment diagram, the following artifacts should be identified –

- Nodes
- Relationships among nodes

Following is a sample deployment diagram to provide an idea of the deployment view of order management system. Here, we have shown nodes as –

- Monitor
- Modem
- Caching server
- Server

Deployment diagram of an order management system



Where to Use Deployment Diagrams?

- Deployment diagrams are mainly used by system engineers. These diagrams are used to describe the physical components (hardware), their distribution, and association.
- Deployment diagrams can be visualized as the hardware components/nodes on which the software components reside.
- Software applications are developed to model complex business processes. Efficient software applications are not sufficient to meet the business requirements. Business requirements can be described as the need to support the increasing number of users, quick response time, etc. To meet these types of requirements, hardware components should be designed efficiently and in a cost-effective way.
- Now-a-days software applications are very complex in nature. Software applications can be standalone, web-based, distributed, mainframe-based and many more. Hence, it is very important to design the hardware components efficiently.

Where to Use Deployment Diagrams?

Deployment diagrams can be used –

- To model the hardware topology of a system.
- To model the embedded system.
- To model the hardware details for a client/server system.
- To model the hardware details of a distributed application.
- For Forward and Reverse engineering.



CMRIT



SOFTWARE DESIGN AND ENGINEERING
II YEAR B.TECH. I – SEM.
UNIT-IV



TESTING STRATEGIES: A STRATEGIC APPROACH TO SOFTWARE TESTING, TEST STRATEGIES FOR CONVENTIONAL SOFTWARE, BLACK-BOX AND WHITE-BOX TESTING, VALIDATION TESTING, SYSTEM TESTING, THE ART OF DEBUGGING.

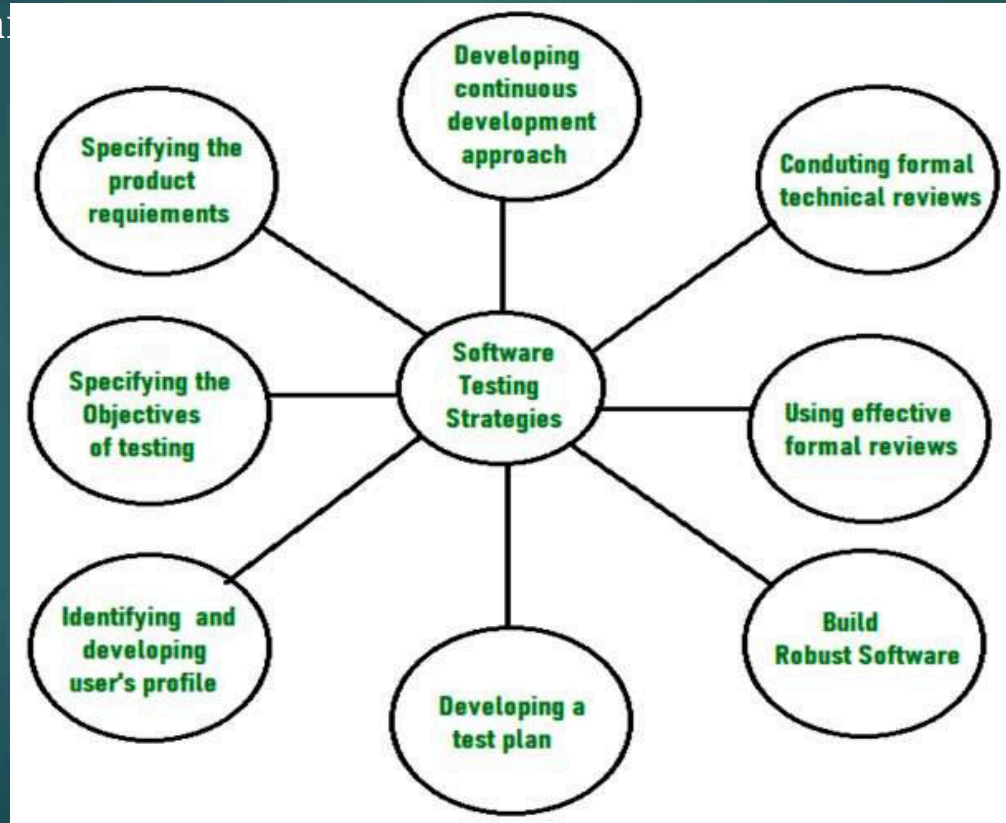
PRODUCT METRICS: SOFTWARE QUALITY, METRICS FOR ANALYSIS MODEL, METRICS FOR DESIGN MODEL, METRICS FOR SOURCE CODE, METRICS FOR TESTING, METRICS FOR MAINTENANCE.

METRICS FOR PROCESS AND PRODUCTS: SOFTWARE MEASUREMENT, METRICS FOR SOFTWARE QUALITY.

A strategic Approach for Software

testing:

Software Testing is a type of investigation to find out if there is any default or error present in the software.



Specify the requirements of the product

Before testing starts, it's necessary to identify and specify the requirements of the product in a quantifiable manner.

Different characteristics quality of the software is there such as

- Maintainability
 - Probability
 - usability
- . All these characteristic qualities should be specified in a particular order to obtain clear test results without any error.

Specifying the objectives of testing

Specifying the objectives of testing in a clear and detailed manner

- Effectiveness
- Any failure
- Cost of defects or errors

All these objectives should be clearly mentioned in the test plan.



Identifying and developing user's profile

Use cases describe the interactions and communication among different classes of users and the system to achieve the target. So as to identify the actual requirement of the users and then testing the actual use of the product.

Developing a test plan to give value and focus on rapid-cycle testing.

Rapid Cycle Testing is a type of test that improves quality by identifying and measuring the any changes that need to be required for improving the process of software. Therefore, a test plan is an important and effective document that helps the tester to perform rapid cycle testing.

Conduct formal technical reviews

The formal technical review helps in detecting any unfilled gap in the testing approach. Hence, it is necessary to evaluate the ability and quality of the test strategy and test cases by technical reviewers to improve the quality of software.

For the testing process, developing an approach for the continuous development.

As a part of a statistical process control approach, a test strategy that is already measured should be used for software testing to measure and control the quality during the development of software.

Robust software is developed that is designed to test itself

The software should be capable of detecting or identifying different classes of errors. Moreover, software design should allow automated and regression testing which tests the software to find out if there is any adverse or side effect on the features of software due to any change in code or program.

Before testing, using effective formal reviews as a filter

Formal technical reviews is technique to identify the errors that are not discovered yet. The effective technical reviews conducted before testing reduces a significant amount of testing efforts and time duration required for testing software so that the overall development time of software is reduced.

Testing Strategies for Conventional

Software:

▶ you can wait until the system is fully constructed and then conduct tests on the overall system in hopes of finding errors.

▶ you could conduct tests on a daily basis, whenever any part of the system is constructed.

Types:

- 1) Unit Testing
- 2) Integration Testing
- 3) Validation Testing and
- 4) System Testing

1) Unit Testing:

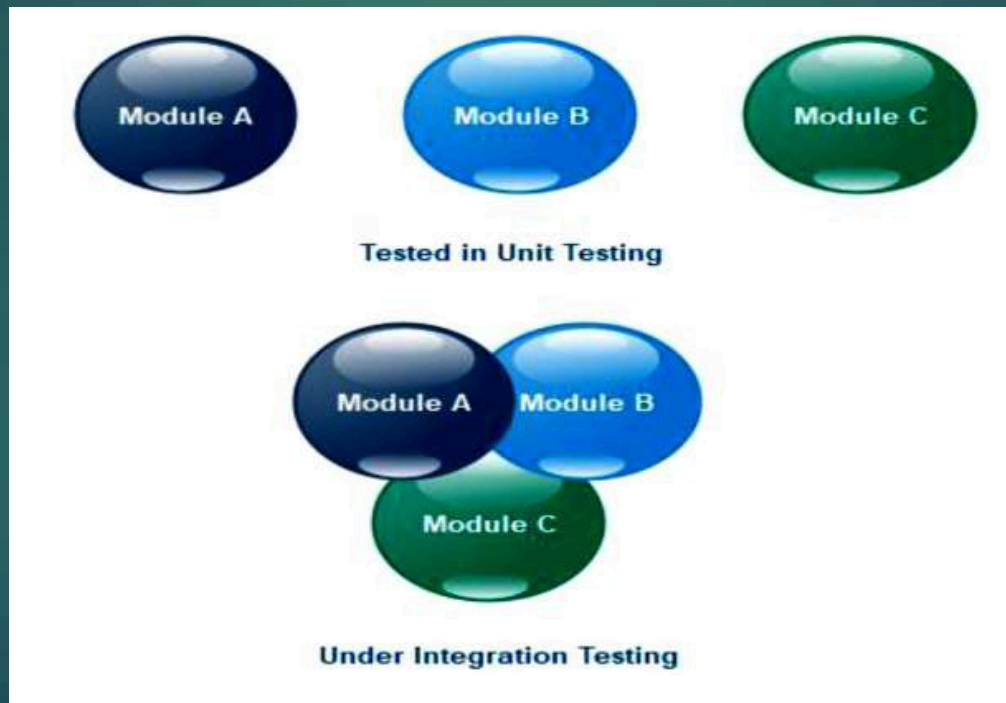
▶ individual units or components of a software are tested.

▶ done during the development (coding phase) of an application by the developers.

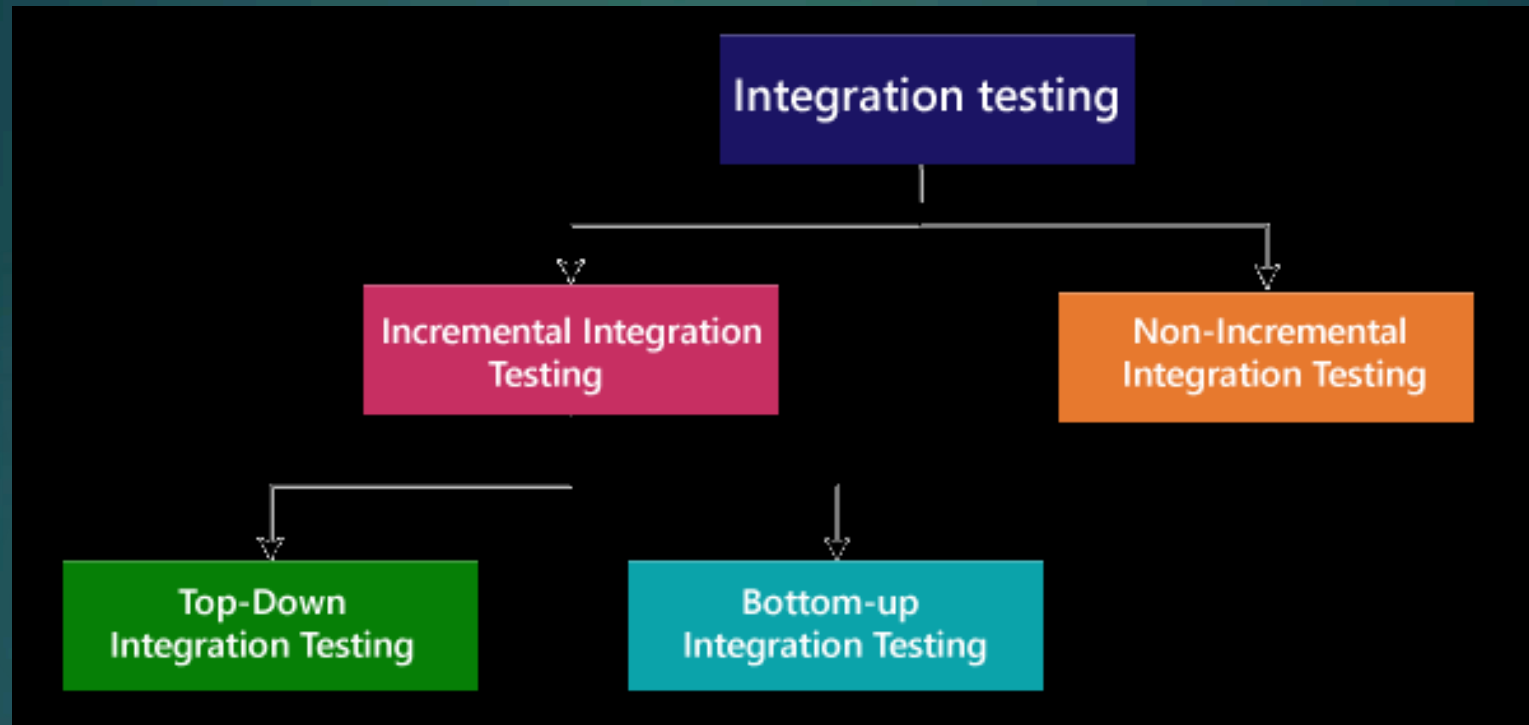
▶

2) Integration Testing:

- ▶ second level of the software testing process comes after unit testing .
- ▶ The focus of the integration testing level is to expose defects at the time of interaction between integrated components or units.



Types of Integration Testing :



Incremental Approach:

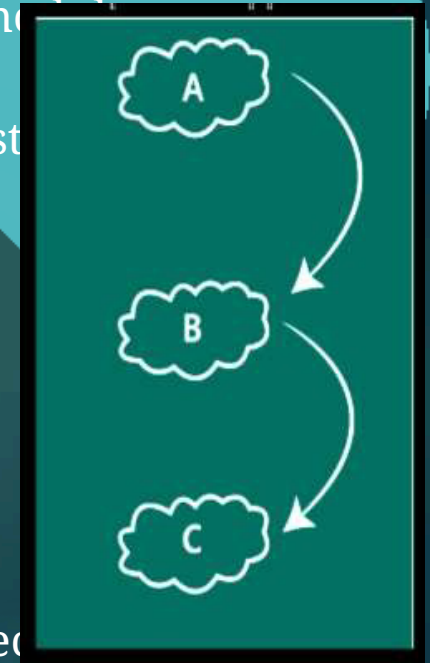
- ▶ modules are added in ascending order one by one or according to need .
- ▶ Generally, two or more than two modules are added and tested to determine the correctness of functions.
- ▶ The process continues until the successful testing of all the modules.

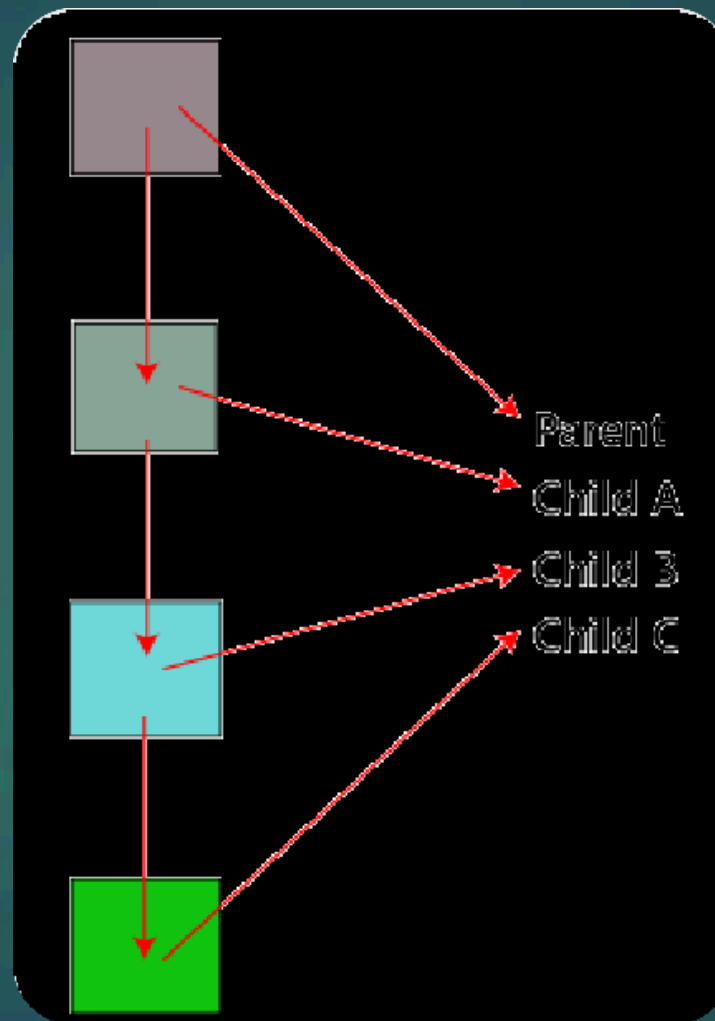
Top-Down Approach:

- ▶ deals with the process in which higher level modules are tested first and then lower level modules .
- ▶ Major design flaws can be detected and fixed early.
- ▶ the module we are adding is the **child of previous one**.

Disadvantages:

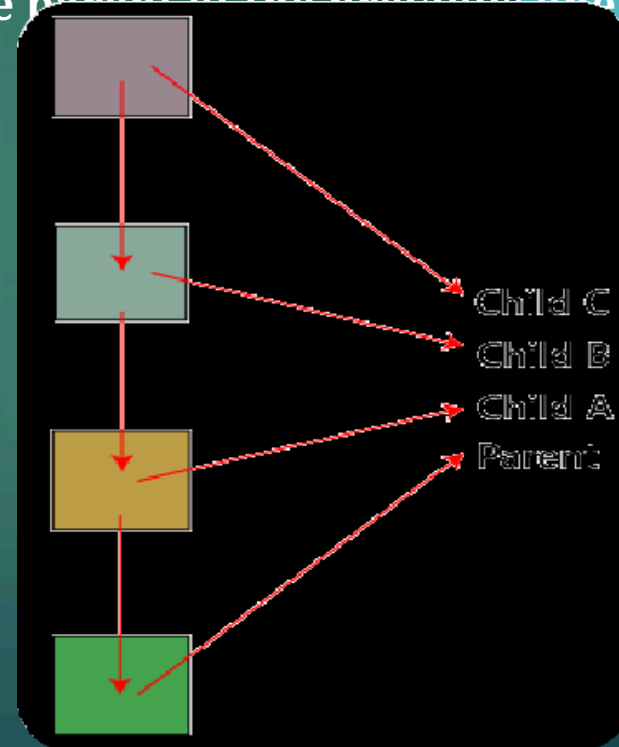
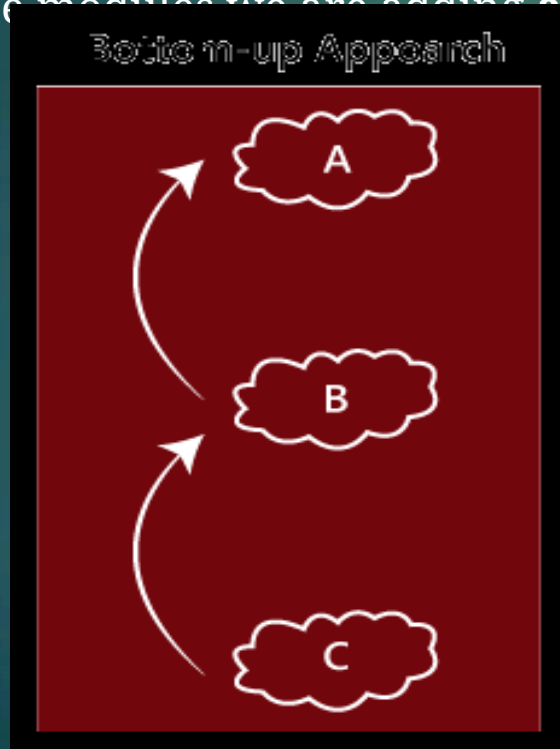
- ▶ Due to the high number of stubs, it gets quite complicated.
- ▶ Lower level modules are tested inadequately.
- ▶ Critical Modules are tested first so that fewer chances of defects.





Bottom-Up Method:

- ▶ the process in which lower level modules are tested with higher level modules.
- ▶ Top level critical modules are tested at last.
- ▶ the modules we are adding are the parent of the previous one.



- **Advantages**

Identification of defect is easy.

Do not need to wait for the development of all the modules as it saves time.

- **Disadvantages**

Critical modules are tested last due to which the defects can occur.

Hybrid Testing Method

- There is less possibility of occurrence of defect because each module interface is tested.

Advantages

The hybrid method provides features of both Bottom Up and Top Down methods.

It is most time reducing method.

It provides complete testing of all modules.

- **Disadvantages**

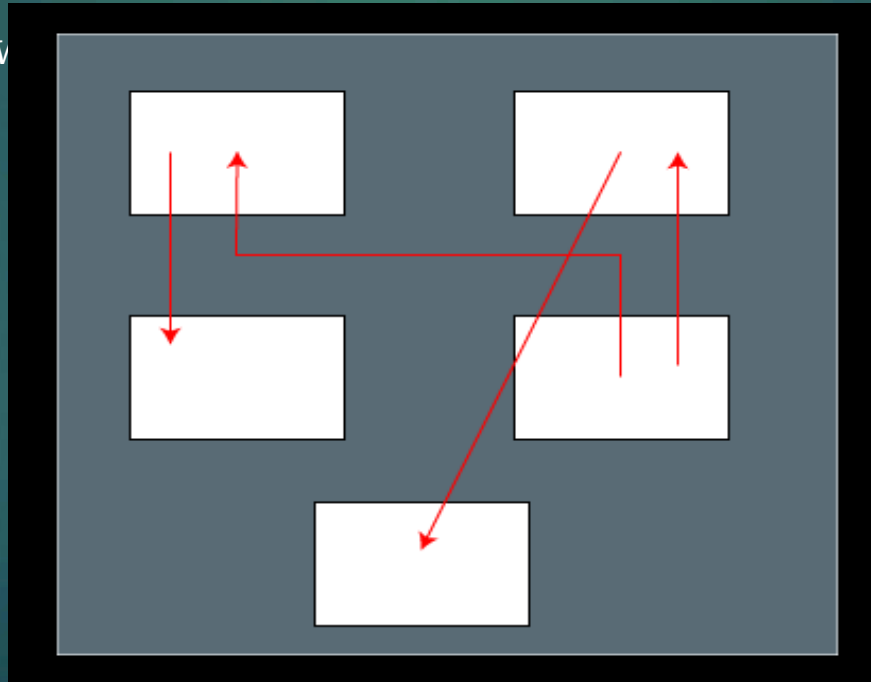
This method needs a higher level of concentration as the process carried out in both directions simultaneously.



Non- incremental integration testing :

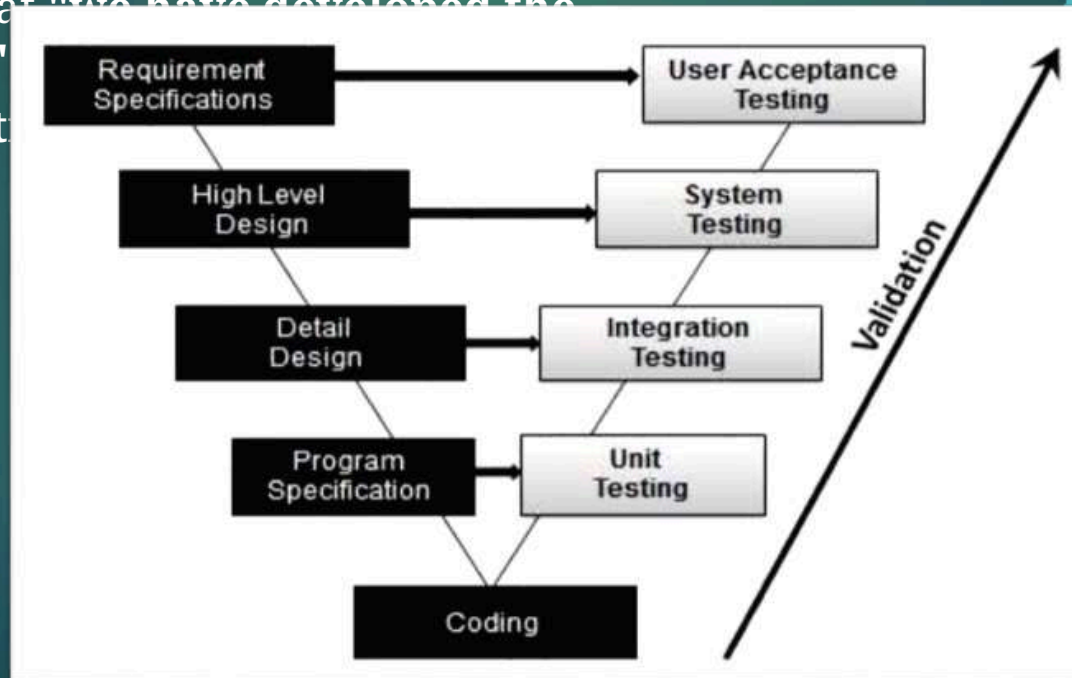
- ▶ when the data flow is very complex and when it is difficult to find who is a parent and who is a child.
- ▶ create the data in any module bang on all other existing modules and check if the data is present.

it is also known



3. Validation Testing:

- ▶ Tester performs functional and non-functional testing .
 - ▶ Validation testing is also known as dynamic testing.
 - ▶ ensuring that "we have developed the product right."
- Validation Test**



4. System Testing:

- ▶ includes testing of a fully integrated software system.
- ▶ To check the end-to-end flow of an application or the software as a user.

System Testing includes the following steps.

- ▶ Verification of input functions of the application to test whether it is producing the expected output or not.
- ▶ Testing of integrated software by including external peripherals to check the interaction of various components with each other.
- ▶ Testing of the whole system for End to End testing.
- ▶ Behavior testing of the application via a user's experience

Software Testing

Two major categories of software testing

- ▶ Black box testing
- ▶ White box testing

Black box testing

- ▶ functionalities of software applications are tested.
- ▶ mainly focuses on input and output of software applications.

Types of Black Box Testing

- ▶ Functional testing
- ▶ Non-functional testing
- ▶ Regression

Black Box Testing Techniques

- ▶ Equivalence Class Partitioning
- ▶ Boundary Value Analysis
- ▶ Decision Table Testing



Equivalence Partitioning Testing:

(ECP)

- ▶ can be applied to all levels of software testing like unit, integration, system
- ▶ divides the input test data of the application under test into each partition at least once of equivalent data from which test cases can be derived.

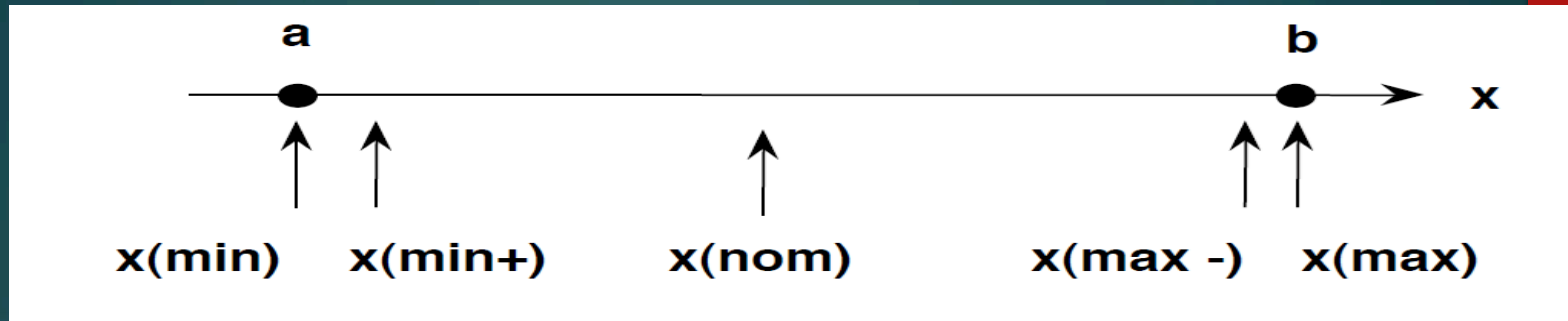
Example:

Assume that the application accepts an integer in the range 100 to 999

- ▶ Valid Equivalence Class partition: 100 to 999 inclusive.
- ▶ Non-valid Equivalence Class partitions: less than 100, more than 999, decimal numbers and alphabets/non-numeric characters.

Boundary Value Analysis:

- ▶ testing between extreme ends or boundaries between partitions of the input values.
- ▶ these extreme ends like **Start- End, Lower- Upper, Maximum-Minimum, Just Inside-Just Outside** values are called boundary values and the testing is called "boundary testing".



Example: Input Box should accept the Number 1 to

Test Scenario
Description

- ▶ Boundary Value = 0
- ▶ Boundary Value = 1
- ▶ Boundary Value = 2
- ▶ Boundary Value = 9
- ▶ Boundary Value = 10
- ▶ Boundary Value = 11

Expected

Outcome System

should NOT accept
System should accept
System should accept
System should accept
System should NOT
accept

Equivalence and Boundary Value:

- ▶ Pizza values 1 to 10 is considered valid. A success message is shown.
- ▶ While value 11 to 99 are considered invalid for order and an error message will appear, "Only 10 Pizza can be ordered"

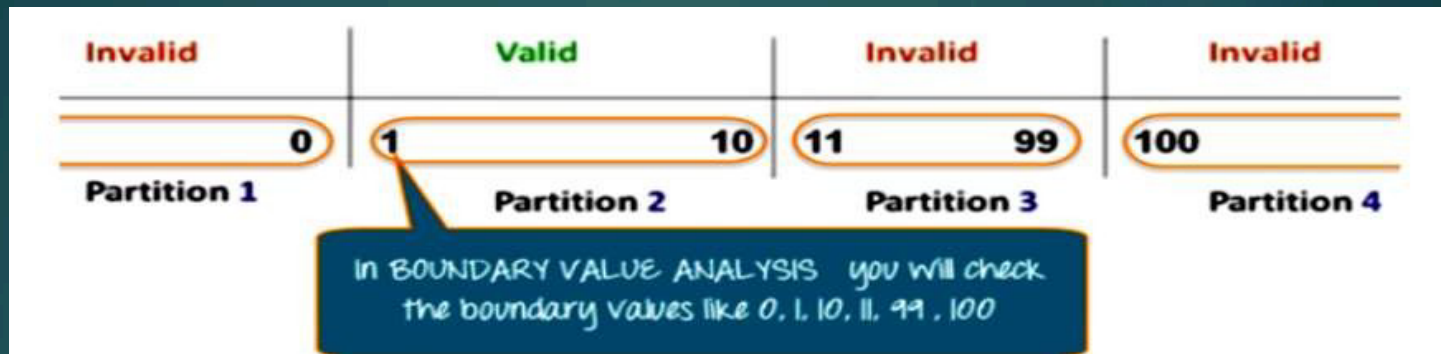
Order Pizza:

Here is the test condition

1. Any Number greater than 10 entered in the Order Pizza field(let say 11) is considered invalid.
2. Any Number less than 1 that is 0 or below, then it is considered invalid.
3. Numbers 1 to 10 are considered valid
4. Any 3 Digit Number say -100 is invalid.

Invalid	Valid	Invalid	Invalid
0	1	10	11
Partition 1	Partition 2	Partition 3	Partition 4

Equivalence Class
Partitioning



Boundary value analysis(BVA)

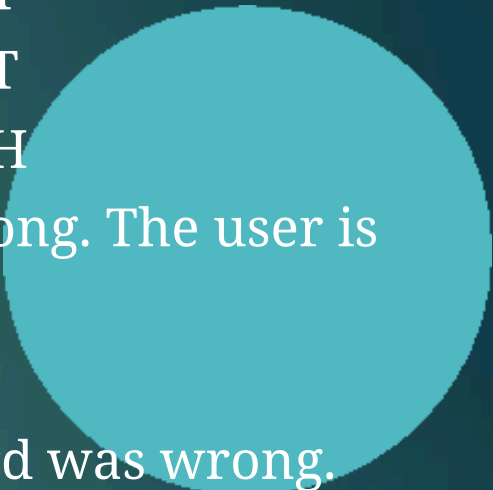
Decision Table :

- ▶ a tabular representation of inputs versus rules/cases/ test conditions.
- ▶ The conditions and values.

Email

Password

Log in



Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

- ▶ Case 1 – Username and password both were wrong. The user is shown an error message.
- ▶ Case 2 – Username was correct, but the password was wrong. The user is shown an error message.
- ▶ Case 3 – Username was wrong, but the password was correct. The user is shown an error message.
- ▶ Case 4 – Username and password both were correct, and the

White Box Testing:

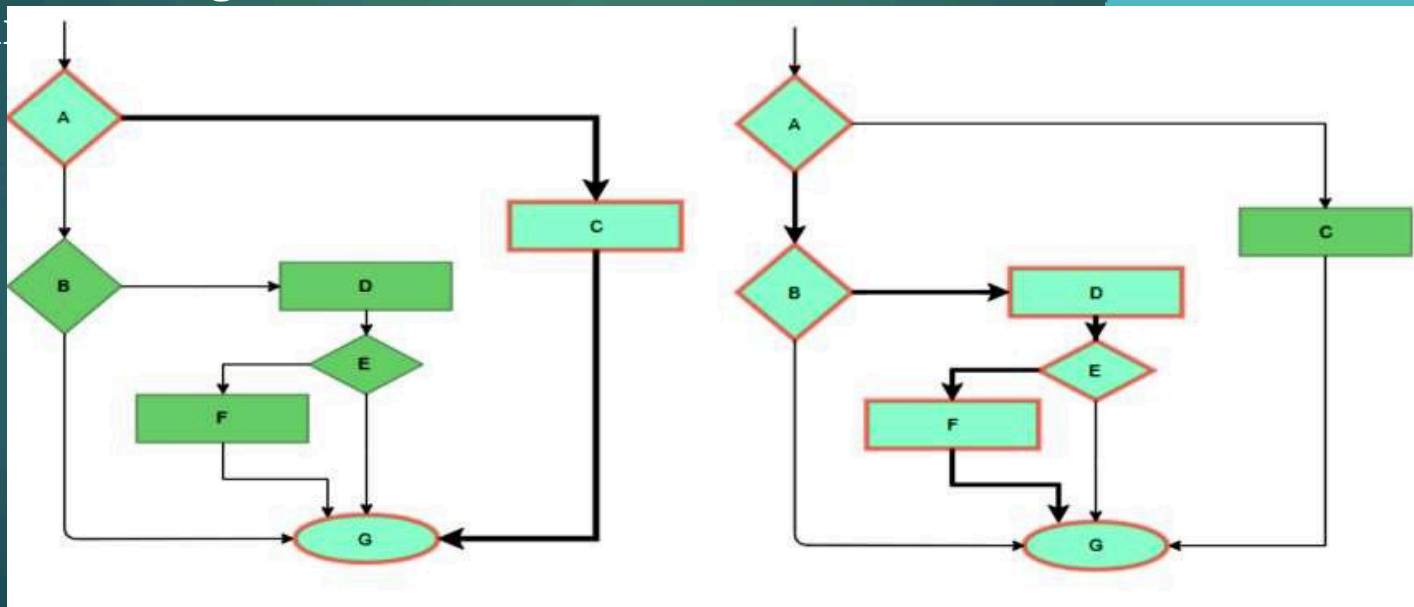
- ▶ examines the program structure and derives test data from the program logic/code.

White Box Testing Techniques:

- ▶ Statement Coverage
- ▶ Branch Coverage
- ▶ Path Coverage

Statement coverage:

- ▶ the a



```
1 ▾ Prints (int a, int b) {  
2   int result = a+ b;  
3   If (result> 0)  
4     Print ("Positive", result)  
5   Else  
6     Print ("Negative", result)  
7 }  
8
```

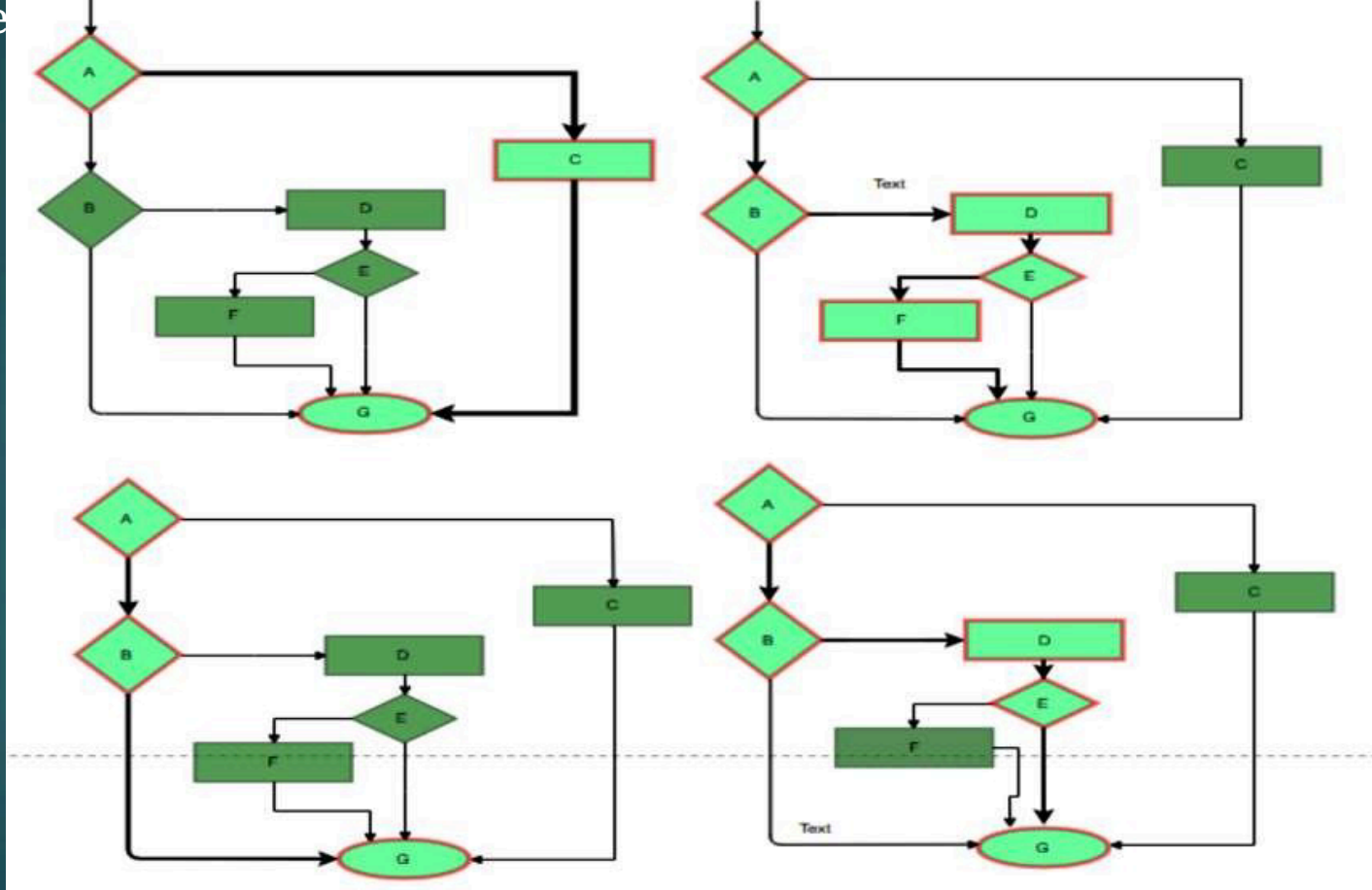
Number of executed
statements = 5 Total number
of statements = 7 Statement
Coverage: $5/7 = 71\%$

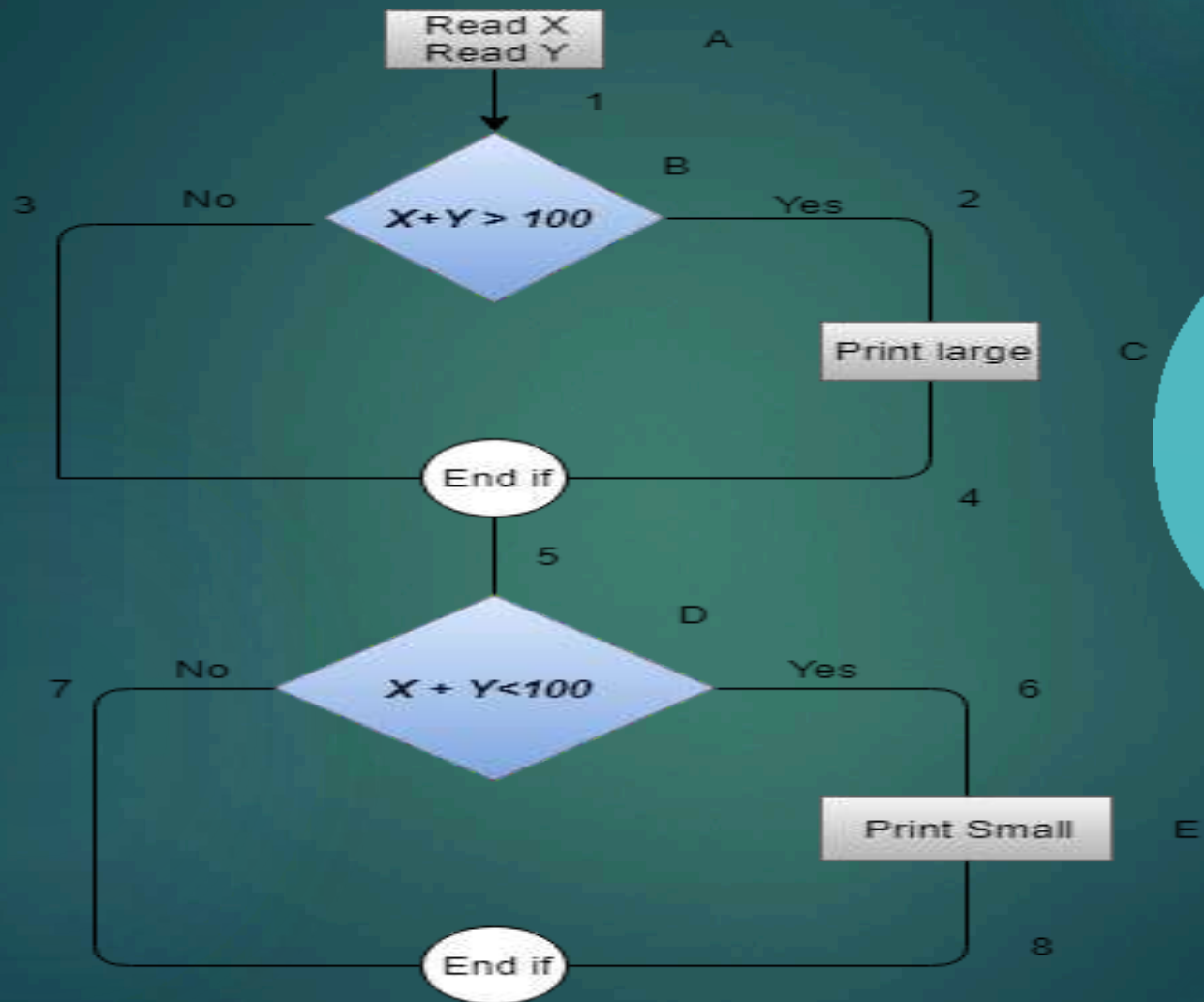
```
1 ▾ Prints (int a, int b) {  
2   int result = a+ b;  
3   If (result> 0)  
4     Print ("Positive", result)  
5   Else  
6     Print ("Negative", result)  
7 }  
8
```

Number of executed
statements = 6 Total number
of statements = 7 Statement
Coverage: $6/7 = 85\%$

Branch Coverage:

- ▶ test cases are designed so that each branch from all decision points are traverse





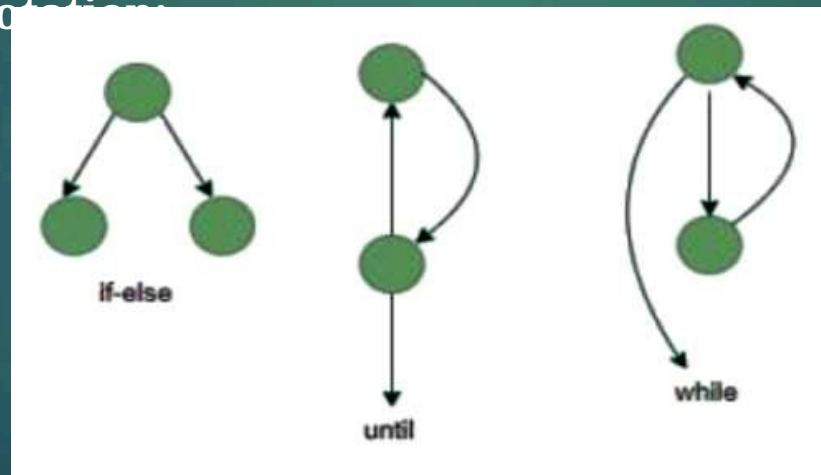
Basis Path Testing:

- ▶ control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths.

Steps:

1. Make the corresponding control flow graph
2. Calculate the cyclomatic complexity
3. Find the independent paths
4. Design test cases corresponding to each independent path

Flow graph notations



Cyclomatic Complexity:

- ▶ measure the logical complexity of the software and is used to define the number of independent paths.

- ▶ For a graph G, $V(G)$ is its cyclomatic complexity

- ▶ $V(G) = E - N + 2$, where E is the number of edges
N is the total number of nodes

$V(G) = 4$, No of independent paths = 4

#P1: 1 – 2 – 4 – 7 – 8

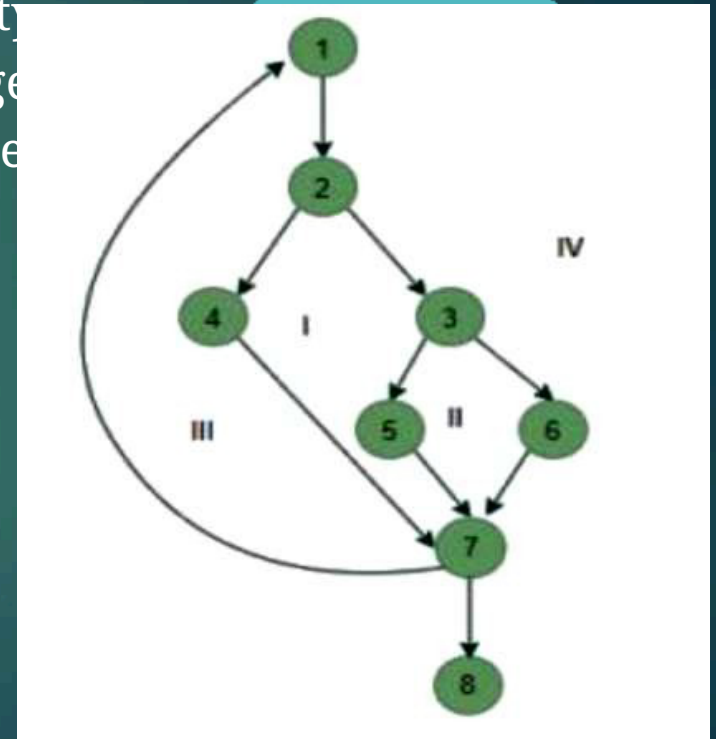
#P2: 1 – 2 – 3 – 5 – 7 – 8

#P3: 1 – 2 – 3 – 6 – 7 – 8

#P4: 1 – 2 – 4 – 7 – 1 – . . . – 7 – 8

Loop Testing:

- ▶ Simple loops
- ▶ Nested loop
- ▶ Concatenated loops



Art of debugging

Introduction:

Debugging is the process of fixing a bug in the software.

It refers to identifying, analyzing and removing errors.

This activity begins after the software fails to execute properly and concludes by solving the problem and successfully testing the software.

It is considered to be an extremely complex and tedious task because errors need to be resolved at all stages of debugging.

Debugging Process:

Steps involved in debugging are:

- ❑ Problem identification and report preparation.
- ❑ Assigning the report to software engineer to the defect to verify that it is genuine.
- ❑ Defect Analysis using modeling, documentations, finding and testing candidate flaws, etc.
- ❑ Defect Resolution by making required changes to the system.
- ❑ Validation of corrections

Debugging Strategies:

- Study the system for the larger duration in order to understand the system.
- Backwards analysis of the problem which involves tracing the program backward from the location of failure message in order to identify the region of faulty code.
- Forward analysis of the program involves tracing the program forwards using breakpoints or print statements at different points in the program and studying the results. The region where the wrong outputs are obtained is the region that needs to be focused to find the defect.
- Using the past experience of the software debug the software with similar problems in nature. The success of this approach depends on the expertise of the debugger.

Debugging Tools:

Debugging tool is a computer program that is used to test and debug other programs. A lot of public domain software like gdb and dbx are available for debugging. They offer console-based command line interfaces. Examples of automated debugging tools include code based tracers, profilers, interpreters, etc.

Some of the widely used debuggers are: [Radare2](#)
[WinDbg](#) [Valgrind](#)

Software Quality:

- ▶ Software quality product is defined in term of its fitness of purpose.
- ▶ a quality product does precisely what the users want it to do.

Several quality methods:

- ▶ Portability
- ▶ Usability
- ▶ Reusability
- ▶ Correctness
- ▶ Maintainability

Various metrics formulated for products in the development process are listed below.

- ▶ Metrics for analysis model
- ▶ Metrics for design model
- ▶ Metrics for source code
- ▶ Metrics for testing
- ▶ Metrics for maintenance

Metrics for the Analysis Model:

- ▶ used to examine the analysis model with the objective of predicting the size of the resultant system.
- ▶ Function point and lines of code are the commonly used methods for size estimation.

1. Function Point (FP) Metric:(A.J Albrecht)

- ▶ measure the functionality delivered by the system, estimate the effort, predict the number of errors, and estimate the number of components in the system.
- ▶ Function point is derived by using a relationship between the complexity of software and the information domain value.

2. Lines of Code (LOC) :

- ▶ most widely used methods for size estimation.
- ▶ defined as the number of delivered lines of code, excluding comments



Metrics for Software Design:

- ▶ Various design metrics such as architectural design metrics, component-level design metrics, user-interface design metrics, and metrics for object-oriented design are used to indicate the complexity, quality, and so on of the software design

1. Architectural Design Metrics:

Focus on the features of the program architecture with stress on architectural structure and effectiveness

of components (or modules) within the architecture.

Three software design complexity measures are defined, namely, structural complexity, data complexity,

and system complexity.

The complexity of a system increases with increase in structural complexity, data complexity, and system complexity.

module 'j', **structural complexity** is calculated by the following equation.

$$S(j) = f^2 \text{ out}(j)$$

Where $f \text{ out}(j)$ = fan-out of module 'j' (number of modules that are subordinating module j)

Complexity in the internal interface for a module 'j' is indicated with the help of **data**

complexity, which is calculated by the following equation.

$$D(j) = V(j) / [f_{\text{out}}(j) + 1]$$

Where $V(j)$ = number of input and output variables passed to and from module 'j'.

System complexity is the sum of structural complexity and data complexity and is calculated by the following equation.

$$C(j) = S(j) + D(j)$$

2. Component-level design:

The software engineer can judge the quality of the component-level design by measuring module cohesion, coupling and complexity;

Cohesion metrics

- Data slice
- Data tokens
- Glue tokens
- Superglue tokens
- Stickiness

Coupling Metrics:

A metric for module coupling has been proposed, which includes data and control flow coupling, global coupling, and environmental coupling.

- **Measures defined for data and control flow coupling are**

d_i = total number of input data parameters

c_i = total number of input control parameters d_o = total number of output data parameters c_o = total number of output control parameters

§ Measures defined for global coupling are

gc = number of global variables utilized as data

gc = number of global variables utilized as control

• § Measures defined for environmental coupling are

w = number of modules

r = number of modules calling the modules under consideration By using the above mentioned

measures,

module-coupling indicator (mc) is calculated by using the following equation. $mc =$

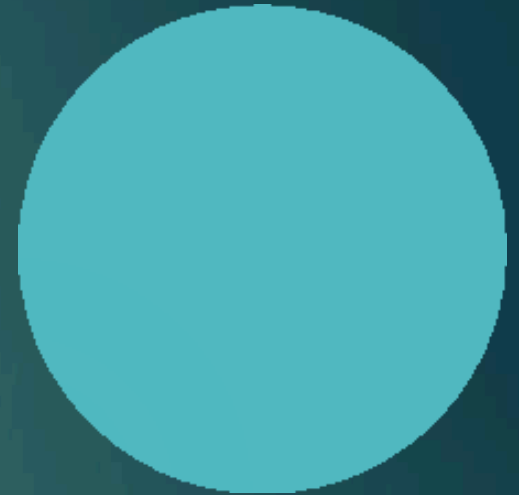
K/M Where K = proportionality constant

$M = di + (a*ci) + do + (b*co) + gd + (c*gc) + w + r$.

Note that K , a , b , and c are empirically derived. The values of mc and overall module coupling are inversely proportional to each other. In other words, as the value of mc increases, the overall module coupling decreases.

Metrics for user interface design

1.layout appropriateness
metric 2.cohesion
metrics



4. Metrics for Object-oriented

Design:

In order to develop metrics for object-oriented (OO) design, some distinct and measurable characteristics of OO design are considered, which are listed below.

- Complexity
- Coupling
- Sufficiency
- Cohesion
- Primitiveness
- Similarity
- Volatility
- Size

Metrics for Coding:

▶ Hairstad proposed the first analytic laws for computer science by using a set of primitive measures,

which can be derived once the code is generated.

▶ Program length (N) can be calculated by using the following equation.

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2.$$

▶ Program volume (V) can be calculated by using the following equation.

$$V = N \log_2 (n_1 + n_2).$$

▶ n_1 = number of distinct operators in a program

▶ n_2 = number of distinct operands in a program

▶ N_1 = total number of operators

▶ N_2 = total number of operands.

Volume ratio can also be calculated by using the following equation.

$$L = (2/n_1) * (n_2/N_2).$$

Program difficulty level (D) and effort (E) can be calculated by using the following equations.

$$D = (n_1/2) * (N_2/n_2). \quad E = D * V.$$

Metrics for Software Testing:

- ▶ Generally, testers use metrics for analysis, design, and coding to guide them in design and execution of test cases.
- ▶ Function point can be effectively used to estimate testing effort.
- ▶ Metrics used for architectural design can be used to indicate how integration testing can be carried out.
- ▶ In addition, cyclomatic complexity can be used effectively as a metric in the basis-path testing to determine the number of test cases needed.
- ▶ Halstead measures can be used to derive metrics for testing effort.
By using program volume (V) and program level (PL), Halstead effort (e) can be calculated by the following equations.

$$e = V / PL \text{ Where } PL = 1 / [(n1/2) * (N2/n2)] \dots (1)$$

- ▶ For a particular module (z), the percentage of overall testing effort

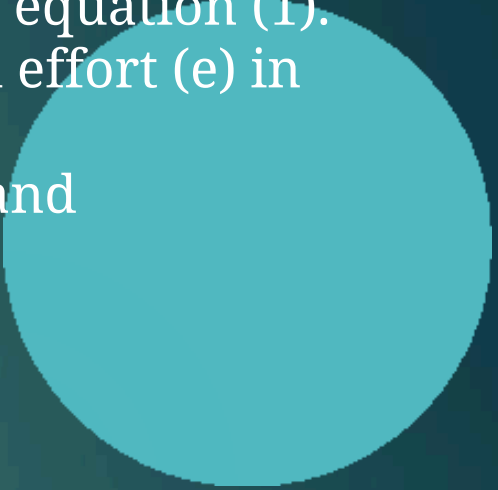


Percentage of testing effort (z) = $e(z)/\sum e(i)$

Where, $e(z)$ is calculated for module z with the help of equation (1).
Summation in the denominator is the sum of Halstead effort (e) in all the modules of the system

While developing metrics for OO testing, inheritance and encapsulation are also considered.

A set of metrics proposed for OO testing is

- Percent public and protected (PAP)
 - Public access to data members (PAD)
 - Number of root classes (NOR)
 - Fan-in (FIN)
- 

Metrics for Software Maintenance:

- ▶ IEEE have proposed Software Maturity Index (SMI), which provides indications relating to the stability of software product. For calculating SMI, following parameters are considered.
 - ▶ Number of modules in current release (MT)
 - ▶ Number of modules that have been changed in the current release (Fe)
 - ▶ Number of modules that have been added in the current release (Fa)
 - ▶ Number of modules that have been deleted from the current release (Fd)
- ▶ Once all the parameters are known, SMI can be calculated by using the following equation.

$$SMI = [MT - (Fa + Fe + Fd)] / MT.$$

Metrics for Process and Products

A measurement is a manifestation of the size, quantity, amount or dimension of a particular attributes of a product or process.

Software measurement process is defined and governed by ISO Standard.

Need of Software Measurement:

1. Create the quality of the current product or process.
2. Anticipate future qualities of the product or process.
3. Enhance the quality of a product or process.
4. Regulate the state of the project in relation to budget and schedule.

Classification of Software Measurement:

1. **Direct Measurement:** the product, process or thing is measured directly using standard scale.
2. **Indirect Measurement:** the quantity or quality to be measured is measured using

Metrics:

A metrics is a measurement of the level that any impute belongs to a system product or process.

There are 4 functions related to software metrics:

1. Planning
2. Organizing
3. Controlling
4. Improving

Characteristics of software Metrics:

1. Quantitative
2. Understandable
3. Applicability
4. Repeatable
5. Economical
6. Language Independent

Metrics for software quality:

- ▶ Software quality metrics are a subset of software metrics that focus on the quality aspects of the product, process, and project.

Software quality metrics can be further divided into three categories –

- ▶ Product quality metrics
- ▶ In-process quality metrics
- ▶ Maintenance quality metrics



Product Quality Metrics:

- ▶ This metrics include the following –
 - ▶ Mean Time to Failure
 - ▶ Defect Density
 - ▶ Customer Problems
 - ▶ Customer Satisfaction

Mean Time to Failure

- ▶ It is the time between failures.

Defect Density

- ▶ It measures the defects relative to the software size expressed as lines of code or function point.

Customer Problems

- ▶ It measures the problems that customers encounter when using the product.

PUM = Total Problems that customers reported (true defect and non-defect oriented problems) for a time period + Total number of license months of the software during the period

Number of license-month of the software = Number of install license of the software ×

Number of months in the calculation period

Customer

Satisfaction Customer satisfaction is often measured by customer survey data through the five-point scale

Very satisfied Satisfied Neutral Dissatisfied Very dissatisfied



In-process Quality Metrics:

▶ In-process quality metrics deals with the tracking of defect arrival during formal

machine testing for some organizations. This metric includes –

- ▶ Defect density during machine testing
- ▶ Defect arrival pattern during machine testing
- ▶ Phase-based defect removal pattern
- ▶ Defect removal effectiveness

$$\text{DRE} = \frac{\text{Defect removed during a development phase}}{\text{Defects latent in the product}} \times 100\%$$

Maintenance Quality Metrics:

▶ Following are the fixes that can be carried out to eliminate the defects as soon as

possible with excellent fix quality.

- ▶ Fix backlog and backlog management index
- ▶ Fix response time and fix responsiveness
- ▶ Percent delinquent fixes
- ▶ Fix quality

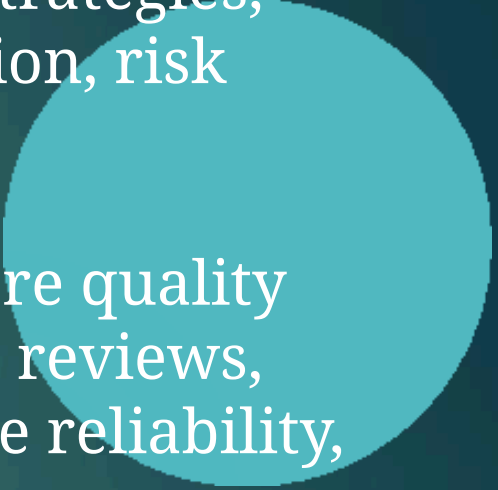


CMRIT

SOFTWARE DESIGN AND ENGINEERING

II YEAR B.TECH. I- SEM

UNIT-V



Risk management: Reactive Vs proactive risk strategies, software risks, risk identification, risk projection, risk refinement, RMMM, RMMM plan.

Quality Management: Quality concepts, software quality assurance, software reviews, formal technical reviews, statistical software quality assurance, software reliability, the ISO 9000 quality standards.

RISK MANAGEMENT

REACTIVE VS. PROACTIVE RISK STRATEGIES

REACTIVE RISK STRATEGIES (Fire Fighting Mode) Project team reacts to risks when they occur .

Mitigation—Plan for additional resources in anticipation of fire fighting.

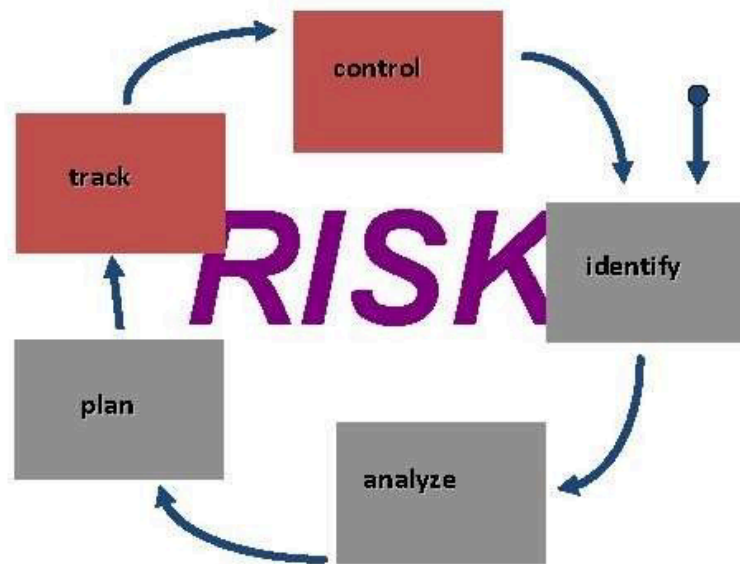
Fix on failure—Resource are found and applied when the risk strikes. Crisis management—Failure does not respond to applied resources.



PROACTIVE RISK STRATEGIES

- ▶ A proactive strategy begins long before technical work is initiated.
- ▶ Potential risks are identified, their probability and impact are assessed, and they are ranked by importance.
- ▶ Then, the software team establishes a plan for managing risk.
 1. Formal risk analysis is performed
 2. Organization corrects the root causes of risk
 1. Examining risk sources that lie beyond the bounds of the software.
 2. Developing the skill to manage change.

Risk Management Paradigm



SOFTWARE RISK

Risk always involves two characteristics

- ▶ Uncertainty
- ▶ Loss

Different categories of risks are considered

1. Project risks
2. Technical risks
3. Business risks
 1. Market risk
 2. Strategic risk
 3. Management risk
 4. Budget risks
4. Known risks
5. Predictable risks and unpredictable risks



RISK IDENTIFICATION

Risk identification is a systematic attempt to specify threats to the project plan. There are two distinct types of risks.

- ▶ 1) Generic risks and
- ▶ 2) product-specific risks

Known and predictable risks in the following generic subcategories:

- ▶ Product size
- ▶ Business impact
- ▶ Customer characteristics
- ▶ Development environment
- ▶ Technology to be built
- ▶ Staff size and experience



Assessing Overall Project Risk

The questions are ordered by their relative importance to the success of a project.

- ▶ 1. Have top software and customer managers formally committed to support the project?
- ▶ 2. Are end-users enthusiastically committed to the project and the system/product to be built?
- ▶ 3. Are requirements fully understood by the software engineering team and their customers?
- ▶ 4. Have customers been involved fully in the definition of requirements?
- ▶ 5. Do end-users have realistic expectations?
- ▶ 6. Is project scope stable?
- ▶ 7. Does the software engineering team have the right mix of skills?
- ▶ 8. Are project requirements stable?
- ▶ 9. Does the project team have experience with the technology to be Implemented?
- ▶ 10. Is the number of people on the project team adequate to do the job?

▶

Risk Components and Drivers

The risk components are defined in the following manner:

- ▶ • Performance risk
- ▶ • Cost risk
- ▶ • Support risk
- ▶ • Schedule risk

The impact of each risk driver on the risk component is divided into one of four impact categories—negligible, marginal, critical and catastrophic.

RISK PROJECTION (Risk Estimation)

Attempts to rate each risk in two ways

The likelihood or probability that the risk is real and

The consequences of the problems associated with the risk, should it occur.

The project planner, along with other managers and technical staff, performs four risk projection activities:

- (1) Establish a scale that reflects the perceived likelihood of a risk,
- (2) Delineate the consequences of the risk,
- (3) Estimate the impact of the risk on the project and the product, and
- (4) Note the overall accuracy of the risk projection so that there will be no misunderstandings.

Developing a Risk Table Building a Risk

T1 Performance
T2 Support
T3 Cost
T4 Schedule

Example of Risk Table

Risk Summary	Risk Category	Risk Probability (In %)	Impact(Scale 1-4)	Mitigation
Computer Failure	T1	20%	1	Plan regular backup-storage in Cloud

IMPACT
1- CATASTROPHIC
2- CRITICAL
3- MARGINAL
4- NEGLIGIBLE



Assessing Risk Impact

Three factors affect the consequences that are likely if a risk does occur:

1. its nature,
2. its scope, and
3. its timing.

The overall Risk Exposure, RE, is determined using the following relationship

$$RE = P \times C$$

Risk Identification. Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Risk probability. 80% (likely).

Risk impact. 60 reusable software components were planned.

Risk exposure. $RE = 0.80 \times 25,200 \sim \$20,200$. The total risk exposure for all risks (above the cut off in the risk table) can provide a means for adjusting the final cost estimate for a project etc.

RISK REFINEMENT

One way for risk refinement is to represent the risk in condition-transition-consequence (CTC) format.

This general condition can be refined in the following manner:

Sub condition 1. Certain reusable components were developed by a third party with no knowledge of internal design standards.

Sub condition 2. The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.

Sub condition 3. Certain reusable components have been implemented in a language that is not supported on the target environment

RISK MITIGATION, MONITORING, AND MANAGEMENT

An effective strategy must consider three issues:

- **Risk avoidance**
- **Risk monitoring**
- **Risk management and contingency planning .**

If a software team adopts a proactive approach to risk, avoidance is always the best strategy.

To **Mitigate** possible steps to be taken are

- ❖ Meet with current staff to determine causes for turnover (e.g., poor working conditions, low pay, and competitive job market).
 - ❖ Mitigate those causes that are under our control before the project starts.
 - ❖ Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
 - ❖ Organize project teams so that information about each development activity is widely dispersed.
 - ❖ Define documentation standards and establish mechanisms to be sure that documents are developed in a timely manner.



As the project proceeds, risk monitoring activities commence. The following factors can be monitored:

- ❑ General attitude of team members based on project pressures.
 - ❑ The degree to which the team has jelled.
 - ❑ Interpersonal relationships among team members.
 - ❑ Potential problems with compensation and benefits.
 - ❑ The availability of jobs within the company and outside it.
- 

Software safety and hazard analysis

THE RMMM PLAN

A risk management strategy can be included in the software project plan or the risk management steps can be organized into a separate Risk Mitigation, Monitoring and Management Plan. The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan.

Risk monitoring is a project tracking activity with three primary objectives:

- 1) to assess whether predicted risks do, in fact, occur;
- 2) to ensure that risk aversion steps defined for the risk are being properly applied; and
- 3) to collect information that can be used for future risk analysis.

QUALITY MANAGEMENT

1. Quality

Quality of design

Quality of conformance

User satisfaction = compliant product + good quality + delivery within budget and schedule.

2. Quality Control 3. Quality Assurance 4. Cost of Quality

Quality costs may be divided into costs associated with prevention, appraisal, and failure.

4.1 Prevention costs include

- ✓ quality planning
- ✓ formal technical reviews
- ✓ test equipment
- ✓ training

4.2 Appraisal costs include activities to gain insight into product condition the “first time through” each process.

Examples of appraisal costs include

- ✓ in-process and inter process inspection
- ✓ equipment calibration and maintenance
- ✓ testing

4.3 Failure costs are those that would disappear if no defects appeared before shipping a product to customers.

Failure costs may be subdivided into internal failure costs and external failure costs.

Internal failure costs are incurred when we detect a defect in our product prior to shipment. 4.3.1. Internal failure costs include

- ✓ rework
- ✓ repair
- ✓ failure mode analysis

4.3.2. External failure costs are associated with defects found after the product has been shipped to the customer.

Examples of external failure costs are

- ✓ complaint resolution
- ✓ product return and replacement
- ✓ helpline support

SOFTWARE QUALITY ASSURANCE

The software quality definition serves to emphasize three important points:

- 1) Software requirements are the foundation from which quality is measured. Lack of conformance to requirements is lack of quality.
- 2) Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.
- 3) A set of implicit requirements often goes unmentioned (e.g., the desire for ease of use and good maintainability).

If software conforms to its explicit requirements but fails to meet implicit requirements, software quality is suspect.

Background Issues

1. The first formal quality assurance and control function was introduced at Bell Labs in 1916 and spread rapidly throughout the manufacturing world.
2. During the 1940s, more formal approaches to quality control were suggested. These relied on measurement and continuous process improvement as key elements of quality management. Today, every company has mechanisms to ensure quality in its products.
3. During the early days of computing (1950s and 1960s), quality was the sole responsibility of the programmer. Standards for quality assurance for software were introduced during the 1970s.

SQA (software quality assurance)

Software quality assurance is composed of a variety of tasks associated with two different constituencies—

1. The software engineers who do technical work and
2. SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

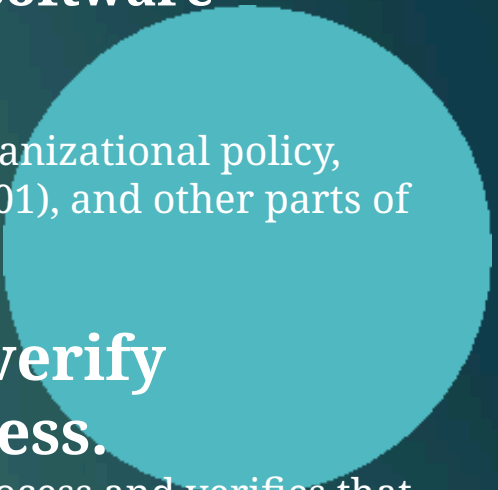
The Software Engineering Institute recommends a set of SQA activities that address quality assurance planning, oversight, record keeping, analysis, and reporting.

These activities are performed (or facilitated) by an independent SQA group that conducts the following activities.

1. Prepares an SQA plan for a project.

The plan is developed during project planning. Quality assurance activities performed by the software engineering team. The plan identifies

- ✓ Evaluations to be performed
- ✓ Audits and reviews to be performed
- ✓ Standards that are applicable to the project
- ✓ Procedures for error reporting and tracking
- ✓ Documents to be produced by the SQA group



2. Participates in the development of the project's software process description.

The software team selects a process for the work to be performed.

The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

3. Reviews software engineering activities to verify compliance with the defined software process.

The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

4. Audits designated software work products to verify compliance with those defined as part of the software process.

The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to

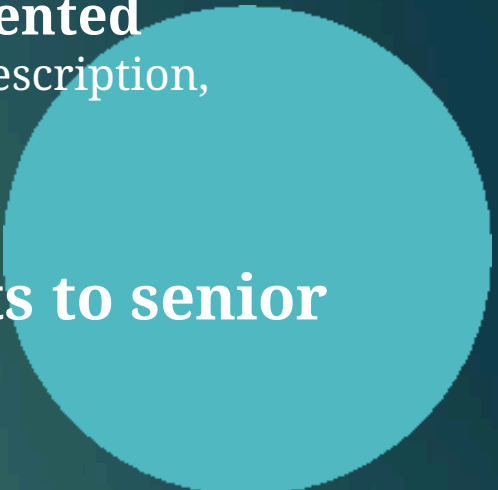


5. That deviations in software work and work products are documented and handled according to a documented procedure.

Procedures may be encountered in the project plan, process description, applicable standards, or technical work products.

6. Records any noncompliance and reports to senior management.

Noncompliance items are tracked until they are resolved.



SOFTWARE REVIEWS

Software reviews are a "filter" for the software engineering process. That is, reviews are applied at various points during software development and errors and defects that can then be removed.

Software reviews "purify" the software engineering activities that we have called analysis, design, and coding.

Types of reviews

- ✓ Formal reviews
- ✓ Informal reviews

Cost Impact of Software Defects:

What is error and what is defect?

A number of industry studies indicate that design activities introduce between 50 and 65 percent of all errors.

By detecting and removing a large percentage of these errors, the review process substantially

FORMAL TECHNICAL REVIEWS (FTR)

A formal technical review is a software quality assurance activity performed by software engineers (and others).

The objectives of the FTR are

- (1) to uncover errors in function, logic, or implementation for any representation of the software;
- (2) to verify that the software under review meets its requirements;
- (3) to ensure that the software has been represented according to predefined standards;
- (4) to achieve software that is developed in a uniform manner; and
- (5) to make projects more manageable

The Review Meeting

Review Reporting and Record Keeping

At the end of the review meeting and a review issues list is produced. In addition, a formal technical review summary report is completed. A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

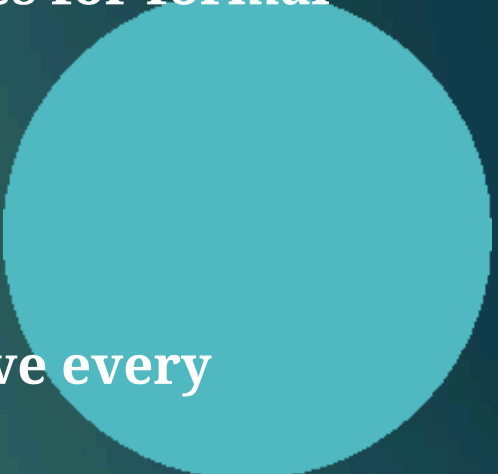
The review summary report is a single page form.

It is important to establish a follow-up procedure to ensure that items on the issues list have been properly corrected.

Review Guidelines



The following represents a minimum set of guidelines for formal technical reviews:

1. Review the product, not the producer.
 2. Set an agenda and maintain it.
 3. Limit debate and rebuttal.
 4. Enunciate problem areas, but don't attempt to solve every problem noted.
 5. Take written notes.
 6. Limit the number of participants and insist upon advance preparation.
 7. Develop a checklist for each product that is likely to be reviewed.
 8. Allocate resources and schedule time for FTRs.
 9. Conduct meaningful training for all reviewers.
 10. Review your early reviews.
- 

Sample-Driven Reviews (SDRs)

SDRs attempt to quantify those work products that are primary targets for full FTRs. To accomplish this the following steps are suggested...

- ✓ Software work product i
Fraction of each software work product a_i
No. Of faults in a_i is f_i .
- ✓ Gross estimation of no. Of faults within work product i is by multiplying f_i by $1/a_i$ ie f_i/a_i .
- ✓ Sort the work products in descending order to the gross estimation of the no. of faults in each.
- ✓ Focus available review resources on those work products that have the highest estimated number of faults.
The fraction of the work product that is sampled must

STATISTICAL SOFTWARE QUALITY ASSURANCE

For software, statistical quality assurance implies the following steps:

1. Information about software defects is collected and categorized.
2. An attempt is made to trace each defect to its underlying cause (e.g., non-conformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the "vital few").
4. Once the vital few causes have been identified, move to correct the problems .

The application of the statistical SQA and the pareto principle can be summarized in a single sentence: **spend your time focusing on things that really matter, but first be sure that you understand what really matters.**

Six Sigma for software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today.

An extremely high quality standard.

The Six Sigma methodology defines three core steps:

- ✓ **Define** customer requirements and deliverables and project goals via well-defined methods of customer communication
- ✓ **Measure** the existing process and its output to determine current quality performance (collect defect metrics)
- ✓ **Analyze** defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps.

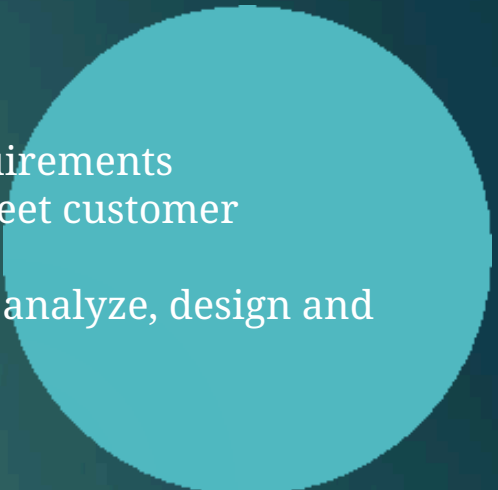
- ✓ **Improve** the process by eliminating the root causes of defects.
 - ✓ **Control** the process to ensure that future work does not reintroduce the causes of .
- These core and additional steps are sometimes referred to as the DMAIC (define,



If any organization is developing a software process (rather than improving and existing process), the core steps are augmented as follows:

- ✓ **Design** the process to
avoid the root causes of defects and to meet customer requirements
- ✓ **Verify** that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design and verify) method.



THE ISO 9000 QUALITY STANDARDS

A quality assurance system may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management.

ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered.

ISO 9001:2000 is the quality assurance standard that applies to software engineering.

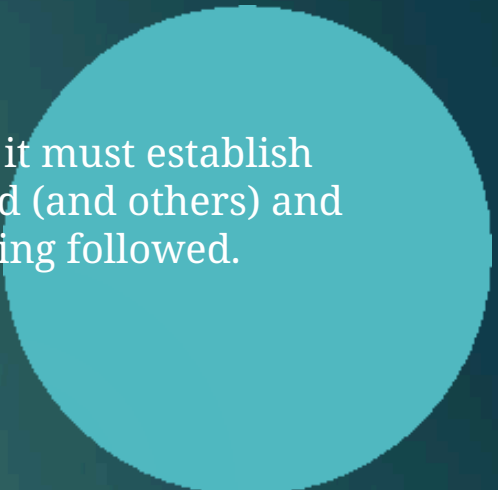
The standard contains 20 requirements that must be present for an effective quality assurance system.

Because the ISO 9001:2000 standard is applicable to all engineering disciplines, a special set of ISO guidelines have

been developed to help interpret the standard for use in the software process.

The requirements delineated by ISO 9001 address topics such as

- ✓ management responsibility,
- ✓ quality system, contract review,
- ✓ design control,
- ✓ document and data control,
- ✓ product identification and traceability,
- ✓ process control,
- ✓ inspection and testing,
- ✓ corrective and preventive action,

- 
- 
- ✓ training,
 - ✓ servicing and
 - ✓ statistical techniques.

In order for a software organization to become registered to ISO 9001, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed.

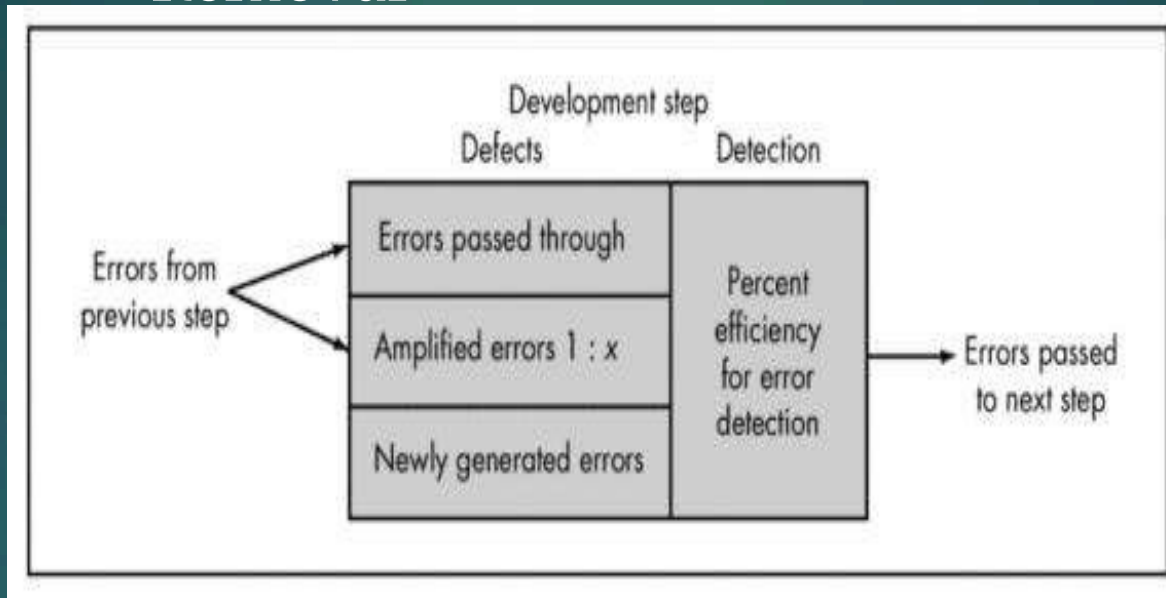
SOFTWARE RELIABILITY

- **Software reliability** is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time".
 - all software failures can be traced to design or implementation problems;
- A simple measure of reliability is meantime-between-failure (MTBF), where $MTBF = MTTF + MTTR$
- **Software availability** is the probability that a program is operating according to requirements at a given point in time and is defined as $Availability = [MTTF / (MTTF + MTTR)] 100\%$
- The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software.

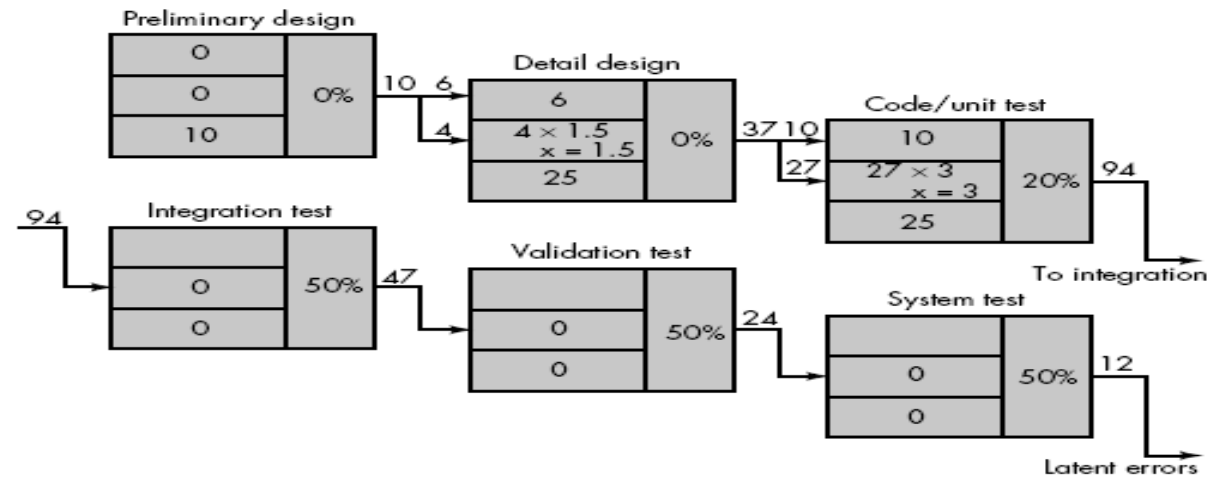
Software Safety

- Software safety is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail.
- If hazards can be identified early in the software engineering process, software design features can be specified that will either eliminate or control potential hazards.
- Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence.
- Analysis techniques such as fault tree analysis [VES81], real-time logic [JAN86], or petri net models [LEV87] can be used to predict the chain of events that can cause hazards and the probability that each of the events will occur to create the chain.
- Once hazards are identified and analyzed, safety-related requirements can be specified for the software. That is, the specification can contain a list of undesirable events and the desired system responses to these events.
- Although software reliability and software safety are closely related to one another, software reliability uses statistical analysis to determine the likelihood that a software failure will occur. However, the occurrence of a failure does not necessarily result in a hazard or mishap. Software safety examines the ways in which failures result in conditions that can lead to a mishap.

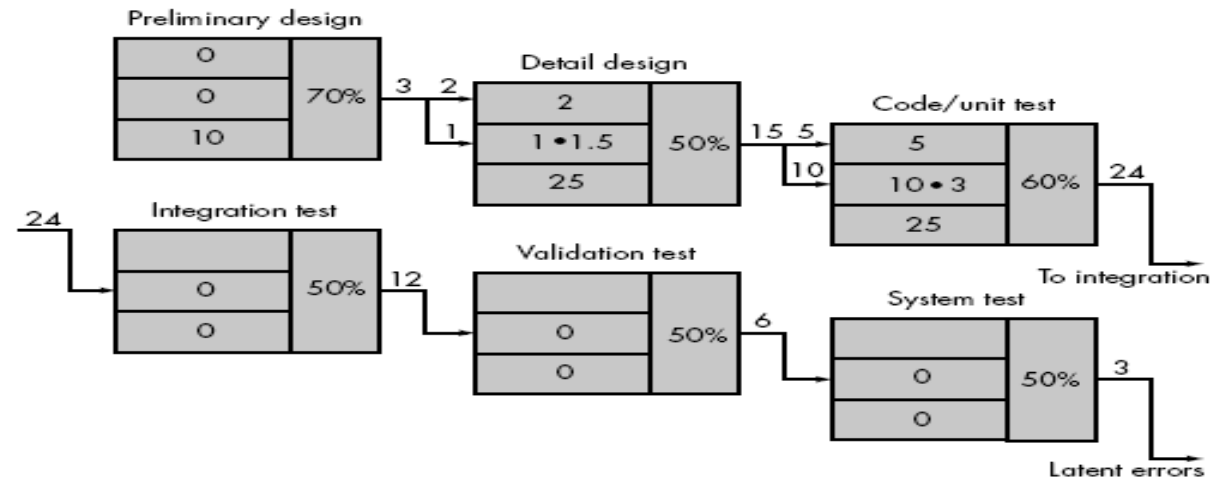
Defect Amplification and Removal



Defect amplification,
no reviews



Defect amplification,
reviews conducted



ISO 9000 Certification

- ISO (International Standards Organization) is a group of 63 countries established to plan and fosters standardization. ISO declared its 9000 series of standards in 1987.
- It serves as a reference for the contract between independent parties.
- The ISO 9000 standard determines the guidelines for maintaining a quality system.
- The ISO standard mainly addresses operational methods and organizational methods such as responsibilities, reporting, etc.
- ISO 9000 defines a set of guidelines for the production process and is not directed about the product.

ISO 9000 is a series of three standards:



Types of ISO 9000 Quality Standards

The ISO 9000 series of standards is based on the assumption that if a proper stage is followed for production, then good quality products are bound to follow automatically. The types of industries to which the various ISO standards apply are as follows.

1. **ISO 9001:** This standard applies to the organizations engaged in design, development, production, and servicing of goods. This is the standard that applies to most software development organizations.
2. **ISO 9002:** This standard applies to those organizations which do not design products but are only involved in the production. Examples of these category industries contain steel and car manufacturing industries that buy the product and plants designs from external sources and are engaged in only manufacturing those products. Therefore, ISO 9002 does not apply to software development organizations.
3. **ISO 9003:** This standard applies to organizations that are involved only in the

How to get ISO 9000 Certification

An organization determines to obtain ISO 9000 certification applies to ISO registrar office for registration.

